

MyCaTDD · METHOD SERIES

CaTDD in Action

Comment-alive Test-Driven Development · A Field Guide for the LLM Era

Method author: EnigmaWU

Contents

Chapter 1 · defineConcepts	5
What Is CaTDD?	5
Category-Specific Method Prompts	6
The Four-Layer Architecture	8
The Feedback Loop	11
The Design Skeleton Contract	11
The Priority Framework	12
Context-Specific Priority Adjustments	14
Risk-Driven Priority Adjustment	16
The US/AC/TC Contract	16
The TDD Red→Green Cycle (CaTDD Style)	18
Quality Gates	19
The Fast-Fail Six	20
Test Organization Strategies	21
The Implementation Tracking Template	22
Design as a Living Contract	23
From Concepts to Action	23
Chapter 2 · chatVibeCoding	25
What Is Vibe Coding?	25
The Two Modes of CaTDD Interaction	25
When to Use VibeCoding	26
When to Switch to SpecCoding	27
How VibeCoding Works with methodPrompts	27
The VibeCoding Workflow	28
VibeCoding vs SpecCoding: The Developer's Decision	30
The Method Prompt as Chat Context	31
The Model Tier Strategy for VibeCoding	32
Common VibeCoding Patterns	32
The Two Design Stages in VibeCoding	33
VibeCoding with Real-Time Feedback	34
The CodeAgent's Role in VibeCoding	35
VibeCoding Anti-Patterns	35
From VibeCoding to SpecCoding: The Bridge	36
Chapter 3 · callSlashCommands	37
What Are Slash Commands?	37
The Three Command Families	37
The Universal Command Template	39

The Four Command Flows and the HarnessKits Kit	40
P0 FuncTestsFlow — The First Flow to Learn	40
The RED→GREEN Discipline in Slash Commands	44
P1-DesignTestsFlow and P2-QualityTestsFlow	45
Px-SpecFlow — The Master Orchestration Flow	47
Model Tier Guidance for Commands	55
Chain-of-Thought Patterns in Commands	56
SpecFlow Artifacts	57
The Command Template as a Contract	58
Conflict Guard Rules	59
Execution Modes and the ONE-MORE-THING Stop Rule	59
Command Invocation in Practice	60
When to Use Which Flow	61
The Subagent Pattern	61
From Commands to Automation	62
Chapter 4 · asyncCodeAgent	63
What Is a Code Agent?	63
The Two Code Agents	63
The CaTDD-Native Contract	64
The Agent Workflow Checklist	65
Agent Troubleshooting: The Six Common Issues	68
The Agent's "Do and Don't" Contract	69
Trace Collection and Reflection	70
utCodeAgentCLI Architecture	70
specCodeAgentCLI Architecture	74
Async Code Agent Workflow	75
The Agent's Relationship with Developer Checkpoints	76
Agent Skills vs Code Agents	77
The Future of Code Agents	77
From Automation to Integration	77
Chapter 5 · applyClassicSWE	79
The Knowledge Book of Software Engineering	79
The LLM Era Changes Everything — and Nothing	79
TDD in the LLM Era	80
BDD in the LLM Era	81
DDD in the LLM Era	83
CaTDD: The Synthesis of TDD + BDD + DDD	85
The LLM as a Knowledge Book Reader	86
BDD → CaTDD: From Feature Files to Comment Chains	86
DDD → CaTDD: From Domain Models to Comment Skeletons	87

The P0→P3 Framework as Strategic DDD	88
Living Documentation in the LLM Era	89
Applying the Knowledge Book: A Complete Example	89
The Knowledge Book Applied	91
Conclusion: The Triangle of Classic SWE	92
Chapter 6 · beyondXyzSpec	93
The Spec-Driven Development Landscape	93
GitHub Spec Kit — Document-Centric Spec-Driven Development	93
What CaTDD Adopted from Spec Kit	95
OpenSpec — Change-Centric Spec-Driven Development	95
CaTDD Px-SpecFlow — User-Story-Centered SpecCoding	96
The UT Flow — CaTDD's Embedded Verification Engine	99
Three-Way Comparison	101
Why CaTDD Is BEYOND	102

defineConcepts

devBooks/Action/01_defineConcepts.md

What Is CaTDD?

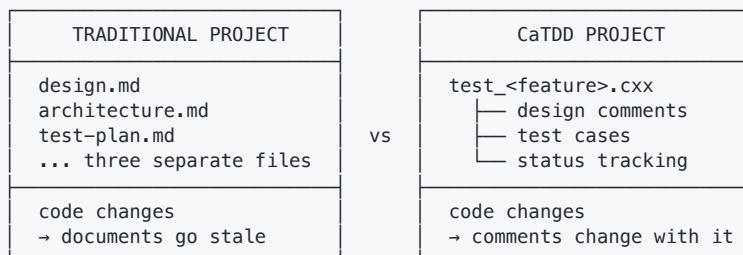
CaTDD stands for **Comment-alive Test-Driven Development**. It is a way to build software where the design lives inside the code file as structured comments, instead of in separate documents that slowly stop matching the code.

EnigmaWU created the method in 2023.10, and it has been used on real projects ever since.

The whole idea fits in one sentence:

Comments is Verification Design. LLM Generates Code. Iterate Forward Together.

Why it matters. In a normal project, design is written once, then the code changes, and the document quietly becomes wrong. Nobody notices until someone trusts the old document and makes a bad decision. CaTDD removes that failure mode: the design sits next to the code it describes, so you cannot change one without seeing the other.



What "Comment-alive" Means

"Comment-alive" is the part that makes CaTDD different. It has four dimensions.

- | | |
|---|---|
| 1. DESIGN LIVES IN THE FILE
no separate spec doc | test_feature.cxx
├─ // @[US-1] ...
├─ // @[AC-1] ...
└─ // @[TC-1] ... |
| 2. COMMENTS MOVE WITH CODE | code refactored → comments updated
in the same edit |
| 3. MACHINES CAN READ THEM | an LLM sees @[US] @[AC] @[TC] markers
and knows what to build |
| 4. THE CHAIN NEVER BREAKS | human need → US → AC → TC → assertion |

- Design detail lives in the test file.** You do not maintain a spec document, an architecture document, and a test plan in three places. The verification design sits where the code sits.
- Comments evolve with the code.** When you refactor production code, you update the comments in the same file and the same commit. Intent and implementation stay in sync by construction, not by discipline.

3. **Comments are first-class artifacts that LLMs can parse and update.** An LLM reading a CaTDD test file sees `@[US]`, `@[AC]`, and `@[TC]` markers it can trace, validate, and generate code from. That is why CaTDD is LLM-friendly TDD.
4. **US/AC/TC connects human intent to machine checks.** A User Story states business value, Acceptance Criteria make it testable, and Test Cases turn it into concrete assertions. The chain runs from human need to machine verification without a gap.

The "TDD" Part

CaTDD keeps the classic Red → Green → Refactor cycle. What it adds is structure before the code: you write the design as comments first.

Traditional TDD	CaTDD
Write a failing test	Write the structured comment design first (US/AC/TC), then the failing test
Implement to pass	Implement the smallest production code that passes
Refactor	Refactor test and production code together
Repeat	Move the TC status (○ → ● → ●), then take the next test case
Design is not captured	Design lives in the same file as the tests

Key point — CaTDD is not a replacement for TDD. It is TDD with design structure, traceability, and machine-readability built in.

Example — the same fix, two ways. Suppose `I0C_postEVT` starts failing when the queue is full.

Without CaTDD

1. bug reported
2. search the code
3. guess the intent
4. patch, hope nothing breaks
5. no record of why

With CaTDD

1. read the design comment:

```
// @[AC-2] WHEN queue is full
  THEN return immediately
```
2. the intent is already written down
3. write the test that proves it
4. patch until it is GREEN
5. the reason stays in the file

The second path is faster because the why was never lost.

Category-Specific Method Prompts

CaTDD does not treat every test as the same kind of test. It defines **15 test categories**, and gives each one its own method prompt: `CaTDD_methodPrompt4Cat-*.md`. A CodeAgent reads the prompt for the category it is working in, so it uses the right vocabulary, the right test points, and the right constraints.

15 TEST CATEGORIES

P0 functional	P1 design	P2 quality	P3 addons
Typical Edge Misuse Fault	State Capability Interaction Concurrency	Performance Robust Compatibility Configuration Diagnosis Security	Demo/ Example

Category	Prompt File	What it is for
Typical	<code>CaTDD_methodPrompt4Cat-Typical.md</code>	The happy path: one behavior per test, at most 3 assertions
Edge	<code>CaTDD_methodPrompt4Cat-Edge.md</code>	Boundaries, limits, and modes; one edge per test
Misuse	<code>CaTDD_methodPrompt4Cat-Misuse.md</code>	Wrong API usage: bad order, invalid parameters, double init
Fault	<code>CaTDD_methodPrompt4Cat-Fault.md</code>	Outside failures and recovery: network down, disk full
State	<code>CaTDD_methodPrompt4Cat-State.md</code>	Lifecycle and state machines; invalid transitions
Capability	<code>CaTDD_methodPrompt4Cat-Capability.md</code>	Designed limits and responsibilities
Interaction	<code>CaTDD_methodPrompt4Cat-Interaction.md</code>	Order between collaborators and handoff contracts
Concurrency	<code>CaTDD_methodPrompt4Cat-Concurrency.md</code>	Threads, races, deadlocks, ordering
Performance	<code>CaTDD_methodPrompt4Cat-Performance.md</code>	SLOs, latency, throughput, resource budgets
Robust	<code>CaTDD_methodPrompt4Cat-Robust.md</code>	Stress, repetition, long runs, degraded conditions
Compatibility	<code>CaTDD_methodPrompt4Cat-Compatibility.md</code>	Versions, platforms, protocols, schemas, toolchains
Configuration	<code>CaTDD_methodPrompt4Cat-Configuration.md</code>	Defaults, precedence, feature flags, bad config
Diagnosis	<code>CaTDD_methodPrompt4Cat-Diagnosis.md</code>	Observability, debuggability, useful failure evidence
Security	<code>CaTDD_methodPrompt4Cat-Security.md</code>	Threats, trust boundaries, secrets, protection
Demo/Example	<code>CaTDD_methodPrompt4Cat-DemoExample.md</code>	Tutorials and documented examples

Every category prompt answers the same questions, so you always know where to look:

Section	The question it answers
Position	Where does this category sit in the priority framework?
Use When	When is this the right category?
Do Not Use When	When should this scenario move to another category?

Section	The question it answers
TestPointsInMind	What should I probe for in this category?
Design Skeleton	What is the contract shape for this category?
Checklist	How do I know the work is good enough?

Ten of the fifteen files (the four P0 categories plus State, Capability, Interaction, Concurrency, Diagnosis, and Security) also carry **Design Focus**, **US/AC/TC Pattern**, **Naming Examples**, and **Common Mistakes**. The five remaining files (Performance, Robust, Compatibility, Configuration, Demo/Example) stay lean and route discovery through `TestPointsInMind`.

Why one file per category? Because "write a test" is not one skill. A boundary test, a race-condition test, and a security test fail in different ways and need different design habits. Splitting them keeps each habit sharp.

Example — category discipline prevents a weak test. A developer writes one test that fills a queue and checks five things at once: it is full, the return code is right, no event was queued, the counter did not change, and the next call still works.

One test, five checks	Split by category
Typical? Edge? Misuse? Fault? When it fails, you must re-read the whole test to learn why	Edge : fill to capacity, one more Misuse : call again after full Typical: normal post still works

Four small tests tell you which rule broke. One large test only tells you something broke.

The Four-Layer Architecture

MyCaTDD is built in four layers. Each layer has one job, and each layer depends only on the one above it.

- | | |
|--------------------------------|---|
| [1] <code>methodPrompts</code> | – where the method is defined
CaTDD semantics, category meaning, US/AC/TC
skeleton rules, status discipline, templates |
| [2] <code>slashCommands</code> | – how the method is invoked
Portable UT_*, SPEC_*, HARNESS_* commands
input/output handoff, tool-neutral execution |
| [3] <code>codeAgents</code> | – who executes it
<code>utCodeAgentCLI</code> (unit testing)
<code>specCodeAgentCLI</code> (SpecCoding lifecycle)
planning, trace collection, reflection loops |
| [4] <code>agentSkills</code> | – how it is packaged to go
Comment-alive TDD skill, SpecCoding skill
packaged for Copilot, Cline, and other agents |

A simple way to remember the layers:

<code>methodPrompts</code>	=	the RULES
<code>slashCommands</code>	=	the BUTTONS
<code>codeAgents</code>	=	the WORKERS
<code>agentSkills</code>	=	the SHIPPING BOX

Layer 1: methodPrompts — The Source of Truth

`methodPrompts/` is the official definition of CaTDD. Everything else is derived from it. If the method changes, this layer changes first.

What is in it	What it gives you
<code>CaTDD_methodPrompt.md</code>	The master entry point and the stable CaTDD contract: skeleton shape, category semantics, mandatory traceability, test-point discovery, workflow, and the default execution order
<code>CaTDD_methodPrompt-*.md</code>	Subtopic prompts with the deeper material: category semantics, test-point discovery, workflow, test structure, file naming, agent workflow, troubleshooting, and worked examples
<code>CaTDD_methodPrompt4Cat-*.md</code>	One deep-dive prompt per test category, so a CodeAgent can apply the right rules inside that category
<code>CaTDD_designAndImplTemplate.cxx/.ts/.py/.go</code>	Working file templates for C++, TypeScript, Python, and Go. They all follow the same OVERVIEW → DESIGN → IMPLEMENTATION → TODO structure. No language is required by the method
<code>README_UserGuide.md</code> , <code>README_UserGuide_ZH.md</code>	Standalone guides that explain how to use the prompts, who uses them, when to apply them, and where to put them

Why a "source of truth" matters. When two documents disagree, someone must decide which one wins. CaTDD makes that decision once: `methodPrompts` wins. Every other layer is a projection of it.

Layer 2: slashCommands — The Commandization Layer

`slashCommands/` turns the stable steps of the method into commands you can invoke. It is the bridge between method semantics and the tool you happen to use: Copilot, Cline, Continue, or `utCodeAgentCLI`.

The rule is simple:

```
methodPrompts  —defines→  what must happen
slashCommands  —wraps→    how to ask for it
```

```
if a command and the method disagree:
    methodPrompts wins
```

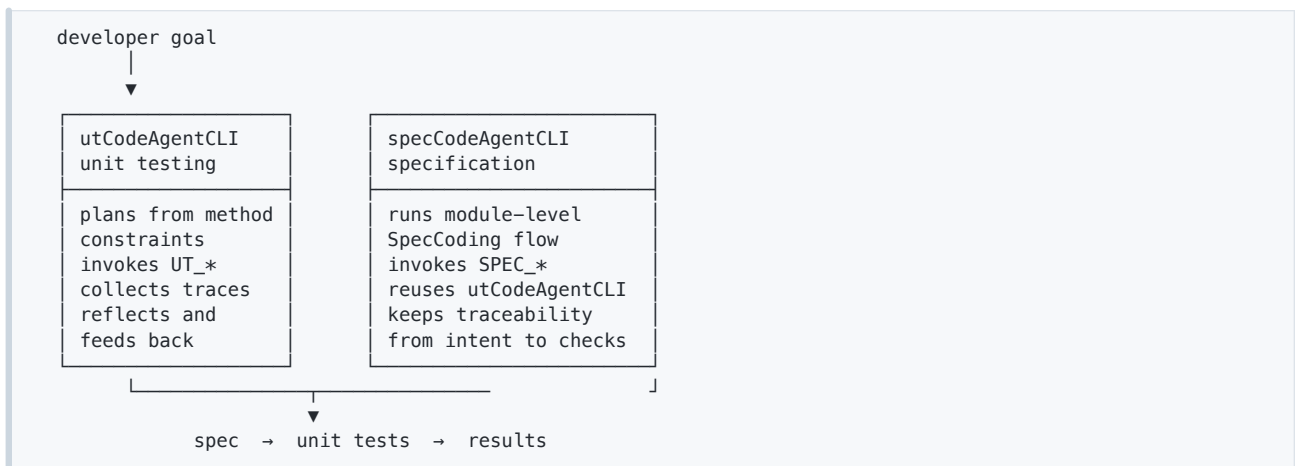
The layer contains:

Part	Location	What it is
Flow documents	<code>flows/</code>	P0-FuncTestsFlow, P1-DesignTestsFlow, P2-QualityTestsFlow, Px-SpecFlow. Each flow is a repeatable command sequence with entry points, gates, and loop-back paths
Kit documents	<code>kits/</code>	Px-HarnessKits groups the <code>HARNESS_*</code> tool points that maintain CaTDD source, adapters, execution, diagnostics, and patch-back safety, without forcing a strict lifecycle

Part	Location	What it is
Command templates	UT_slashCommandTemplate.md, SPEC_slashCommandTemplate.md	The shared shape every command follows: Command Header / CoT Pattern / WHO / WHAT / WHEN / WHERE / WHY / HOW / Input Contract / Output Contract / CodeAgent Compatibility. The SPEC template adds a Subagent Recommendation
Command files	commands/	The three families UT_*, SPEC_*, and HARNESS_*, grouped by flow or kit. Each file tells a CodeAgent what to read, what to produce, what to preserve, and what to do next

Layer 3: codeAgents — The Execution Layer

`codeAgents/` describes the CaTDD-native agents. Today these are design and architecture specifications; the runnable implementations are in progress. Two agents are defined.



- **utCodeAgentCLI** takes a developer goal, plans the work from CaTDD method constraints, invokes standardized slash command steps, collects traces, reflects on the outcome, and feeds reusable patterns back into the method and command layers. It preserves the design skeleton contract, US/AC/TC traceability, category classification, and the RED/GREEN status discipline.
- **specCodeAgentCLI** orchestrates module-level SpecCoding from input to output. It is built on Px-SpecFlow, reuses the unit-testing strength of `utCodeAgentCLI`, and organizes scenario-level verification from spec intent to executable checks. It keeps traceability between the spec flow, the validation checkpoints, and the implementation outcome.

Layer 4: agentSkills — The Packaging Layer

`agentSkills/` packages CaTDD and SpecCoding as skills any CodeAgent can pick up. Two are authored:

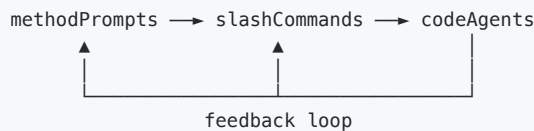
1. **comment-alive-test-driven-development** — the CaTDD testing method as a skill: WHO / WHAT / WHEN / WHERE / WHY sections, phase-by-phase instructions, input and output contracts, constraints, and validation rules. When a developer says "use CaTDD", this skill gives the agent everything it needs.
2. **user-story-centered-spec-coding** — the SpecCoding lifecycle as a skill. It covers the full story path `pendingNews` → `todoUS` → `doingUS` → `doneUS`, with `abortUS` preserving unsafe active stories for later analysis, and CaTDD as the default unit-testing method.

The packaging script `makeSkill.sh` produces self-contained distributable packages by copying references

from `methodPrompts/` and `slashCommands/`. The authored source is the durable asset; the packages are build output.

The Feedback Loop

The four layers are not a one-way pipe. Information flows back up.



Direction	What moves
<code>methodPrompts</code> → <code>slashCommands</code>	Stable method steps become slash commands
<code>methodPrompts</code> → <code>codeAgents</code>	Method semantics constrain what agents may do
<code>slashCommands</code> → <code>codeAgents</code>	Agents invoke standardized command steps
<code>slashCommands</code> → <code>methodPrompts</code>	Running commands exposes gaps in the method
<code>codeAgents</code> → <code>slashCommands</code>	Reflection finds reusable command patterns
<code>codeAgents</code> → <code>methodPrompts</code>	Real execution experience improves the methodology

Why the loop matters. A method that cannot learn from being used turns into folklore. Every downstream layer both consumes and improves the layers above it, so CaTDD evolves from real usage rather than from theory.

Example — a gap found by using it. A CodeAgent repeatedly stops mid-flow because the command it just ran never says which command comes next.

```

today      : command ends → agent guesses → wrong next step → rework
feedback   : "Output Contract must name the next command"
result     : the template is updated once, every command benefits
  
```

One fix in `slashCommands/` removes the same confusion from every future run.

The Design Skeleton Contract

In CaTDD, "design" is not a UML diagram or a Word file. It is a **reusable comment skeleton inside the test file**.

Every skeleton is organized by two labels:

- **Class** — the priority family: `P0 Functional`, `P1 Design`, `P2 Quality`, `P3 Addons`
- **Category** — the verification angle: `Typical`, `Edge`, `Misuse`, `Fault`, `State`, `Capability`, `Interaction`, `Concurrency`, `Performance`, `Robust`, `Compatibility`, `Configuration`, `Diagnosis`, `Security`, `Demo/Example`

Every skeleton keeps this minimum shape:

```
//=====
// [Class] / [Category] Design Skeleton
//=====
// @[SUT]: [Declared SUT matching file overview]
// @[TestLevel]: UnitTesting (or SysTesting / UserTesting)
// @[Class]: P0 Functional / ValidFunc
// @[Category]: Typical
// @[Intent]: What this category proves for this component
// @[UseWhen]: When this category applies
// @[AvoidWhen]: When to move the scenario to another category
// @[US]: User Story IDs covered by this category
// @[AC]: Acceptance Criteria IDs covered by this category
// @[TC]: Test Cases, status, and expected TDD next action
//=====
```

Read the skeleton as a set of promises:

@[SUT]	what is under test	→ stops scope creep
@[TestLevel]	unit / system / user	→ sets how deep to test
@[Intent]	what this proves	→ the one-sentence "why"
@[UseWhen]	when it applies	→ protects the category
@[AvoidWhen]	when to move elsewhere	→ prevents misclassification
@[US] [AC]	what requirement it serves	→ traceability upward
@[TC]	the cases and their status	→ the work list

This skeleton is the contract that developers and CodeAgents both honor. A developer fills it with verification intent. A CodeAgent reads it and writes test code that satisfies that intent. Both sides update it as the code evolves.

What the contract buys you. A new engineer, or a new LLM session, can open the file and answer "what does this prove, for whom, and what is still missing?" without asking anyone.

Example — an empty skeleton beats an empty promise. A reviewer asks "is the full-queue path covered?" Two possible answers:

file with a skeleton	file without one
// @[Category]: Edge	(nothing)
// @[Intent]: queue boundary behavior	
// @[TC]: TC-1 ○ TODO	"I think so?"
"not yet – TC-1 is still TODO"	(search code for 20 minutes)

The skeleton turns an unanswerable question into a status line.

The Priority Framework

Not every test is equally urgent. CaTDD sorts tests into four priority levels and gives a default order to run them in.

default order

```
P0 Functional → P1 Design → P2 Quality → P3 Addons → release
"does it work?" "is the design right?" "is it fast, stable, safe, compatible?" "can someone learn from it?"
```

Level	Question it answers	Categories
P0 Functional	Does it work, and does it fail safely?	Typical, Edge, Misuse, Fault

Level	Question it answers	Categories
P1 Design	Are the architectural decisions real?	State, Capability, Interaction, Concurrency
P2 Quality	Does it hold up under real conditions?	Performance, Robust, Compatibility, Configuration, Diagnosis, Security
P3 Addons	Can others learn to use it?	Demo/Example

P0: Functional Testing

P0 has two halves. One proves the system works; the other proves it fails in a controlled way.

```

P0 Functional = ValidFunc + InvalidFunc

ValidFunc  — the system works correctly
| Typical  core happy path
| Edge     boundaries, limits, modes
|
InvalidFunc — the system fails gracefully
| Misuse   the caller used the API wrongly
| Fault    the outside world failed

```

ValidFunc — Proves the system works correctly

Category	What it covers	Example test points
Typical ★	Core happy-path workflows	Service registration, event publishing, command execution
Edge □	Boundary values, limits, modes	Min/max values, empty inputs, Block / NonBlock / Timeout modes

InvalidFunc — Proves the system fails gracefully

Category	What it covers	Example test points
Misuse ☹	Incorrect API usage	Wrong call sequence, double init, invalid parameters
Fault ▲	Outside failures and recovery	Network failure, disk full, process crash recovery

Why split ValidFunc and InvalidFunc? A function that passes every success case can still corrupt data on the first bad call. The two halves catch different bugs, and both belong to P0.

P1: Design-Oriented Testing

P1 tests the architectural decisions: state, limits, collaboration order, and concurrency. These are the choices that are expensive to change later.

Category	What it covers	Example test points
State ↻	Lifecycle and state machines	Init → Ready → Running → Stopped
Capability ★	Designed capacity and limits	Max connections, queue limits, pool exhaustion

Category	What it covers	Example test points
Interaction ➦	Collaborator order and handoffs	Orchestrator → plugin order, validate-before-dispatch, rollback after partial failure
Concurrency ▶	Threads and races	Parallel access, deadlock, lock-free validation

Example — Design versus functional. Registering a plugin is a Typical test. The rule that all plugins must be validated before any plugin is dispatched is an Interaction test. The first checks a feature; the second checks an architectural promise.

P2: Quality-Oriented Testing

P2 covers non-functional requirements: speed, stability, compatibility, configuration, diagnosability, and security.

Category	What it covers	Example test points
Performance »	Speed, throughput, resources	Latency benchmarks, memory growth
Robust ■	Stress, repetition, long runs	1000x repetition, 24-hour soak
Compatibility ↻	Platforms, versions	Windows / Linux / macOS, API version compatibility
Configuration ≡	Settings and deployment	Debug versus Release, feature flags, environment variables
Diagnosis ○	Observability and evidence	Correlation IDs in logs, actionable errors, health output
Security ♦	Protection under threat	AuthN / authZ denial, secret redaction, trust boundaries, injection defense

P3: Addons Testing

Category	What it covers
Demo/Example 📄	End-to-end demonstrations, tutorials, and best-practice illustrations

Default Test Order

```

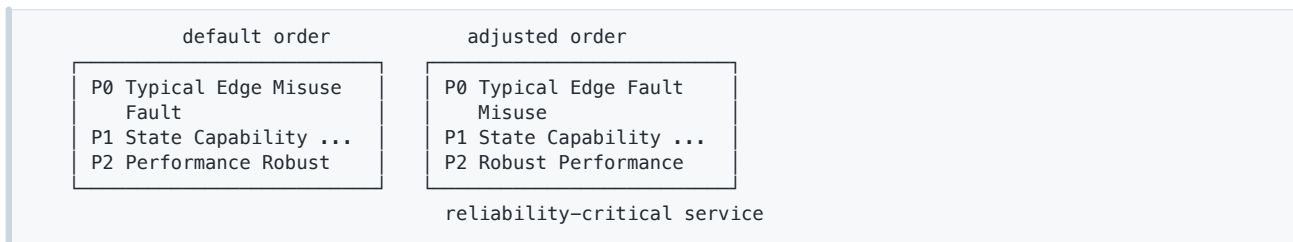
P0: Typical → Edge → Misuse → Fault
P1: State → Capability → Interaction → Concurrency
P2: Performance → Robust → Compatibility → Configuration → Diagnosis → Security
P3: Demo/Example

```

This is the default, not a law. The rest of this section explains when to move a category earlier.

Context-Specific Priority Adjustments

Different products carry different risk. A device driver and a batch report generator should not be tested in the same order. CaTDD lists adjustment rules for common situations.



New Public API

P0: Typical → Edge → Misuse → Fault (complete P0 thoroughly)
P1: State → Capability → Interaction → Concurrency
P2: Performance

Why: the API contract must be correct before anything advanced is worth testing.

Stateful/FSM-Heavy Component

P0: Typical → Edge (basic functional first)
P1: State (promoted early) → Capability → Interaction → Concurrency
P0: Misuse → Fault (finish functional coverage)
P2: Performance → Robust

Why: the state machine is the architectural core. Test it right after basic function works.

Reliability-Critical Service

P0: Typical → Edge → Fault (promoted) → Misuse
P1: State → Capability → Interaction → Concurrency
P2: Robust (promoted) → Performance → Compatibility

Why: when downtime is expensive, error handling and stability outrank feature breadth.

High-Performance System (SLOs)

P0: Typical → Edge → Misuse
P2: Performance (promoted) → Robust
P1: State → Capability → Interaction → Concurrency
P0: Fault (finish functional coverage)

Why: when a latency budget is a design constraint, measuring it early prevents a redesign later.

Highly Concurrent Design

P0: Typical → Edge → Misuse
P1: Concurrency (promoted first) → State → Capability → Interaction
P0: Fault (finish functional coverage)
P2: Performance → Robust

Why: thread safety is the foundation everything else sits on.

Data Processing Pipeline

P0: Typical → Edge → Fault → Misuse
P2: Performance (promoted) → Robust (promoted)
P1: State → Capability → Interaction → Concurrency

Why: data integrity and throughput are the quality attributes that define the product.

Common mistake — promoting a category and then never finishing P0. A promoted category is a temporary reorder. The rule that P0 completes before P1 still holds.

Risk-Driven Priority Adjustment

When a component fits none of the situations above, score the risk instead of guessing.

Risk Score = Impact × Likelihood × Uncertainty

Impact	1 (low)	→	3 (critical)
Likelihood	1 (rare)	→	3 (frequent)
Uncertainty	1 (known)	→	3 (unknown)

maximum score: 27

Then move the category:

Score	Action
≥ 18	Move it immediately after Edge
12 – 17	Move it up 2 positions
9 – 11	Move it up 1 position
≤ 8	Keep the default position

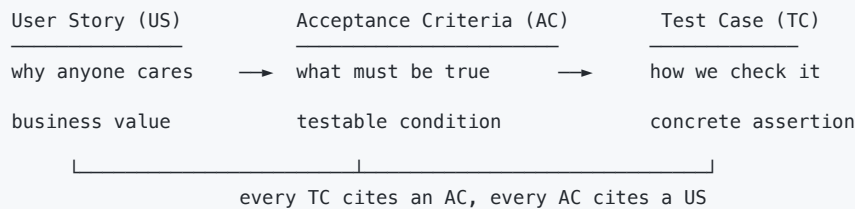
Example — two categories, two very different scores.

Concurrency in a multi-threaded queue			
Impact	3	data corruption	
Likelihood	3	many threads touch the queue	
Uncertainty	3	interactions are hard to reason about	
Score	27	→ test right after Edge	
Performance in a batch processor			
Impact	2	slower, but still correct	
Likelihood	2	depends on the load	
Uncertainty	2	some benchmarks already exist	
Score	8	→ keep the default position	

Why multiply instead of add? Multiplying keeps a category low when any factor is low. A rare failure with huge impact and unknown behavior still scores 18 and moves up; a harmless and well-understood one does not.

The US/AC/TC Contract

CaTDD connects a human need to a machine check with three linked artifacts. Each one answers a different question, and each one points at the next.



User Story (US) Template

```
US-n: As a [specific role/personal],
      I want [specific capability or feature],
      So that [concrete business value or benefit].
```

A module usually carries 2–5 User Stories. Each one should represent a distinct piece of user value, not a piece of code.

Example from the IOC Event System:

```
US-1: As an event producer in high-load scenarios,
      I want to post events without blocking when the queue is full,
      So that my application remains responsive under load.
```

Test for a good US: can a product owner read it and say "yes, that is what I asked for"? If only an engineer can judge it, it is too technical.

Acceptance Criteria (AC) Template

```
AC-n: GIVEN [initial context and preconditions],
      WHEN [specific trigger, action, or event],
      THEN [expected observable outcome or behavior],
      AND [additional expected outcomes if any].
```

Define 1–4 ACs per User Story. Each AC must be verifiable on its own.

Example:

```
AC-1: GIVEN an event producer calling IOC_postEVT_inConlesMode,
      WHEN IOC's EvtDescQueue is full in ASyncMode by blocking consumer,
      THEN producer returns immediately without waiting,
      AND returns IOC_RESULT_TOO_MANY_QUEUEING_EVTDESC,
      AND the event is not queued for processing.
```

This is one sentence, and it already tells the tester three separate facts to check.

Test Case (TC) Template

```
[@AC-n,US-n]
TC-n:
  @[Name]: verifyBehavior_byCondition_expectResult
  @[Purpose]: Why this test matters and what it validates
  @[Brief]: What the test does in simple terms
  @[Steps]: Detailed execution steps (optional, for complex tests)
  @[Expect]: How to verify success
  @[Notes]: Additional context, gotchas, or dependencies
```

The name follows one pattern everywhere, so a test name reads like a sentence:

```

verifyServiceRegistration_byValidName_expectSuccess
verifyEventPost_byFullQueue_expectNonBlockReturn
verifyCommandExec_byMultipleClients_expectIsolatedExecution
verifyStateTransition_byInvalidSequence_expectError
    └── behavior ───┬── condition ─┬── outcome ─┘

```

The full chain in one place

A single requirement is traceable end to end:

```

US-1  a producer must not block when the queue is full
├── AC-1  returns immediately, with IOC_RESULT_TOO_MANY_QUEUING_EVTDESC
│   ├── TC-1  verifyEventPost_byFullQueue_expectNonBlockReturn
│   ├── TC-2  verifyEventPost_byFullQueue_expectNotQueued
│   └── TC-3  verifyEventPost_byFullQueue_expectImmediateReturn
├── code    the queue check in IOC_postEVT_inConsolesMode
└── commit  "AC-1: non-blocking post when queue is full"

```

One AC can produce several TCs, because "returns immediately", "returns this code", and "does not queue the event" are three different claims.

Example — the same requirement, badly written. Compare these two ways to record the same intent:

Weak AC

"handle full queue properly"

- what does "properly" mean?
- how would a test fail?

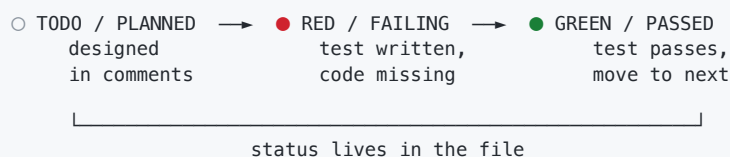
Strong AC

GIVEN the queue is full
 WHEN the producer posts
 THEN it returns immediately
 AND returns
 IOC_RESULT_TOO_MANY_QUEUING_EVTDESC
 AND the event is not queued

Only the right-hand version can be turned into a test without asking a question.

The TDD Red→Green Cycle (CaTDD Style)

CaTDD keeps the classic cycle and adds an explicit status for every test case.



Status	Meaning	What you do next
○ TODO	The test is designed in comments but not written	Write the test code
● RED	The test exists and fails because production code is missing	Implement the smallest production code that passes
● GREEN	The test passes	Move to the next test case

Never skip RED. If a test is green the first time you run it, either the code already exists or the test is not checking anything. Both are worth knowing before you trust it.

The 4-Phase Test Structure

Every test is written in the same four phases, in the same order.

```
TEST(CategoryName, verifyBehavior_byCondition_expectResult) {
    //===SETUP===
    // Create resources and set up the preconditions.

    //===BEHAVIOR===
    printf("▶ BEHAVIOR: verifyBehavior_byCondition_expectResult\n");
    // Perform the single action this test is about.

    //===VERIFY===
    // Check the outcome. Keep at most three key assertions.

    //===CLEANUP===
    // Release resources and restore the state.
}
```

```

SETUP  → BEHAVIOR → VERIFY → CLEANUP
  |         |         |         |
preconditions one at most no leaks
are explicit  action 3 checks for the next test

```

Example — why three assertions is a real limit. One test that checks return code, queue depth, counter value, log output, and thread state fails with a wall of output. You then spend ten minutes finding out which claim broke. Three focused tests fail with one clear message each.

Why at most 3 assertions per test?

- You can see immediately **what** failed
- Tests stay independent of each other
- Each test has one clear purpose
- Need more checks? Write another test case

Quality Gates

Between priority levels there are gates. A gate is not paperwork; it is a checkpoint that stops a known-bad state from moving forward.

```

P0 Functional → [ GATE P0 ] → P1 Design
                |
P1 Design     → [ GATE P1 ] → P2 Quality
                |
P2 Quality    → [ GATE P2 ] → release
                |
P3 Addons     → [ GATE P3 ] → documentation complete

```

Gate P0: Before Leaving Functional Testing

Must complete: `ValidFunc(Typical + Edge) + InvalidFunc(Misuse + Fault)`

- All Typical tests GREEN (80–90% core workflow coverage)
- All Edge tests GREEN (boundaries and limits validated)
- All Misuse tests GREEN or documented

- All Fault tests GREEN or documented
- No critical correctness bugs
- **Fast-Fail Six** tests all passing
- Basic memory and resource leak checks clean

Exit criteria: the API contract is tested on both the success and the failure path.

Gate P1: Before Quality-Oriented Testing

- State tests GREEN (if the component is stateful)
- Capability tests GREEN (limits are characterized)
- Interaction tests GREEN (if sequence, collaboration, or handoff rules exist)
- Concurrency tests GREEN (if the component is multi-threaded)
- No known deadlocks or races
- ThreadSanitizer / AddressSanitizer clean
- The architecture matches the design requirements

Gate P2: Before Release

- Performance tests GREEN (SLOs met, if defined)
- Robust tests GREEN (stress and soak tests pass)
- Compatibility tests GREEN (if multi-platform)
- Configuration tests GREEN (if configurable)
- Diagnosis tests GREEN (when observability or explainability is required)
- Security tests GREEN (when threat, policy, or protection requirements exist)
- Production readiness criteria met

Optional Gate P3: Documentation Complete

- Demo/Example tests GREEN
- Tutorial code validated
- Best practices documented

Common mistake — treating a gate as a formality because the tests are green. A gate asks "is the claim covered?", not "did the run pass?" Fifty green tests that never touch the failure path still fail Gate P0.

The Fast-Fail Six

Six cheap tests catch the majority of API-level mistakes. Run them early, before detailed test design.

1 null / empty input	4 illegal call sequence
2 zero / negative timeout	5 buffer full and buffer empty
3 duplicate registration	6 double close / re-init

minutes to write → hours of debugging saved

#	Test	The question it asks
1	Null / Empty Input	Does every API reject <code>NULL</code> and empty strings with a proper error code?
2	Zero / Negative Timeout	What happens with timeout <code>0</code> , <code>-1</code> , or <code>UINT_MAX</code> ?
3	Duplicate Registration	Does double registration return <code>ALREADY_EXISTS</code> ?
4	Illegal Call Sequence	What happens if you call before init, or after cleanup?
5	Buffer Full / Empty	Fill to capacity and try one more. Empty it and try one more.
6	Double Close / Re-Init	Is the operation idempotent, or does it return a proper error?

Example — the failure these tests prevent.

```
// Looks harmless, ships a segfault
handle = IOC_create();
IOC_destroy(handle);
IOC_destroy(handle);           // ← second close, no check anywhere
```

Fast-Fail Test 6 would have caught this at design time:

```
IOC_destroy(handle);           → OK
IOC_destroy(handle);           → must return an error, not crash
```

Six small tests, written once, protect every caller of the API.

Test Organization Strategies

CaTDD supports two layouts. Pick by project size, not by preference.

SINGLE FILE	MULTI-FILE
<pre>test_queue.cxx ├── Typical suite ├── Edge suite ├── Misuse suite └── Fault suite</pre>	<pre>test_queue_funcValidTypical.cxx test_queue_funcValidEdge.cxx test_queue_funcInvalidMisuse.cxx test_queue_funcInvalidFault.cxx test_queue_designState.cxx test_queue_qualityPerformance.cxx └── one file per category</pre>
< 50 tests simplest to start	grows without collisions clearer ownership per category

Single File Strategy (simpler projects, under 50 tests)

Keep every test for a component in one file and organize it with TEST suites per category. Good for small and medium modules, and the fastest way to start.

Multi-File Strategy (larger projects)

One file per category, using the canonical name `test_{feature}_{category}.<ext>`. `{feature}` is a stable `lower_snake_case` usage slice; `{category}` is a fixed CaTDD token.

File	Category
<code>test_{feature}_freelyDrafts.cxx</code>	Exploration and idea capture (Stage-0 drafts)
<code>test_{feature}_funcValidTypical.cxx</code>	Core workflows

File	Category
<code>test_{feature}_funcValidEdge.cxx</code>	Edge cases, boundaries, limits
<code>test_{feature}_funcInvalidMisuse.cxx</code>	API abuse patterns
<code>test_{feature}_funcInvalidFault.cxx</code>	Error handling and recovery
<code>test_{feature}_designState.cxx</code>	State transitions
<code>test_{feature}_designCapability.cxx</code>	Capability limits and responsibilities
<code>test_{feature}_designInteraction.cxx</code>	Collaborator sequence and handoffs
<code>test_{feature}_designConcurrency.cxx</code>	Thread safety
<code>test_{feature}_qualityPerformance.cxx</code>	SLOs and resource budgets
<code>test_{feature}_qualityRobust.cxx</code>	Stress and long-running stability
<code>test_{feature}_qualityCompatibility.cxx</code>	Version, platform, and toolchain compatibility
<code>test_{feature}_qualityConfiguration.cxx</code>	Feature flags and environment variations
<code>test_{feature}_qualityDiagnosis.cxx</code>	Observability and failure evidence
<code>test_{feature}_qualitySecurity.cxx</code>	Protection properties under threat
<code>test_{feature}_addonDemoExample.cxx</code>	Tutorials and documented examples

Mature test points move out of the freely-drafts file into their category file. Each feature keeps one file per canonical token; a category with nothing to test keeps `@[NoTestPoints] : <reason>`, which is a living decision rather than an empty file.

Why keep a file for a category with no tests? Because "we decided this does not apply" and "we forgot about it" look identical in a repository. The `@[NoTestPoints]` line makes the decision visible.

Example — a name that tells you where to look. A failing build reports

`test_queue_designConcurrency.cxx`. You already know the failure is about thread safety, not about the happy path, before you open the file.

The Implementation Tracking Template

Every CaTDD test file carries a TODO / Implementation Tracking section. It lists every test case, its status, and where the work stands.

```
//=====
// ★ HIGH PRIORITY – Core Functionality
//=====
// ○ [@AC-1,US-1] TC-1: verifyCore_byBasicOperation_expectSuccess
// ● [@AC-2,US-1] TC-1: verifyCore_byMaxCapacity_expectProperHandling
//
//=====
// ☆ MEDIUM PRIORITY – Edge & Error Handling
//=====
// ○ [@AC-3,US-1] TC-1: verifyEdge_byEmptyQueue_expectEmptyResult
// ○ [@AC-4,US-2] TC-1: verifyMisuse_byDoubleInit_expectError
//
//=====
// ○ LOW PRIORITY – Advanced Scenarios
//=====
// ○ [@AC-5,US-2] TC-1: verifyPerformance_byHighLoad_expectAcceptableLatency
//
// ▶ GATE P0: All P0 tests must be GREEN before proceeding to P1.
```

Read it as a dashboard:

- not written yet → the backlog for this file
- written, still failing → what you are working on now
- passing → done, and provably done

every line also names its @[AC] and @[US], so a status report can be traced back to a requirement

Example — the section answers the daily question. "What is left before I can start P1?" You read the P0 block, count the ○ and ● lines, and answer in seconds. No separate status meeting and no separate tracker to update.

Design as a Living Contract

CaTDD redefines design. Design is not an activity that finishes before coding starts. It is a living contract that lives in the test file, changes with the code, and is readable by both people and LLMs.

The contract answers five questions, in one place:

1 WHAT MATTERS	User Stories with business value
2 WHAT TO VERIFY	Acceptance Criteria, GIVEN/WHEN/THEN
3 HOW TO VERIFY	Test Cases with concrete assertions
4 WHAT PRIORITY	P0 → P1 → P2 → P3
5 WHAT STATUS	○ → ● → ●

the design skeleton holds all five

This is the foundation of everything else in CaTDD. It is what `methodPrompts` defines, what `slashCommands` operationalizes, what `codeAgents` execute, and what `agentSkills` package. Every other layer depends on these concepts.

From Concepts to Action

The remaining chapters move from what to how.

Ch 1	defineConcepts	— the vocabulary	(you are here)
Ch 2	chatVibeCoding	— talk to an LLM, with structure	
Ch 3	callSlashCommands	— run the command flows	
Ch 4	asyncCodeAgent	— let an agent do the work	
Ch 5	applyClassicSWE	— the TDD / BDD / DDD foundation	
Ch 6	beyondXYZSpec	— where CaTDD sits next to other tools	

Chapter	Chapter	What you get
Chapter 2	chatVibeCoding	How to work with an LLM in chat, and the difference between VibeCoding and SpecCoding
Chapter 3	callSlashCommands	How to invoke the slash command system for repeatable execution
Chapter 4	asyncCodeAgent	How code agents automate test design, test implementation, and the SpecCoding lifecycle
Chapter 5	applyClassicSWE	The Knowledge Book of Software Engineering (TDD, BDD, DDD) and how CaTDD combines them
Chapter 6	beyondXYZSpec	How CaTDD compares with other spec-driven tools, and what it adds

chatVibeCoding

devBooks/Action/02_chatVibeCoding.md

What Is Vibe Coding?

Vibe coding means talking to an LLM in plain language and letting it produce code. You describe what you want, the LLM writes an implementation, and you refine it by talking.

It is fast and it feels natural. It also has a well-known failure mode: the conversation becomes the only place where the design exists. When the chat is gone, the reasoning is gone too.

CaTDD keeps the speed and removes the failure. It treats conversation as the **first mode** of LLM work — the exploratory mode — and gives it a structure that survives the chat.

plain vibe coding	CaTDD VibeCoding
"write me a test"	"read the method prompt, then design the skeleton"
→ code appears	→ US/AC/TC comments appear
→ you run it	→ you review the design
→ it passes	→ then implement, RED → GREEN
→ nobody knows why	→ the design is now in the file

The conversation is still free-form. What changes is that every useful result gets written down in a place the next session can find.

The Two Modes of CaTDD Interaction

CaTDD has exactly two ways to work with an LLM, and they use different layers of the method.

Mode	What you use	How it feels
VibeCoding	<code>methodPrompts</code> directly in chat	Free conversation with method guidance
SpecCoding	<code>slashCommands</code>	A structured flow with explicit lifecycle steps



Aspect	VibeCoding	SpecCoding
Definition	Flexible, method-guided conversation	Structured flow with explicit lifecycle commands

Aspect	VibeCoding	SpecCoding
Method source	methodPrompts directly	slashCommands (which wrap methodPrompts)
Control	The developer drives the conversation	The flow commands drive the lifecycle
Artifacts	Comments, test files, code — whatever you make	SpecFlow artifacts: pendingNews, todoUS, doingUS, abortUS, doneUS
Traceability	Optional, depends on the developer	Mandatory, enforced by the flow
Best for	Exploration, drafting, brainstorming, quick fixes	Structured delivery, teamwork, reproducible workflows
How you invoke it	You type a free-form prompt	You invoke /UT_* or /SPEC_* commands

The key idea — these are not competing tools. They are two stages of the same workflow, and CaTDD tells you when to move between them.

When to Use VibeCoding

VibeCoding is the right choice in five situations.

1	exploring a new feature	idea is still rough
2	freely drafting test ideas	Stage-0, no categories yet
3	clarifying requirements	what does "correct" mean?
4	troubleshooting	why is this test failing?
5	one-off work	too small to formalize

1. **You are exploring a new feature.** You have a rough idea and need help thinking through coverage, edge cases, and risks. You paste the API header, ask "what should I test, and what could go wrong?", and iterate on the answers.
2. **You are freely drafting test ideas.** CaTDD calls this Stage-0 (Freely Drafting). You capture raw scenarios, risks, examples, and open questions without forcing a category decision too early. This is pure VibeCoding.
3. **You are clarifying requirements.** You paste a module README, ask the LLM to find the ambiguous acceptance criteria, and discuss what the behavior should be. This is the informal version of the `SPEC_clearStoryIntent` step.
4. **You are troubleshooting.** A test is failing. You paste the error and the test code, and the LLM helps you decide whether it is a setup problem, an assertion problem, or a real bug.
5. **You are doing one-off work.** A small validation script, a configuration change, a documentation update. The overhead of a formal lifecycle costs more than it returns.

Example — a good first VibeCoding prompt. Notice that it sets the method, the component, and the question, and nothing else.

Using CaTDD (methodPrompts/CaTDD_methodPrompt.md), help me design the P0 Functional skeleton for an event queue component.

Context:

- producers post events asynchronously
- consumers pull events, optionally blocking
- the queue has a fixed capacity

Start with coverage dimensions, then propose US/AC/TC.
Do not write implementation code yet.

The last line is the important one. It forces the design step to happen before the code step.

When to Switch to SpecCoding

VibeCoding has a natural limit. You have reached it when you notice any of these symptoms.

SYMPTOM	WHAT IT MEANS
repeating the same instructions	the method is not written down
losing track of decisions	the design has no home
unsure which tests exist	there is no status list
can't hand work to a teammate	progress is not shareable
can't resume after a week	context lived only in the chat

When you see these, switch. VibeCoding is for exploring; SpecCoding is for delivering. They are complementary stages in one workflow, not competitors.



How VibeCoding Works with methodPrompts

VibeCoding works because the method travels with the conversation. You paste the method prompt once, and the LLM stops behaving like a general code generator. It starts behaving like a CaTDD practitioner: it designs before it codes, it labels categories, and it keeps a status list.

The Basic Pattern

1	SHARE	paste methodPrompts/CaTDD_methodPrompt.md
2	DESCRIBE	"Using CaTDD, design tests for [component]"
3	SUPPLY	API headers, spec docs, existing test files
4	ITERATE	review, refine, ask for alternatives
5	PRODUCE	US/AC/TC design, test code, production code
6	PRESERVE	copy the output into your own files

Step 1 is the one people skip, and it is the one that changes the answer. Without the method prompt you get generic TDD: a test appears, it passes, and nobody can say which requirement it defends. With the method prompt you get a US/AC/TC chain you can review before any code exists.

Common mistake — Treating step 6 as optional. The LLM's answer lives in a chat window; your design has to live in your repository. Copy the skeleton into the test file in the same session, or you will be retyping it from memory tomorrow.

Example: VibeCoding a Typical Skeleton

Developer:

Using CaTDD methodology (from methodPrompts/CaTDD_methodPrompt.md), help me design the Typical skeleton for an event queue component.

Key concerns:

- Producers post events asynchronously
- Consumers pull events with optional blocking
- Queue has a fixed capacity

Coverage dimensions: Producer × Consumer × Mode

Priority: Default P0 order (Typical → Edge → Misuse → Fault)

Three details in that prompt do the heavy lifting: the method source, the coverage dimensions, and the priority order. The LLM does not have to guess any of them.

CodeAgent (after reading the method prompt):

Coverage Matrix: Producer × Consumer × Mode

Producer	Consumer	Mode	Scenario
Single	Single	Callback	US-1: basic event flow
Single	Single	Pull	US-2: polling flow
Multiple	Single	Callback	US-3: fan-in

P0 Functional / Typical User Stories:

US-1: As an event producer,
I want to post events to a queue for asynchronous processing,
So that my producer thread is not blocked by consumer work.

US-2: As an event consumer,
I want to pull events from the queue with optional blocking,
So that I can process events when ready without busy-waiting.

US-3: As a system operator,
I want multiple producers to safely post to the same queue,
So that events from different sources are handled correctly.

The conversation continues in the same shape: the developer reviews, asks for Edge coverage, asks for implementation, refines. The LLM is not "writing code for you" — it is applying the method with you at conversation speed.

Notice what the matrix buys you. Three rows is not a test plan; it is a claim about which combinations are covered. When someone asks next week "is fan-in covered?", the row answers in one second instead of a source re-read.

The VibeCoding Workflow

CaTDD VibeCoding follows the same phases the method already defines. The difference is that a conversation carries you between them.

PHASE 1 Understanding →	PHASE 2 Design →	PHASE 3 Implementation →	PHASE 4 Iteration
read files ask questions no output yet	write comments US / AC / TC no code yet	RED → GREEN one TC at a time code appears	next TC, gate review status updated

Phase 1: Understanding (method-guided exploration)

What you say to the LLM:

"I'm building a [component]. Here are the API headers.
Using CaTDD, what should I test and how should I prioritize?"

What the LLM does:

- Reads your interfaces
- Checks existing related tests
- Identifies dependencies and constraints
- Presents an understanding summary
- Asks clarifying questions before proceeding

CaTDD contract: the LLM follows the Agent Workflow Checklist from the methodPrompt (Phase 1: Understanding). It reads files, identifies APIs, finds similar tests, and clarifies ambiguities before designing anything.

Phase 2: Design (comment-first, no code yet)

What you say:

"Good. Now design the coverage matrix and User Stories."

What the LLM does:

- Fills the OVERVIEW section (WHAT/WHERE/WHY/SCOPE)
- Defines Coverage Matrix dimensions
- Writes User Stories (2–5, in As a / I want / So that format)
- Writes Acceptance Criteria (GIVEN/WHEN/THEN)
- Details Test Cases with structured metadata
- Populates the TODO tracking section

CaTDD contract: design before code. The LLM writes the US/AC/TC chain as comments before generating any test implementation. This is the Comment-alive part: the design lives in comments that stay with the code.

Phase 3: Implementation (TDD Red→Green)

What you say:

"Implement the first Typical test case. Follow RED→GREEN."

What the LLM does:

- Writes the test implementation (4-phase structure)

- Marks it ● RED in the TODO section
- Confirms it fails (it should fail: the code does not exist yet)
- Implements minimal production code
- Confirms it passes
- Marks it ● GREEN

CaTDD contract: strict TDD discipline. The LLM never implements production code before a failing test exists, and it updates the status markers immediately after each action.

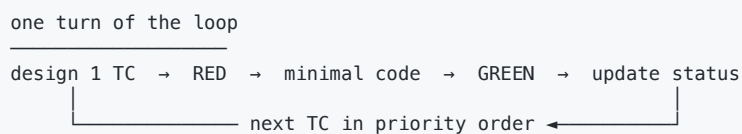
Phase 4: Iteration and Refinement

What you say:

"Now the Edge tests. And add tests for queue full behavior."

What the LLM does:

- Continues through the priority framework
- Advances TC by TC through the RED→GREEN cycle
- Stops at each quality gate for your review
- Reports coverage gaps and recommends next steps



VibeCoding vs SpecCoding: The Developer's Decision

The choice is a spectrum, not a switch. Most work starts in VibeCoding and finishes in SpecCoding.

Start in VibeCoding When

the component is new, requirements are still emerging
 you are the only developer on it
 you need to explore before committing to structure
 scope is small: <5 User Stories, <20 test cases
 you want fast iteration with no lifecycle overhead

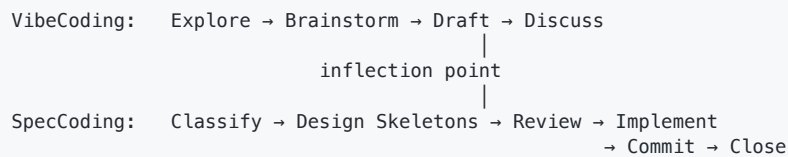
Switch to SpecCoding When

requirements stabilized and need traceability
 teammates are involved
 work must pause and resume across days
 scope is large: 5+ User Stories, 20+ test cases
 you want reproducible /UT_* and /SPEC_* sequences
 stakeholders need progress visibility

That last point has a concrete shape. Progress is visible as story state: `pendingNews` → `todoUS` → `doingUS` → `doneUS`, with `abortUS` available when an active story should not continue.

The Natural Transition Point

The most common transition happens right after Stage-0 Freely Drafting. You explored in VibeCoding, brainstormed risks, sketched scenarios. The moment you can sort those drafts into CaTDD categories, you are at Stage-1 — and structured commands start paying for themselves.



The Method Prompt as Chat Context

In VibeCoding the method prompt is your shared context. Share it once and the LLM applies it for the rest of the conversation — no need to re-explain CaTDD every session.

What to Share

1	master method prompt	CaTDD_methodPrompt.md	priority framework, categories, US/AC/TC format, status tracking, quality gates
2	category prompt	CaTDD_methodPrompt4Cat-*.md	only when the work sits inside one specific category
3	API interfaces	headers, spec docs, protocol defs	the component under test
4	existing test files	so the LLM follows your conventions and does not duplicate what already exists	

How Much Context Is Enough?

The CaTDD method prompt is comprehensive (1500+ lines of specification). You do not need to paste all of it into every conversation:

- **First conversation** — share the full method prompt. The LLM now understands CaTDD.
- **Subsequent conversations** — reference it: "Continue using CaTDD method as before."
- **Focused conversations** — share only the category prompt that applies to the work.

The context window is your constraint, so spend it in priority order:

highest	component interfaces + spec docs	MANDATORY
priority	relevant category prompt	IMPORTANT
	existing test code	HELPFUL
lowest	tells the LLM WHAT → HOW → house style	

Drop the interfaces and the LLM invents an API. Drop the category prompt and it designs a Typical case in Edge clothing. Drop the existing tests and it invents a second naming convention beside yours.

The Model Tier Strategy for VibeCoding

VibeCoding puts you in direct conversation with a model, so the tier you pick sets the ceiling on decision quality. CaTDD recommends the smallest tier that preserves that quality.

Tier	Use it for	Why
SOTA reasoning (GPT-5.5-xHigh)	Architecture decisions, system boundaries, quality trade-offs, cross-module constraints	The conversation contains irreversible choices
High performance	Requirements analysis, US/AC/TC design, test design, code review	Reasoning must span several artifacts: specs, tests, production code
Flash speed	Single test implementation, TODO status updates, simple refactoring	Inputs are clear and the action is narrow

```

task feels shallow or mis-categorized?
flash  —————> high performance
high performance ———> SOTA reasoning
                        (architecture feels risky)

```

Escalation rule: if a flash-speed model produces a design that looks shallow, escalate to high-performance. If a high-performance model makes an architecture decision that feels risky, escalate to SOTA.

Why this matters — a story split that is wrong at the architecture tier costs a rewrite of every AC and TC underneath it. A TODO marker updated by a cheaper model costs nothing. Spend the expensive model where the decision fans out.

Common VibeCoding Patterns

Four conversations cover most of what developers actually do.

```

1 DESIGN REVIEW      "revise US-2, then re-check its ACs"
2 COVERAGE GAP      "what am I missing against the framework?"
3 TROUBLESHOOTING    "TC-3 fails – test bug or product bug?"
4 WHAT'S NEXT        "P0 Typical is done. Now what?"

```

Pattern 1: The Design Review Loop

```

Developer: "Design the Typical skeleton for this API."
CodeAgent: [produces US/AC/TC design]
Developer: "US-2 doesn't capture the non-blocking requirement. Revise."
CodeAgent: [revises US-2 and updates the ACs and TCs that depend on it]
Developer: "Good. Now add coverage for the timeout mode."
CodeAgent: [adds AC-4 and the TCs for timeout behavior]
Developer: "Looks complete. Switch to SpecCoding and start implementation."

```

This is the most natural pattern: you review the design, the LLM keeps traceability while it edits. Revise one US and the affected ACs and TCs move with it — which is exactly the bookkeeping humans skip when they edit by hand.

Pattern 2: The Coverage Gap Analysis

```
Developer: "Here are my current test files. What coverage gaps do you see?"
CodeAgent: [checks existing US/AC/TC chains against the priority framework]
CodeAgent: "Missing: P0 Edge for queue full behavior, P0 Misuse for double-close,
            P1 State for lifecycle transitions after cleanup."
Developer: "Design the Edge skeleton first. Then we'll do Misuse."
```

The LLM uses the priority framework as a checklist against what you already have. It answers a question that is hard to answer by reading your own test file: what should exist and does not.

Pattern 3: The Troubleshooting Session

```
Developer: "TC-3 is failing. Expected IOC_RESULT_INVALID_PARAM, got success."
CodeAgent: [reads the test code, reads the production code]
CodeAgent: "The test calls IOC_post(NULL, ...) but production has a guard clause
            returning SUCCESS for a NULL handle. Either:
            A) production bug – it should return INVALID_PARAM, or
            B) test misunderstanding – NULL is allowed by this API.
            Which one matches the API contract?"
```

The LLM compares the test's expectation with the production behavior and surfaces the ambiguity in the contract. That ambiguity is the real finding — the failing assertion is only its symptom.

Pattern 4: The "What's Next" Inquiry

```
Developer: "I've completed all P0 Typical tests. What should I do next?"
CodeAgent: [reads the TODO tracking section]
CodeAgent: "You have 4 Edge cases at ○ TODO, 2 Misuse cases at ○ TODO,
            and 1 Fault case at ○ TODO. In the P0 default order, Edge comes
            next. TC-Edge-1 depends on a capacity API you haven't implemented –
            consider marking it BLOCKED and proceeding with TC-Edge-2."
```

The LLM reads your tracking section and recommends the next action from priority order plus dependency analysis. This is the cheapest use of a model in the whole workflow, and one of the most valuable: it removes the "where was I?" tax at the start of every session.

The Two Design Stages in VibeCoding

The method prompt supports two design stages, and VibeCoding is where they feel natural.

STAGE-0 freely drafting	STAGE-1 classifying design
raw ideas, risks, questions →	US / AC / TC per category
breadth over structure	structure over breadth
"what could go wrong?"	"which category, which gate?"
output: messy notes	output: reviewable skeleton

Stage-0: Freely Drafting

This is pure VibeCoding: capture scenarios, risks, examples, and open questions without forcing categories. The goal is coverage breadth, not structure.

What you say:

```
"I'm building an event queue. What could go wrong? What should I test?"
```

What the LLM produces:

```

FreelyDrafts:
- What if the queue is full and a producer tries to post?
- What if a consumer tries to pull from an empty queue?
- What if multiple producers post simultaneously?
- What if a consumer blocks waiting for events that never arrive?
- What if queue capacity is 0 – should that be allowed?
- What if we post after the queue has been destroyed?
- What if event sizes vary dramatically?
- What if the callback consumer throws an exception?
- Risk: consumer deadlock if a callback posts back to the same queue
- Risk: memory leak if events are posted but never consumed
- Question: should a zero-capacity queue be a valid configuration?
- Question: should we support priority-based event ordering?

```

Nothing here is a test yet. That is the point: two open questions and two risks are cheaper to discuss now than to discover after the implementation is written.

Stage-1: Classifying Design

Now you sort the drafts into CaTDD categories and write real skeletons. This is the bridge between VibeCoding and SpecCoding.

What you say:

```
"Classify these drafts into CaTDD categories and design the skeletons."
```

What the LLM produces (structured by category):

```

P0 Functional / Typical:
  US-1: As a producer, I want to post events so they are queued for processing.
  AC-1: GIVEN queue with available capacity, WHEN producer posts, THEN event is queued.
  TC-1: verifyEventPost_byAvailableCapacity_expectEventQueued

P0 Functional / Edge:
  US-2: As a producer, I want non-blocking behavior when the queue is full.
  AC-2: GIVEN full queue, WHEN producer posts with NonBlock, THEN returns immediately with error.
  TC-2: verifyNonBlockPost_byFullQueue_expectImmediateReturn

P0 Functional / Misuse:
  US-3: As a developer, I want proper error codes for incorrect queue usage.
  AC-3: GIVEN destroyed queue, WHEN producer posts, THEN returns INVALID_STATE.
  TC-3: verifyPost_byDestroyedQueue_expectInvalidState

P0 Functional / Fault:
  US-4: As a system operator, I want graceful recovery from consumer failures.
  AC-4: GIVEN consumer callback crashes, WHEN posted events remain, THEN queue stays operational.
  TC-4: verifyQueue_byConsumerCrash_expectQueueRemainsOperational

```

Compare the two blocks. Stage-0 asked "what if?" four different ways. Stage-1 answers each one with a category, a story, a criterion, and a test name — so a slash command can pick up TC-1 and run it end to end.

VibeCoding with Real-Time Feedback

VibeCoding's strength is instant feedback: you see output and steer immediately.

```

your prompt → LLM response → your review → revised prompt → better response
      ↑                                     |
      └────────── continuous refinement ─────────┘

```

The loop is fast, and it is fragile in one specific way: it has no memory of its own. If you do not copy results out — into test files, into the TODO section — then the chat is the only record of the decision.

chat open	design lives here	→	you can still resume
chat closed	design is gone	→	you rebuild it from memory

The VibeCoding preservation rule: after every significant design output, copy it into a file. After every test implementation, run it and update the TODO section. Never let the chat be your only source of truth.

The CodeAgent's Role in VibeCoding

In VibeCoding the CodeAgent is a **method-aware conversation partner**, not an autocomplete with manners.

reads method prompts	CaTDD semantics, not generic TDD
preserves skeleton	@[US] @[AC] @[TC] are structural
follows priority	P0 before P1, Typical before Edge
tracks status	updates ○ → ● as work moves
asks instead of guessing	when product intent is unclear
reports the unknown	gaps stay visible, not papered over

The last two rows are what separate a partner from a generator. A generator that does not know the intended behavior will invent one that sounds reasonable — and a reasonable invention is the most expensive kind of defect, because it passes review.

The CodeAgent does not replace the developer's judgment. It applies the method consistently and produces structured output faster than a human types, which leaves your attention for the part only you can do: deciding whether the design is right.

VibeCoding Anti-Patterns

Anti-Pattern 1: VibeCoding Forever

Some developers stay in VibeCoding mode indefinitely and treat every conversation as disposable. What accumulates:

- Isolated test cases with no traceability to requirements
- Tests that pass without verifying anything meaningful — the green-but-hollow problem
- Inconsistent naming, structure, and priority order across files
- Design decisions that exist only in an expired chat session

Fix: switch to SpecCoding once the design stops changing. Move from conversation to structured artifacts.

Anti-Pattern 2: Skipping Design in VibeCoding

VibeCoding's speed tempts you to jump straight to implementation: "write me a test for the event queue." The LLM produces test code, you run it, it passes. It feels productive. But:

- No US/AC/TC traceability — which business value does this test defend?
- No priority classification — is this Typical, Edge, or an accident?
- No coverage reasoning — what should be tested and is not?

Fix: even in VibeCoding mode, keep design-before-code. Ask for the skeleton first, review it, then ask for the implementation.

Anti-Pattern 3: Context Starvation

You ask the LLM to design tests without giving it API headers, existing test patterns, or project conventions. It fills the gaps with plausible assumptions, and plausible assumptions are exactly what a reviewer skips over.

Fix: before asking for a design, provide:

- The method prompt — so the LLM follows CaTDD
- The component interfaces — so the LLM knows **what** to test
- Existing test examples — so the LLM follows your conventions
- Your specific concerns or risk areas — so it prioritizes correctly

Anti-Pattern 4: Silent Approval

The LLM produces a design. You read it. It looks reasonable. You say "implement it." But looking reasonable is not the same as being correct.

Fix: review every output against the CaTDD contract by name:

- Do the User Stories express real user value?
- Do the Acceptance Criteria use GIVEN/WHEN/THEN precisely?
- Do the Test Cases trace back to an AC and a US?
- Is the category right — is this really Edge and not Misuse?
- Does the assertion verify the claim the test is named for?

From VibeCoding to SpecCoding: The Bridge

The bridge between VibeCoding (this chapter) and SpecCoding (chapter 03) is the slash command system. Five steps carry your conversation into a structured flow:

```

1  preserve    copy US/AC/TC design into the test file as a comment skeleton
2  initialize  /SPEC_initProjectContext      shared project context
3  import      /SPEC_importFeature           traceable story artifacts
               /SPEC_importIssue
4  open        /SPEC_openUserStory           move the story into active work
5  execute     /UT_designTypicalSkeleton     reproducible, traceable steps
               /UT_implTestCase

```

The handoff is the whole point of having two modes. VibeCoding finds the design; SpecCoding delivers it. Nothing you discovered in the conversation is thrown away — it becomes the skeleton the commands operate on.

chapter 02 conversation US/AC/TC drafted by hand in chat traceability optional	→	chapter 03 lifecycle US/AC/TC carried by commands traceability enforced
--	---	---

callSlashCommands

devBooks/Action/03_callSlashCommands.md

What Are Slash Commands?

A slash command is a Markdown file that tells a CodeAgent what to do next. It is not a shell command. It is a written instruction with a contract: read this, do exactly this one step, produce this, keep these things intact, and then say what comes next.

That is why the same command works in different tools. The file is plain Markdown, so Copilot, Cline, Continue, and `utCodeAgentCLI` can all read it.

<u>methodPrompts</u>		<u>slashCommands</u>		<u>CodeAgent</u>
what the method means: categories, gates, US/AC/TC format, priorities	→	one step, with a contract: what to read, produce, preserve, and report	→	performs the step: reads the files, writes the output, reports the result

The commands live in `slashCommands/`. They follow one architectural rule, and it matters more than any other rule in this chapter:

When a slash command conflicts with methodPrompts, methodPrompts always wins.

Commands are the operational layer. They sequence work; they do not invent method semantics. A command tells you when to design a Typical skeleton. The method prompt still defines what a Typical skeleton is.

Why commands beat ad-hoc requests

```
ad-hoc: "add some tests for IOC_post"
        → code appears, passes, and nobody can say which
           requirement it defends

command: /UT_designTypicalSkeleton
        → US/AC/TC chain + coverage matrix + TODO status
        → reviewable before a single line of test code exists
```

Both cost one message to the agent. Only the second leaves a reviewer something to check.

The Three Command Families

UT_*	test design and implementation works on: skeletons, test files, TODO status
SPEC_*	story lifecycle orchestration works on: context, news, stories, designs, commits
HARNESS_*	engineering tool points around the harness works on: installs, diagnostics, sessions, patches

UT Commands — Unit Test Design and Implementation

UT_<verb><Object>

UT commands execute CaTDD test development steps. They operate on test files, design skeletons, and implementation artifacts, and they belong to category-specific flows (P0, P1, P2).

Examples:

- `UT_designTypicalSkeleton` — design the Typical category design skeleton
- `UT_implTestCase` — implement one test case through the RED→GREEN cycle
- `UT_reviewFuncTestsSkeleton` — review the complete P0 functional skeleton set before implementation
- `UT_tellMeNextImplTest` — pick the next test case from the TODO tracking section

SPEC Commands — SpecCoding Lifecycle Orchestration

SPEC_<verb><Object>

SPEC commands drive work through a traceable SpecCoding lifecycle. They operate on lifecycle artifacts — project context, pending news, user stories, design docs, tests, product code, commit state — not on test categories or category skeletons directly.

Examples:

- `SPEC_initProjectContext` — create the first project context file
- `SPEC_analyzeIssue` — turn an issue into a traceable user story
- `SPEC_takeArchDesign` — produce the high-level architecture design
- `SPEC_designUnitTests` — enter CaTDD test design, usually through the P0/P1/P2 flows
- `SPEC_commitWorks` — prepare and commit completed work
- `SPEC_closeUserStory` — move reviewed completed work to the done archive

HARNESS Commands — Harness Engineering Tool Points

HARNESS_<verb><Object>

HARNESS commands are tool points, not story-lifecycle steps. They maintain the CaTDD harness around the method source: installed target-project assets, native adapter wrappers, execution diagnostics, and guarded patch-back to the original CaTDD repository.

Two boundaries define them: they **never move SpecFlow lifecycle state**, and they **never redefine CaTDD category meaning**.

Examples:

- `HARNESS_verifyInstallation` — verify an installed target project has complete, consistent adapter assets
- `HARNESS_diagnoseInstallation` — diagnose a failed or misbehaving installation and recommend repairs
- `HARNESS_diagnoseProject` — diagnose repo correctness and SpecCoding/CaTDD drift without mutating source
- `HARNESS_newTaskSession` — capture session context so a new CodeAgent session can resume without re-investigation
- `HARNESS_evolveHarness` — learn from verified success or failure and evolve the narrowest canonical owner
- `HARNESS_patchCaTDDSource` — patch effective installed-project CaTDD improvements back to the original repository

The three families answer three different questions: is the test design right? (UT), is the story moving correctly? (SPEC), is the tooling around the method healthy? (HARNESS).

The Universal Command Template

Every slash command uses the same Markdown structure, defined in `UT_slashCommandTemplate.md` and `SPEC_slashCommandTemplate.md`:

Section	Content
Command Header	Command name, flow, CaTDD class, category, method source, adapter target
CoT Pattern	Declared reasoning pattern — ReACT, ToT, or Linear — with execution steps for that pattern
WHO	Who invokes this command and who should act on it
WHAT	Exactly what this command does — the single workflow step it performs
WHEN	Valid starting conditions, when NOT to use it, previous/next commands
WHERE	Input files, output files, method references, related flow documents
WHY	Developer value, CaTDD method reason, how it reduces ambiguity
HOW	Execution procedure — what to read, preserve, perform, report
Subagent Recommendation	(SPEC commands only) Whether this command suits background subagent delegation
Input Contract	Command parameters using portable placeholders
Output Contract	Expected response shape — summary, files touched, next command
CodeAgent Compatibility	Plain Markdown, no tool-specific assumptions

```

one command file
├─ WHO / WHEN / WHERE  ← is this even the right step to run now?
├─ WHAT / HOW          ← the single step to perform
├─ Input / Output      ← what it consumes, what it must leave behind
└─ CodeAgent Compat.   ← why any agent can run it

```

Uniformity is the point: a command written once runs everywhere, because nothing in it depends on one tool's prompting format.

The Four Command Flows and the HarnessKits Kit

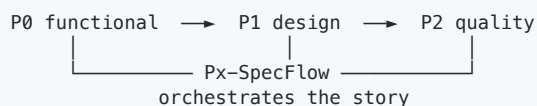
Commands are grouped into four flows plus one operational kit:

```
slashCommands/flows/
├── P0-FuncTestsFlow.md    ← UT commands for functional test design
├── P1-DesignTestsFlow.md  ← UT commands for design-oriented tests
├── P2-QualityTestsFlow.md ← UT commands for quality-oriented tests
└── Px-SpecFlow.md         ← SPEC commands for full lifecycle orchestration

slashCommands/kits/
└── Px-HarnessKits.md      ← HARNESS_* operational tool points
```

Px in SpecFlow means **cross-priority**. It is not a test category priority like P0/P1/P2 — it is a process flow that orchestrates those test layers.

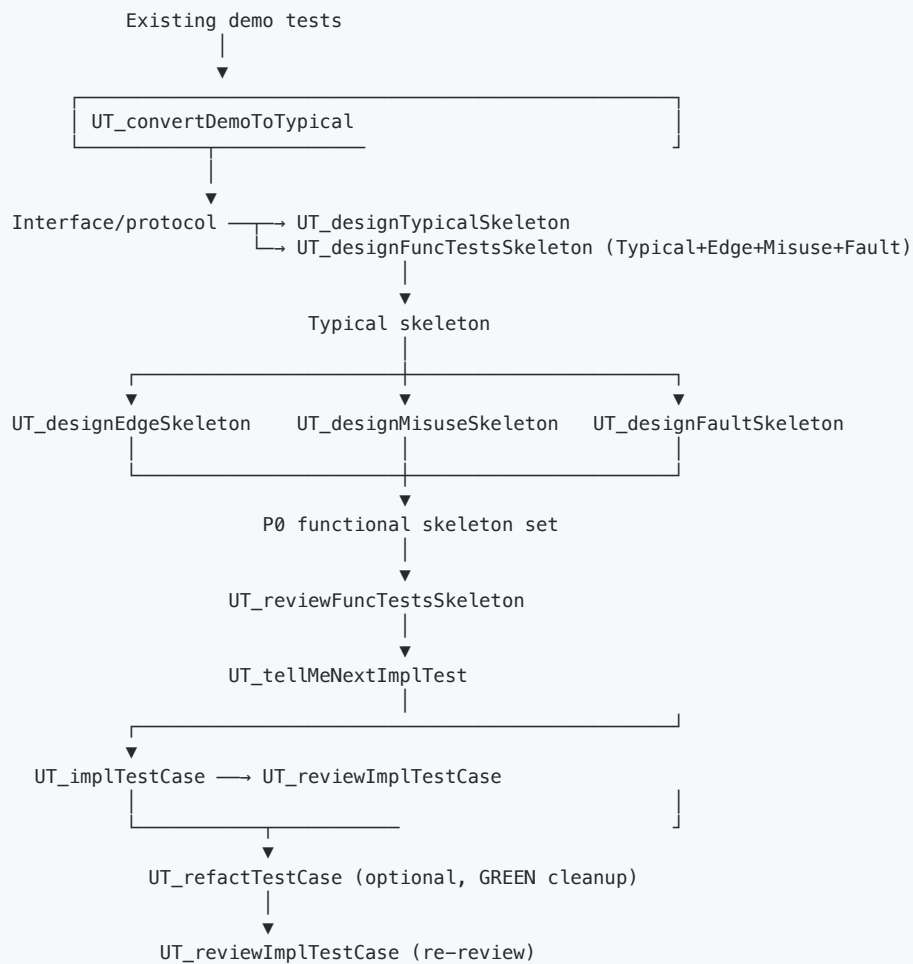
Px-HarnessKits is cross-priority too, but it is a kit rather than a lifecycle flow: it groups tool-point commands without imposing a strict command sequence. Future addon and demo commands belong to P3 Addons, which keeps them aligned with methodPrompts.



Px-HarnessKits ⊥ sits beside the flows:
install, diagnose, session, patch back

P0 FuncTestsFlow — The First Flow to Learn

P0 is the most common entry point because it covers functional test design, which is what every developer needs first. The flow has one job: turn a component into four kinds of functional skeleton (Typical, Edge, Misuse, Fault), review them as a set, then implement them one test case at a time.



Entry Points

You can start the P0 flow from three places:

1. **Existing demo tests** — use `UT_convertDemoToTypical` to extract core behavior into a CaTDD Typical skeleton. This preserves existing work instead of throwing it away.
2. **Interface or protocol** — use `UT_designTypicalSkeleton` to design the Typical skeleton from the API contract.
3. **Full functional set at once** — use `UT_designFuncTestsSkeleton` to produce all four skeletons (Typical + Edge + Misuse + Fault) in a single step.

The Ten Steps at a Glance

#	Command	What it produces
1a	<code>UT_convertDemoToTypical</code>	Typical skeleton extracted from an existing demo test
1b	<code>UT_designTypicalSkeleton</code>	Typical skeleton designed from the interface
3–5	<code>UT_designEdgeSkeleton</code> , <code>UT_designMisuseSkeleton</code> , <code>UT_designFaultSkeleton</code>	The other three P0 skeletons, in priority order
6	<code>UT_reviewFuncTestsSkeleton</code>	PASS/FAIL review report on the whole P0 set
7	<code>UT_tellMeNextImplTest</code>	The next test case to implement, with blockers
8	<code>UT_implTestCase</code>	One implemented test case with RED→GREEN history
9	<code>UT_reviewImplTestCase</code>	PASS/FAIL review of that implementation
10	<code>UT_refactTestCase</code> (optional)	The same behavior, easier to read

Steps 1–5: Design the Skeletons

`UT_convertDemoToTypical` is for demo tests that exercise the component. Note what those tests are not: they are raw input material, not CaTDD P3 Demo/Example tests. The command reads the demo, extracts the core behavior patterns, maps them into CaTDD Typical format, and writes a skeleton with `@[US]`, `@[AC]`, `@[TC]` markers and a populated OVERVIEW section.

`UT_designTypicalSkeleton` is for the no-demo case. It reads the interface specification, analyzes the API contract, finds the happy-path scenarios, and writes the full US/AC/TC chain.

Both paths continue to `UT_designEdgeSkeleton`. The next three commands each design one category skeleton, reading the existing skeletons for consistency, the API interface, and the category-specific method prompt. They run in P0 order: Edge after Typical, Misuse after Edge, Fault after Misuse.

Important: the same US IDs carry across categories. US-1 covers Typical scenarios, and that same US-1 can also have Misuse scenarios — what happens when this capability is used incorrectly inside the same user story. Categories are lenses on the same user value, not four separate backlogs.

Step 6: `UT_reviewFuncTestsSkeleton` — the gate that saves money

When: all P0 skeletons are designed, none implemented.

What it checks:

- **Completeness** — are all four categories present?
- **Traceability** — does every TC reference an AC, and every AC a US?
- **Category correctness** — is anything classified as Edge that is really Misuse?
- **Consistency** — do naming, status markers, and formats follow convention?
- **Gate readiness** — is this design ready for P0 implementation?

Output: a PASS/FAIL report per dimension, with the specific issues to fix.

```
UT_reviewFuncTestsSkeleton → FAIL
  completeness    PASS    4/4 categories present
  traceability     FAIL    TC-Edge-3 references no AC
  category         PASS    no Misuse scenario hiding in Edge
  consistency      PASS    naming and status markers follow convention
  gate readiness   FAIL    fix TC-Edge-3, then re-run this command
```

Next command: PASS → `UT_tellMeNextImplTest`. FAIL → back to the specific design command.

Never skip this gate. A design flaw caught here costs a text edit; the same flaw caught after implementation costs a rewritten test, a rewritten production path, and a re-review — which is exactly the 10x difference the method is built to exploit.

Step 7: `UT_tellMeNextImplTest` — remove the "where was I?" tax

When: ready to begin implementation — all skeletons reviewed and approved.

What it does: reads the TODO tracking section across all test files, applies the P0 default order (Typical → Edge → Misuse → Fault), checks dependencies, and recommends the next test case.

Output: the name of the next TC, the file it lives in, its category, its estimated complexity, and any dependencies or blockers.

```
TODO - IocQueue_Test.cpp
● TC-Typical-1    verifyEventPost_byAvailableCapacity_expectEventQueued
● TC-Typical-2    verifyEventPull_byPostedEvent_expectFifoOrder
○ TC-Edge-1       verifyNonBlockPost_byFullQueue_expectImmediateReturn
○ TC-Edge-2       verifyPull_byEmptyQueue_expectTimeout
○ TC-Misuse-1     verifyPost_byDestroyedQueue_expectInvalidState

→ next: TC-Edge-1  (P0 order, no blockers, complexity: low)
```

Step 8: `UT_implTestCase` — one test case, both colors

When: a specific TC has been selected for implementation.

What it does, in order:

1. Writes the test implementation using the 4-phase structure (SETUP / BEHAVIOR / VERIFY / CLEANUP)
2. Marks it ● RED in the TODO section
3. Runs the test — confirms RED, failing because production code is missing
4. Implements minimal production code
5. Runs the test — confirms GREEN
6. Marks it ● GREEN in the TODO section
7. Updates the TC specification with implementation notes

Output: one implemented test case with its RED→GREEN history and an updated status marker.

Next command: `UT_reviewImplTestCase`.

Step 9: `UT_reviewImplTestCase` — check the claim, not the color

What it reviews:

- **Correctness** — does the test actually verify the AC it claims to verify?
- **Structure** — does it follow the 4-phase pattern?

- **Assertions** — are there ≤ 3 key assertions, and are they meaningful?
- **Traceability** — is the US/AC/TC chain preserved?
- **Isolation** — does cleanup run and prevent state leakage?
- **Status** — are the markers correctly set?

Output: a PASS/FAIL report. PASS loops back to `UT_tellMeNextImplTest`; FAIL routes back for fixes.

Step 10: `UT_refactTestCase` (optional) — clean up without changing meaning

When: a selected TC is GREEN and reviewed, but the test body is hard to read or maintain.

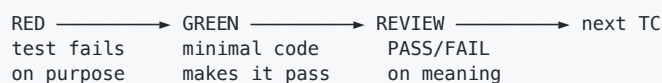
What it does: refactors that single test case for clarity, without changing behavior, category routing, or the US/AC/TC traceability chain.

Hard boundary: missing behavior, missing coverage, or wrong category routing must go back to the design or implementation commands. Refactoring is not a side door for new coverage.

Next command: `UT_reviewImplTestCase` again — to prove the cleanup caused no skeleton drift.

The RED→GREEN Discipline in Slash Commands

`UT_implTestCase` is where CaTDD stops being a design document and starts changing code. The discipline below is what keeps the test honest.



RED Phase

1. Read the TC specification (the comment design)
2. Write the test function following the 4-phase pattern
3. Add `@[Name]` and `@[Steps]` comments above the test function
4. Update the TODO tracking section: $\circ \rightarrow \bullet$ RED/FAILING
5. Compile and run – verify the test FAILS
6. If it passes: investigate. Either:
 - a) the test is poorly written and doesn't test the right thing
 - b) the feature already exists – production code was written first (TDD violation)
 - c) the test framework is misconfigured
7. Report: "RED confirmed. Test fails as expected because [reason]. Ready for production code implementation."

Step 6 is the one people skip. A test that passes before the code exists has told you something important, and none of the three explanations is good news.

GREEN Phase

1. Read the test code and understand what production code is needed
2. Write minimal production code – just enough to make this test pass
3. Do NOT implement untested features or over-engineer
4. Compile and run – verify the test PASSES
5. Update the TODO tracking section: $\bullet \rightarrow \circ$ GREEN/PASSED
6. Run the full test file – verify no regressions
7. Commit the change
8. Report: "GREEN confirmed. Test passes. [N] tests remain in P0."

The "Minimal Production Code" Rule

In the GREEN phase, write **only enough production code to make the current test pass**. Not one line more.

This rule prevents four specific failures:

- **Dead code** — code written but never tested
- **Over-engineering** — features no test ever asked for
- **False confidence** — tests passing because extra code happened to satisfy them
- **Lost traceability** — production code with no corresponding TC

TC-Edge-1 needs: `post()` returns `BUSY` immediately when the queue is full

```
minimal:      if (isFull()) return RESULT_BUSY;
too much:    if (isFull()) { log warn; notify observers;
                                   schedule retry; return RESULT_BUSY; }
```

The extra three behaviors are not "free." They are untested paths that now look like tested ones, and the next developer will trust them.

P1-DesignTestsFlow and P2-QualityTestsFlow

P1 and P2 follow the same shape as P0, with one constraint P0 does not have: the **Design Source Gate**.

The Design Source Gate

Before drafting a P1 or P2 skeleton, the command must find a confirmed design source. If the source is missing, the command **warns and stops** before drafting anything, and asks the developer where the design lives — instead of inventing architectural decisions inside a test file.

Priority	Category	Required Design Source
P1	State	README_StateDesign.md or a State Design chapter in README_ArchDesign.md
P1	Capability	README_DetailDesign.md
P1	Interaction	Sequence, interaction, or collaboration sections in README_ArchDesign.md
P1	Concurrency	README_ResourceDesign.md
P2	Performance	README_PerfDesign.md
P2	Robust	README_ErrorDesign.md
P2	Compatibility	README_CompatDesign.md
P2	Configuration	README_DetailDesign.md
P2	Diagnosis	README_DiagnosisDesign.md / README_VerifyDesign.md evidence requirements
P2	Security	README_SecurityDesign.md (+ threat model, security policy, or constitutional invariants (\$K\$))

```

/UT_designStateSkeleton
├── README_StateDesign.md found?
│   ├── yes → draft the skeleton
│   └── no → stop and ask:
│       "Where does the state architecture design live?
│       I need it before I can design state tests."

```

The gate exists because P1 and P2 tests verify **architectural decisions**, not API behavior. Writing a state test without a state design means the test encodes whatever the agent assumed — and assumptions are much harder to review than designs.

P1-DesignTestsFlow

Addresses architectural validation: State, Capability, Interaction, and Concurrency.

```

P0 functional skeletons (completed)
├── UT_designStateSkeleton
├── UT_designCapabilitySkeleton
├── UT_designConcurrencySkeleton
└── Interaction (direct method prompt, no dedicated command yet)
    │
    ▼
    UT_reviewDesignTestsSkeleton
    │
    ▼
    UT_tellMeNextImplTest → UT_implTestCase → UT_reviewImplTestCase

```

Entry conditions: P0 functional skeletons must exist (especially Typical and Edge). Each P1 category requires a confirmed design source. P1 must have design before skeleton drafting starts.

Routing note: State, Capability, and Concurrency have dedicated `UT_design*Skeleton` commands.

Interaction has no dedicated slash command yet — the flow routes it through

`CaTDD_methodPrompt4Cat-Interaction.md` directly, drafting source-linked US/AC/TC into the canonical `designInteraction` category file under the same Discovery Gate and review contract.

Gate P1: before entering P1, all P0 tests must be GREEN. Before leaving P1, the architecture must be validated: State tests GREEN when stateful behavior exists, Capability tests GREEN when designed limits exist, Interaction tests GREEN when sequence/collaboration/handoff rules exist, Concurrency tests GREEN when shared execution exists — with no deadlocks, no race conditions, ThreadSanitizer clean.

P2-QualityTestsFlow

Addresses non-functional quality attributes: Performance, Robust, Compatibility, Configuration, Diagnosis, Security.

```

P0/P1 stable coverage (completed)
├── UT_designPerformanceSkeleton
├── UT_designRobustSkeleton
├── UT_designCompatibilitySkeleton
├── UT_designConfigurationSkeleton
├── UT_designSecuritySkeleton
└── Diagnosis (direct method prompt)
    │
    ▼
    UT_reviewQualityTestsSkeleton
    │
    ▼
    UT_tellMeNextImplTest → UT_implTestCase → UT_reviewImplTestCase

```

Entry conditions: P0 functional coverage exists, P1 design coverage exists when relevant, and each P2 category has a confirmed project-root design source.

Routing note: Performance, Robust, Compatibility, Configuration, and Security have dedicated `UT_design*Skeleton` commands. Diagnosis has no dedicated slash command yet — the flow routes it directly through `CaTDD_methodPrompt4Cat-Diagnosis.md`, drafting source-linked US/AC/TC into the canonical `qualityDiagnosis` category file.

Gate P2: before entering P2, P1 must be complete. Before leaving P2, quality SLOs must be met, required diagnosis/observability evidence verified, security protection tests GREEN where a threat model or policy applies, and production-readiness criteria satisfied.

Px-SpecFlow — The Master Orchestration Flow

Px-SpecFlow is the largest flow because it does not manage tests — it manages stories. UT flows answer "what should this component do, and is it proven?" SPEC commands answer "where is this work right now, what does it need next, and who can see that it happened?"

```
UT flows    → work inside one category (Typical, Edge, State, ...)
SPEC flow   → work across the whole story lifecycle
              import → analyze → design → test → implement → commit → close
```

The SpecFlow Lifecycle

```
pendingNews —(analyze)—> todoUS → doingUS → doneUS
                        |
                        v
                        analyzedNews (raw input archive after analysis)
                        ^
                        |
                        suspendUS (paused, resumable)
                        ^
                        |
                        abortUS (unsafe active stories)
```

Every work item moves through analyzed, active, and archive lanes. `suspendUS` and `abortUS` exist so that a story that cannot continue is parked with a reason, instead of lingering in `doingUS` and quietly misrepresenting progress.

Stage	Directory	Meaning
pendingNews	<code>.catdd/spec/pendingNews/</code>	Work waiting to be analyzed. Raw issues and feature requests.
analyzedNews	<code>.catdd/spec/analyzedNews/</code>	Raw inputs archived after analysis — the preserved source trace.
todoUS	<code>.catdd/spec/todoUS/</code>	Work analyzed and ready to pick up. Structured user stories with acceptance criteria candidates.
doingUS	<code>.catdd/spec/doingUS/</code>	Active work in progress. Opened user stories going through design, test, or implementation.
suspendUS	<code>.catdd/spec/suspendUS/</code>	Suspended active stories preserved with a durable resume reference (branch/worktree).
abortUS	<code>.catdd/spec/abortUS/</code>	Aborted active work preserved for later analysis or a next-round improvement input.
doneUS	<code>.catdd/spec/doneUS/</code>	Completed work. Reviewed, committed, CI-passed stories.

An example of the lanes doing their job:

```

Day 1  QA files "IOC_post returns success on a destroyed handle"
       SPEC_importIssue      → pendingNews/20260910-ioc-post-destroyed.md
       SPEC_analyzeIssue     → todoUS/20260910-ioc-post-destroyed-UserStory.md
                               analyzedNews/20260910-ioc-post-destroyed.md

Day 2  SPEC_openUserStory → doingUS/...-UserStory.md
       SPEC_makePlan      → doingUS/...-UserStory-Tasks.md

Day 4  SPEC_suspendUserStory → suspendUS/... (waiting for HW trace tool)
Day 9  SPEC_resumeUserStory → doingUS/... (reference still valid)
Day 9  SPEC_commitWorks + SPEC_closeUserStory → doneUS/...

```

Six days of "paused" are now visible as a state with a reason, not as an ambiguous silence.

The Full Command Sequence

The complete SpecFlow lifecycle has **34 SPEC commands** organized into three phases.

Phase A: Pre-Story (input and analysis)

1. SPEC_initProjectContext
Creates .catdd/spec/projectContext.md – the shared project constitution
2. SPEC_updateProjectContext
Updates project facts, constraints, or conventions whenever they change
3. SPEC_importIssue / SPEC_importFeature
Imports work input into pendingNews/
4. SPEC_importUserStory
Queues already-structured user stories directly into todoUS/ (skips analysis)
5. SPEC_analyzeIssue / SPEC_analyzeFeature
Converts pending input into traceable user stories in todoUS/
Moves raw input from pendingNews/ to analyzedNews/ for traceability
6. SPEC_openUserStory
Moves the selected story from todoUS/ to doingUS/ (work begins)
Asks whether a dedicated story branch should be created or switched

Phase B: Design and Planning

- 7. `SPEC_clearStoryIntent` (optional but recommended)
 - Aligns developer intent and CodeAgent intent before design
 - Records Mutual Intent Contract: scope, non-goals, success signal, assumptions

8. `SPEC_makePlan`
Creates the paired task artifact (`*-UserStory-Tasks.md`) in `doingUS/`
Classifies work as: intent-clearing, requirement-oriented,
design-oriented, or implementation-oriented
Decides which `SPEC_*` step comes next

If requirement-oriented

9. SPEC_updateUserStory
Updates README_UserStories.md ledger and paired README_UserGuide.md (plus module surfaces)
10. SPEC_reviewUserStory
Reviews requirement quality
If PASS: commit/close, or transfer to design-oriented next steps

If design-oriented (initial architecture)

11. SPEC_takeArchDesign
Produces README_ArchDesign.md
12. SPEC_reviewArchDesign
Gates architecture quality before detail design
13. SPEC_updateArchDesign (if review fails)

If design-oriented (initial detail)

14. SPEC_takeDetailDesign
Produces README_DetailDesign.md and ACs
15. SPEC_reviewDetailDesign
Gates detail design quality
16. SPEC_updateDetailDesign (if review fails)

`SPEC_whatsNextTask` can be called at any point to read current state and recommend the single next command.

Phase C: Implementation and Closure

```

└─ If implementation-oriented ─┐
17. SPEC_designUnitTests
   Enters CaTDD test design via P0/P1/P2 flows
18. SPEC_implUnitTests
   Implements selected TCs (RED→GREEN)
   then SPEC_reviewImplUnitTests
19. SPEC_implProductCodes
   Implements production code to pass tests
20. SPEC_reviewProductCodes
   Reviews implementation quality; re-runs
   SPEC_reviewImplUnitTests before commit
   SPEC_refactUnitTests (optional) cleans one
   GREEN test, then review again

   If review FAILS:
21. SPEC_abortUserStory (when unsafe to continue)
   Moves the active story to abortUS for analysis

```

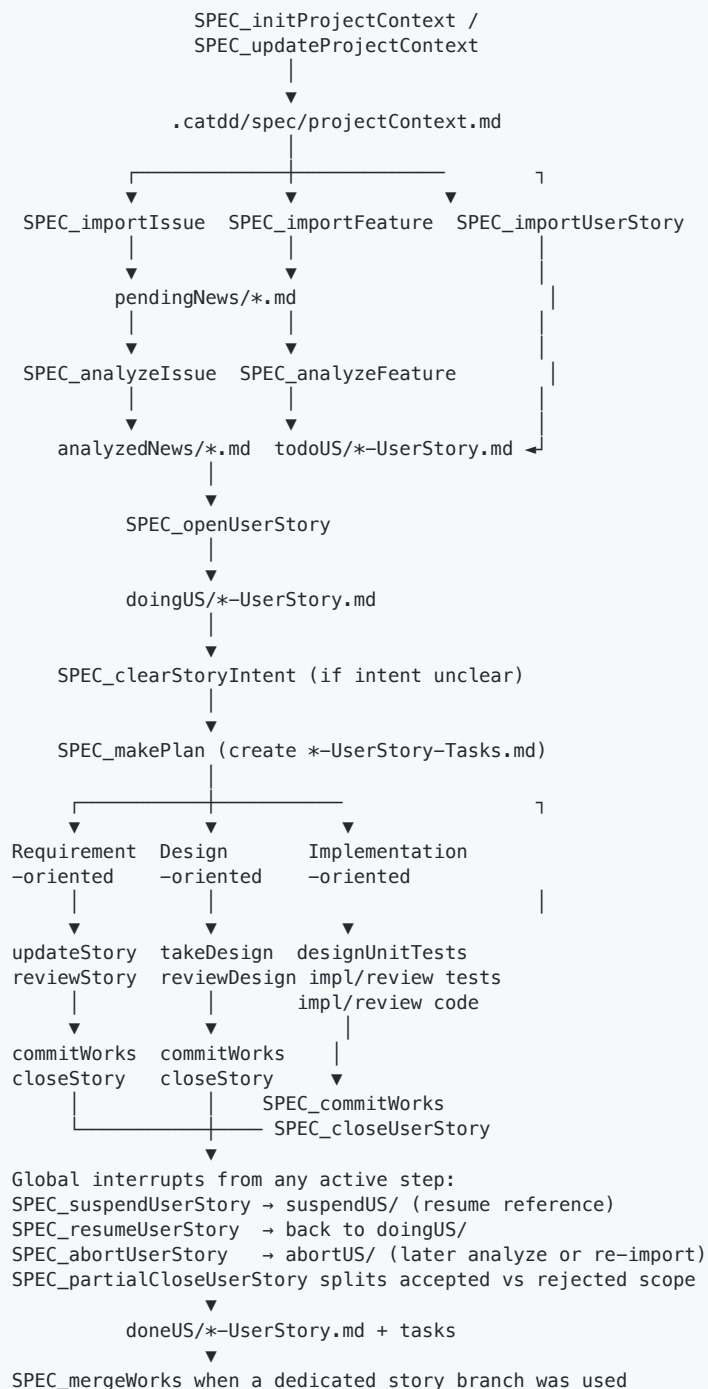
Suspend/resume is a global interrupt at any post-open, pre-close step:
 SPEC_suspendUserStory → suspendUS/ with durable resume ref
 SPEC_resumeUserStory → back to doingUS/ and continue

- 22. SPEC_commitWorks
Prepares and commits completed work
- 23. SPEC_closeUserStory
Moves the reviewed, committed story from doingUS/ to doneUS/
Moves the paired task artifact from doingUS/ to doneUS/
- 24. SPEC_mergeWorks (when a dedicated story branch was used)
Merges the closed branch into the integration branch
Skipped automatically when no story branch was created

SPEC_partialCloseUserStory splits an active story: the accepted slice closes with traceability while the rejected/deferred slice moves to abortUS/ with its reasons preserved.

SPEC_patchOriginalCaTDD (non-default branch) sends effective installed-project CaTDD meta-file improvements back upstream.

Flow Diagram: The Complete Lifecycle



Key SpecFlow Commands in Detail

The 34 commands are easier to remember as six jobs: set context, bring work in, align intent, plan, design-and-gate, then implement-and-close.

CONTEXT	SPEC_initProjectContext / SPEC_updateProjectContext
INTAKE	SPEC_import* / SPEC_analyze*
ALIGN	SPEC_clearStoryIntent
PLAN	SPEC_makePlan → SPEC_whatsNextTask at any time
DESIGN+GATE	SPEC_take*Design → SPEC_review*Design (never one without the other)
DELIVER	SPEC_designUnitTests → SPEC_impl* → SPEC_review* → commit → close

SPEC_initProjectContext

Purpose: create the project's shared constitution — the facts, constraints, conventions, and decisions that every later SPEC command references.

What it records:

- Project facts: repository, owner, primary purpose
- Layer model: methodPrompts → slashCommands → codeAgents → agentSkills
- Installed project surface: the `.catdd/` directory structure
- Stable conventions: documentation split, English/Chinese mirror rules, method-semantics boundaries
- Current design decisions: P0/P1/P2 category lists, design skeleton naming rules
- Validation commands: scripts that verify documentation and packaging integrity

Output: `.catdd/spec/projectContext.md`

Why it matters: without project context, every CodeAgent session starts from zero and re-derives your conventions — usually slightly differently each time. With it, the agent reads one file and knows the rules, the conventions, and the current state.

SPEC_analyzeIssue

Purpose: convert a raw issue or feature request into a traceable user story.

What it does:

1. Reads the pending issue/feature file from `pendingNews/`
2. Analyzes the input against project context and existing stories
3. Produces a structured user story in `todoUS/` containing role/capability/value, acceptance criteria candidates, a risk assessment, and open questions for the developer
4. Moves the raw input to `analyzedNews/` for traceability
5. Does **not** invent product intent — unclear aspects stay as open questions

Output: a user story file in `todoUS/` plus the archived raw input in `analyzedNews/`.

CoT pattern: ReACT — the command inspects current state, decides what to extract, acts by writing the story, and observes the result iteratively.

SPEC_makePlan

Purpose: create the execution plan for an opened user story.

What it does:

1. Reads the active story in `doingUS/`
2. Classifies the work as one of: **intent-clearing**, **requirement-oriented**, **design-oriented** (initial or follow-up), or **implementation-oriented**
3. Creates a `*-UserStory-Tasks.md` file with Markdown checkbox tasks
4. Decides the next SPEC command from that classification

Output: a task artifact in `doingUS/`.

Why it matters: this single classification is what stops teams from jumping to code on a story that first needed intent alignment or architecture. The most expensive mistake in the whole lifecycle is implementing the wrong scope correctly.

SPEC_makePlan classifies, and the classification chooses the road:

```
intent-clearing          → SPEC_clearStoryIntent
requirement-oriented    → SPEC_updateUserStory → SPEC_reviewUserStory
design-oriented          → SPEC_take*Design      → SPEC_review*Design
implementation-oriented → SPEC_designUnitTests → P0/P1/P2 flows
```

SPEC_clearStoryIntent

Purpose: align what the developer thinks the story means with what the CodeAgent thinks it means — before design starts.

What it records (the Mutual Intent Contract):

- **Developer intent** — what the developer believes the story is about
- **CodeAgent intent** — what the agent infers from reading the story
- **In-scope work** — what is included
- **Out-of-scope work** — what is explicitly excluded
- **Success signal** — how we know the story is done
- **Assumptions** — what is assumed but not confirmed
- **Open questions** — what needs a developer answer

Why it matters: if intent is not cleared, the agent may optimize for the wrong success signal. An expensive architecture design that solves the wrong problem is worse than no design at all, because it looks like progress.

SPEC_takeArchDesign and SPEC_takeDetailDesign

Purpose: produce design artifacts at two levels of abstraction.

Architecture Design (SPEC_takeArchDesign):

- High-level module decomposition
- Dependency direction and data flow
- Runtime placement and key trade-offs
- Creates README_ArchDesign.md and other architecture-oriented SPEC docs

Detail Design (SPEC_takeDetailDesign):

- Class-level API signatures and data structures
- State machines and lifecycle transitions
- Acceptance criteria for the active story
- Creates README_DetailDesign.md and other detail-oriented SPEC docs

Model tier: both commands require SOTA reasoning for initial design, because they decide boundaries, trade-offs, and constraints. Follow-up revision (SPEC_updateArchDesign, SPEC_updateDetailDesign) can use High Performance models.

Important: SPEC_take*Design is for **initial** design work. SPEC_update*Design is for **follow-up** revision against existing design evidence, review feedback, or story-level design gaps. Do not mix them.

SPEC_reviewArchDesign and SPEC_reviewDetailDesign

Purpose: gate design quality before downstream lifecycle steps consume it.

What they check:

- **Completeness** — are all necessary design aspects covered?
- **Traceability** — does the design connect to requirements?
- **Consistency** — are there internal contradictions?
- **Feasibility** — is it implementable given the constraints?
- **Clarity** — can a developer read this and implement it correctly?

Critical rule: every design-producing step (`SPEC_take*Design`, `SPEC_update*Design`) must be followed by its review gate before downstream lifecycle steps. Never skip review.

production step		gate step
<code>SPEC_takeArchDesign</code>	→	<code>SPEC_reviewArchDesign</code>
<code>SPEC_takeDetailDesign</code>	→	<code>SPEC_reviewDetailDesign</code>
<code>SPEC_updateArchDesign</code>	→	<code>SPEC_reviewArchDesign</code> (re-gate)
<code>SPEC_updateDetailDesign</code>	→	<code>SPEC_reviewDetailDesign</code> (re-gate)
a revision is a new claim – it gets the same gate as the original		

SPEC_designUnitTests

Purpose: enter CaTDD test design mode, typically routing through the P0/P1/P2 flows.

What it does:

1. Determines the testing method (default: CaTDD)
2. Identifies which test categories the story and design need
3. Routes to the appropriate UT flow commands
4. May invoke `UT_designTypicalSkeleton`, `UT_designEdgeSkeleton`, and so on

Testing method default: CaTDD is the default UnitTesting method in SpecFlow. Typical TDD or another project-specific method may be used when the project explicitly requests it.

SPEC_commitWorks

Purpose: prepare and commit completed work with traceable commit messages.

What it does:

1. Reviews what changed: test files, production code, design docs, spec artifacts
2. Groups changes by story/concern
3. Drafts commit messages referencing US IDs
4. Commits with the appropriate scope
5. Does **not** push — pushing stays a separate developer action

SPEC_closeUserStory

Purpose: move completed work to the done archive.

What it does:

1. Verifies the story is actually complete (reviews, tests, commits done)

2. Moves the story from `doingUS/` to `doneUS/`
3. Moves the paired task artifact alongside it
4. Records the closure date and any lessons learned
5. Refuses to close a story that is not reviewed and committed

SPEC_updateProjectContext, SPEC_suspendUserStory, SPEC_resumeUserStory, SPEC_mergeWorks, and SPEC_partialCloseUserStory

- `SPEC_updateProjectContext` — refresh the shared project constitution whenever project facts, constraints, or conventions change; run it after a merge when lifecycle or project-context facts changed.
- `SPEC_suspendUserStory` — pause an active story at any post-open, pre-close step when an external dependency or environment blocks it. The story and task artifact move to `suspendUS/` with a durable resume reference (branch or worktree).
- `SPEC_resumeUserStory` — move a suspended story and its task artifact back to `doingUS/` after validating that the resume reference still exists, then continue via `SPEC_whatsNextTask`.
- `SPEC_mergeWorks` — after `SPEC_closeUserStory`, merge the closed story branch into the target integration branch when branch integration is still required; it is auto-skipped when no dedicated story branch was used.
- `SPEC_partialCloseUserStory` — split one active story when only part of its scope is accepted: the accepted slice closes with traceability while the rejected or deferred slice moves to `abortUS/` with explicit reasons and evidence.

`SPEC_whatsNextTask` can be invoked at any point to read the current `.catdd/spec/` state and recommend the single next command. It is the cheapest way to answer "what now?" without re-reading the whole lifecycle.

Model Tier Guidance for Commands

Px-SpecFlow gives explicit model tier guidance. The rule is the same as elsewhere in CaTDD: use the smallest model that preserves decision quality.

Tier	Purpose	SPEC Commands
SOTA reasoning	Architecture decisions, system boundaries, quality trade-offs, irreversible choices	<code>SPEC_takeArchDesign</code> , <code>SPEC_reviewArchDesign</code>
High Performance	Requirements analysis, intent alignment, planning, requirement updates, local design, review gates, test design, code review, correction routing, controlled patch-back	<code>SPEC_initProjectContext</code> , <code>SPEC_updateProjectContext</code> , <code>SPEC_analyzeIssue</code> , <code>SPEC_analyzeFeature</code> , <code>SPEC_analyzeAbortedUserStory</code> , <code>SPEC_clearStoryIntent</code> , <code>SPEC_makePlan</code> , <code>SPEC_updateUserStory</code> , <code>SPEC_whatsNextTask</code> , <code>SPEC_takeArchDesign</code> , <code>SPEC_reviewArchDesign</code> , <code>SPEC_updateArchDesign</code> , <code>SPEC_takeDetailDesign</code> , <code>SPEC_reviewDetailDesign</code> , <code>SPEC_updateDetailDesign</code> , <code>SPEC_reviewUserStory</code> , <code>SPEC_designUnitTests</code> , <code>SPEC_reviewImplUnitTests</code> , <code>SPEC_reviewProductCodes</code> , <code>SPEC_patchOriginalCaTDD</code>

Tier	Purpose	SPEC Commands
Flash Speed	Deterministic import, move, suspend, resume, partial-close, abort, commit, close, or small test-driven implementation/refactor steps	SPEC_importIssue, SPEC_importFeature, SPEC_importUserStory, SPEC_openUserStory, SPEC_suspendUserStory, SPEC_resumeUserStory, SPEC_partialCloseUserStory, SPEC_abortUserStory, SPEC_implUnitTests, SPEC_implProductCodes, SPEC_refactUnitTests, SPEC_commitWorks, SPEC_closeUserStory

Escalation rule: move from a lower tier to a higher one when a command reveals architecture-significant uncertainty — competing non-functional requirements, safety or security risk, real-time or embedded constraints, concurrency boundaries, data migration, compatibility matrices, or irreversible module/API ownership decisions.

cheaper model

import, move, commit, close
reversible, mechanical

↔

expensive model

design, review, plan, align
fans out into everything below it

Chain-of-Thought Patterns in Commands

Each command declares one of three reasoning patterns, chosen by decision complexity. The declaration is part of the contract: the agent knows how much thinking the step deserves.

Linear	→	fixed steps, complete inputs	(move it, commit it)
ReACT	→	inspect → act → observe, repeat	(analyze, review)
ToT	→	generate options → compare → pick	(architecture, scope)

ReACT (Reasoning + Acting)

Used when a command must inspect lifecycle state, decide, act, and observe the result iteratively.

1. **Thought** — inspect input artifacts; identify current state, gaps, conflicts
2. **Action** — create, update, review, or move the named artifact
3. **Observation** — verify the output meets the contract; check for missing traceability, unresolved questions, quality failures
4. If observation reveals a quality issue, surface it as a question or assumption and stop

Used by: SPEC_analyzeIssue, SPEC_makePlan, SPEC_reviewArchDesign, SPEC_reviewDetailDesign, SPEC_reviewProductCodes

ToT (Tree of Thoughts)

Used when a command must generate several candidate approaches, evaluate them, and select the best.

1. **Generate** — produce 2+ candidate approaches or artifact outlines
2. **Evaluate** — assess each against quality criteria
3. **Select** — choose the best; if none is clearly best, present the options to the developer
4. **Execute** — apply the selected candidate to produce the output artifact
5. **Verify** — confirm the output meets the contract

Used by: `SPEC_takeArchDesign` (comparing architectural alternatives), `SPEC_clearStoryIntent` (evaluating different scopes)

Linear (Direct Execution)

Used when the action is deterministic given complete, valid inputs.

1. Read inputs and method references
2. Preserve existing traceability, status markers, and conventions
3. Perform only this command's requested step
4. Report the artifact produced and the next recommended command

Used by: `SPEC_importIssue`, `SPEC_openUserStory`, `SPEC_commitWorks`, `SPEC_closeUserStory`

SpecFlow Artifacts

Lifecycle Artifacts (under `.catdd/spec/`)

Artifact	Purpose	Git Policy
<code>projectContext.md</code>	Shared project constitution	Commit — teammates and agents need the same facts
<code>pendingNews/*.md</code>	Imported work waiting for analysis	Commit — team visibility
<code>analyzedNews/*.md</code>	Archived raw inputs after analysis	Commit — traceability preserved
<code>todoUS/*-UserStory.md</code>	Analyzed stories ready to pick up	Commit — team backlog
<code>doingUS/*-UserStory.md</code>	Active stories in progress	Commit — cross-machine visibility
<code>doingUS/*-UserStory-Tasks.md</code>	Active task plan with checkbox tasks	Commit — explicit, checkable steps
<code>suspendUS/*</code>	Suspended stories with durable resume references	Commit — resumable across machines
<code>abortUS/*</code>	Aborted work preserved with reasons	Commit — later analysis or re-import
<code>doneUS/*-UserStory.md</code>	Completed, reviewed, committed stories	Commit — project history
<code>doneUS/*-UserStory-Tasks.md</code>	Completed task artifact	Commit — later diagnosis
<code>WorkingProcessLog.md</code>	Local work-state trace	Gitignore — personal, not team-shared

Project-Root SPEC Docs

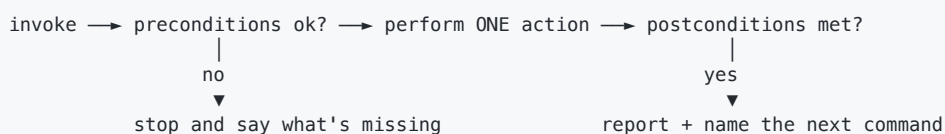
File	Purpose	Managed By
README.md	Project overview, ownership, master directories	Other SPEC steps
README_ArchDesign.md	High-level architecture, modules, dependencies	SPEC_takeArchDesign
README_DetailDesign.md	Class design, API signatures, data structures	SPEC_takeDetailDesign
README_UserStories.md	Project-scoped user stories, trace links	Other SPEC steps
README_UserGuide.md	User-facing runtime usage guidance	Other SPEC steps
README_VerifyDesign.md	Verification topology, test strategy, US/AC/TC traceability	SpecFlow + UT flows
README_ErrorDesign.md	Fault-tolerance architecture, fail-safe states	SPEC_takeArchDesign
README_ResourceDesign.md	Resource allocations, memory/CPU budgets	SPEC_takeArchDesign
README_StateDesign.md	State machines, lifecycle transitions	SPEC_takeDetailDesign
README_PerfDesign.md	Performance budgets, latency limits	SPEC_takeArchDesign
README_CompatDesign.md	Compatibility matrices, platform/toolchain versions	SPEC_takeArchDesign
README_DiagnosisDesign.md	Observability, logging, telemetry, diagnostics	SPEC_takeArchDesign
README_UsageDesign.md	Public boundaries, CLI/API contracts	SPEC_takeArchDesign

Create project-root SPEC docs only when the project needs that surface. Use templates from `slashCommands/templates/` the first time you create one.

The Command Template as a Contract

Every slash command is a contract between the developer and the CodeAgent. The developer invokes it, signalling intent. The CodeAgent executes it, applying the method faithfully. The contract has five parts:

1. **Preconditions** — inputs that must exist before invocation (API headers, existing skeletons, project context)
2. **Action** — the single step the command performs. Not "design, implement, and review" — pick one
3. **Postconditions** — outputs that must exist after execution (a designed skeleton, an implemented test, a review report)
4. **Invariants** — what must be preserved: CaTDD comment skeletons, US/AC/TC traceability, category labels, status markers
5. **Next** — what command should come next. Never leave the CodeAgent guessing



This structure is why commands are reproducible. Every invocation of `UT_designTypicalSkeleton` produces a CaTDD Typical skeleton regardless of which CodeAgent runs it, because the contract defines the output precisely — not the tool, and not the model.

Conflict Guard Rules

Px-SpecFlow defines seven guard rules so the layers cannot violate each other's boundaries:

1. **Px-SpecFlow defines lifecycle orchestration only** — CaTDD method semantics remain in methodPrompts.
2. **SPEC commands may call UT commands, but must not replace P0/P1/P2 category rules.**
3. **Do not skip SPEC_reviewUserStory after SPEC_updateUserStory** in requirement-oriented work.
4. **Do not start design when developer intent and CodeAgent intent are not cleared** for the active story.
5. **After SPEC_makePlan**, use SPEC_take*Design only for initial design work and SPEC_update*Design only for follow-up design revision.
6. **Every design-producing step must be followed by its review gate** before downstream lifecycle steps.
7. **If product intent is unclear, keep the user story open and ask the developer** instead of inventing requirements.

Rules 3, 6, and 7 are the ones that most often save a project. Each of them exists because the cheap-looking shortcut (skip the review, skip the question, revise without re-gating) produces work that looks finished and is not.

Execution Modes and the ONE-MORE-THING Stop Rule

Px-SpecFlow runs the same flow interactively or headlessly.

Mode	How it behaves
manualMode (default)	Interactive chat. One slash command at a time; the assistant asks focused questions when intent, criteria, or safety is unclear.
autonomousMode (opt-in)	Continuous CLI execution via specCodeAgentCLI or entry commands with execution_mode: autonomousMode. The agent auto-advances through safe steps using explicit file artifacts and records assumptions instead of stopping at every turn.

Safety boundary: only implementation-oriented stories support autonomousMode. If autonomy is triggered on an intent-clearing, requirement-oriented, or design-oriented story, the flow must halt, force manualMode, and require developer confirmation. Requirements analysis and architecture design need human intent and trade-offs.

```
autonomousMode allowed?
  implementation-oriented → yes
  requirement-oriented    → no   → halt, force manualMode
  design-oriented         → no   → halt, force manualMode
  intent-clearing         → no   → halt, force manualMode
```

Command-level analysis_mode (BRAINSTORM default, or AUTONOMOUS) is a local setting inside SPEC_analyzeIssue / SPEC_analyzeFeature. It does not switch the flow into autonomousMode.

Every CaTDD command enforces the **ONE-MORE-THING** universal stop rule: ask the developer when something is not certain. In manualMode, the assistant stops and asks immediately. In autonomousMode, autonomy is never a license to guess — the run must halt, output status: manual_required: ONE-MORE-THING: <question>, and wait.

Autonomous runs terminate on exactly three outcomes:

```

SPEC_closeUserStory → exit 0      (completed)
SPEC_abortUserStory → non-zero exit (aborted)
SPEC_suspendUserStory → clean exit (suspended)

```

Command Invocation in Practice

The same command file is invoked differently depending on the CodeAgent.

In Copilot Chat

```
/UT_designTypicalSkeleton
```

The Copilot prompt wrapper (generated by `scripts/makeSlashCmd4Copilot.sh`) adapts the portable Markdown command to Copilot's prompt format. You type the slash command, Copilot reads the adapted prompt, and executes the contract.

In Cline/Continue

The command prompt file is read directly. You type the command name or select it from the assistant's command palette.

In utCodeAgentCLI

```
utCodeAgentCLI --target myTestFile.ts --input spec/IOC.h --behave designTypicalSkeleton
```

The CLI maps `--behave` values to slash command behaviors. Aliases such as `reviewFuncTestsSkeleton` and `tellMeNextImplTest` resolve to their full UT command names.

Portable Placeholders

Command files use portable placeholders that adapters fill in:

- `{{feature_name}}` — name of the feature/module being tested
- `{{category}}` — CaTDD category (Typical, Edge, etc.)
- `{{source_files}}` — API headers or spec documents
- `{{test_files}}` — existing test files to read and extend
- `{{language}}` — target programming language
- `{{test_framework}}` — test framework in use
- `{{developer_goal}}` — developer intent not captured in artifacts

portable command file		adapter		tool
<code>{{feature_name}}</code>	→	filled from context	→	Copilot / Cline /
<code>{{category}}</code>		per tool format		Continue / CLI

When to Use Which Flow

Starting Point	Flow to Use
New component, no existing tests	P0-FuncTestsFlow → UT_designTypicalSkeleton
Existing demo code	P0-FuncTestsFlow → UT_convertDemoToTypical
Full functional test design	P0-FuncTestsFlow → UT_designFuncTestsSkeleton
Need architecture validation	P1-DesignTestsFlow (after P0 complete)
Need performance/capacity data	P2-QualityTestsFlow (after P0/P1 complete)
New feature request from product team	Px-SpecFlow → SPEC_importFeature → SPEC_analyzeFeature
Bug report from QA	Px-SpecFlow → SPEC_importIssue → SPEC_analyzeIssue
Already have a user story	Px-SpecFlow → SPEC_importUserStory → SPEC_openUserStory
Unsure what to do next	Px-SpecFlow → SPEC_whatsNextTask (reads current state and recommends the next action)
Resume a suspended story	Px-SpecFlow → SPEC_resumeUserStory (validates the durable resume reference)
Architecture decision needed	Px-SpecFlow → SPEC_takeArchDesign
Ready to code after design	Px-SpecFlow → SPEC_designUnitTests → P0/P1/P2 flow
Installed CaTDD harness looks broken	Px-HarnessKits → HARNESS_verifyInstallation → HARNESS_diagnoseInstallation
Reusable lesson from a verified run	Px-HarnessKits → HARNESS_evolveHarness

"I have a bug"	→ SPEC_importIssue → SPEC_analyzeIssue
"I have a feature"	→ SPEC_importFeature → SPEC_analyzeFeature
"I have a story"	→ SPEC_importUserStory → SPEC_openUserStory
"I don't know"	→ SPEC_whatsNextTask

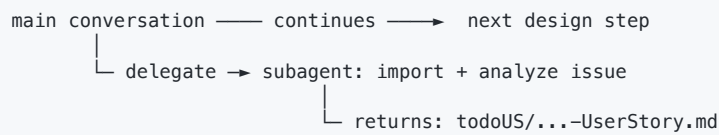
The Subagent Pattern

Some SPEC commands support background delegation via subagents.

- **When to delegate:** a new issue surfaces while other work is in progress
- **What to capture:** the source text or URL and the relevant project context
- **What to expect back:** the output file name (for example, a new story in `todoUS/`)
- **Caller behavior:** the main conversation continues without waiting

Suitable for delegation: `SPEC_importIssue`, `SPEC_importFeature`, `SPEC_analyzeIssue`, `SPEC_analyzeFeature`.

Must run synchronously: `SPEC_takeArchDesign` (its output is needed immediately for review), `SPEC_makePlan` (it determines the next step), `SPEC_commitWorks` (it must be synchronous with the developer's workspace).



The distinction is simple: **delegate intake, keep decisions**. Importing and analyzing produce an artifact you will review later. Designing, planning, and committing change what happens next.

From Commands to Automation

Slash commands are the bridge between VibeCoding's conversational flexibility and CodeAgent automation. When a developer invokes `/UT_implTestCase`, a CodeAgent reads the command contract, applies the CaTDD method, and produces a structured output. When a developer invokes `/SPEC_analyzeIssue`, a CodeAgent reads the raw issue, converts it into a user story, and moves it through the lifecycle.

The next chapter, **asyncCodeAgent**, takes this one step further: instead of waiting for the developer to invoke each command, a CaTDD-native code agent plans, executes, collects traces, and reflects — automating the loop from a goal to a verified implementation.

asyncCodeAgent

devBooks/Action/04_asyncCodeAgent.md

What Is a Code Agent?

A code agent is a tool that runs a development task by itself. It reads the method, plans the work, invokes commands, writes code, runs tests, collects results, and feeds what it learned back into the method. It is not a chatbot waiting for the next prompt — it is an execution engine that keeps going until it hits a checkpoint.

In CaTDD, code agents are the third layer of the four-layer architecture:

```

methodPrompts  ← what to do      (categories, priorities, gates)
    ↓
slashCommands  ← how to invoke it (one step per command)
    ↓
codeAgents     ← who executes it  (this chapter)
    ↓
agentSkills    ← how to package it (for other agents to load)

```

The code agent is the **intelligent execution layer**. It is where reusable CaTDD knowledge turns into opinionated, CaTDD-native execution — and where results flow back up to improve the method.

The Two Code Agents

MyCaTDD defines two agents with different scopes. One owns unit tests. The other owns the story lifecycle.

```

specCodeAgentCLI — owns the story —————
import → analyze → open → plan → route → commit → close
                        |
                        | hands off test work → utCodeAgentCLI
                        |                   owns the tests
                        |                   design → RED →
                        |                   GREEN → review

```

utCodeAgentCLI — The Unit Testing Agent

`utCodeAgentCLI` is the CaTDD-native unit-testing agent. It focuses on one concern: turning a developer goal into verified unit tests.

What it does:

- Takes a developer goal, for example "design and implement P0 functional tests for the EventQueue API"
- Plans work from CaTDD method constraints: priority order, category rules, quality gates
- Invokes standardized slash command steps (`UT_designTypicalSkeleton`, `UT_implTestCase`, and so on)
- Collects execution traces: what ran, what passed, what failed, what was blocked
- Reflects on outcomes: were the results correct, did anything unexpected happen?
- Feeds reusable patterns back into `methodPrompts` and `slashCommands`

What it does NOT do:

- It does not decide product intent — the developer owns that
- It does not redefine CaTDD method semantics — methodPrompts owns that
- It does not manage the SpecFlow lifecycle — specCodeAgentCLI owns that, or you drive it directly

specCodeAgentCLI — The Specification Agent

specCodeAgentCLI is the SpecCoding-oriented agent. It focuses on the whole module-level flow, from issue import to story closure.

What it does:

- Follows Px-SpecFlow lifecycle rules
- Reuses utCodeAgentCLI's unit-testing strengths for test design and implementation
- Organizes scenario-level verification from spec intent to executable checks
- Keeps traceability between spec flow, validation checkpoints, and implementation outcomes

The pipeline:

Together the two agents cover the full path from "we have a feature request" to "tests pass, code is committed, story is closed."

The CaTDD-Native Contract

A CaTDD-native code agent is not a generic LLM wrapper with a method prompt stapled on. It has six obligations:

1 preserve comment skeletons	@[US] @[AC] @[TC] are structure
2 follow priority discipline	P0 before P1, Typical before Edge, respect risk scoring
3 maintain traceability	TC → AC → US, across files
4 respect quality gates	stop and verify, never skip
5 honor status markers	keep ○ ● ● accurate
6 feed back upstream	new patterns → slashCommands, method gaps → methodPrompts

1. **Preserve comment skeleton contracts** — the agent must read, preserve, and update @[US], @[AC], @[TC], @[Category], and status markers. These are structural, not cosmetic.
2. **Follow priority discipline** — complete P0 before P1, Typical before Edge, and respect context-specific priority adjustments and risk scoring.

3. **Maintain US/AC/TC traceability** — every test case traces to an acceptance criterion, every acceptance criterion to a user story. The agent preserves those links across files.
4. **Respect quality gates** — the agent stops at each gate and verifies the criteria before advancing to the next priority level.
5. **Honor status markers** — keep $\circ \rightarrow \bullet \rightarrow \bullet$ markers accurate and synchronized between test files and TODO tracking sections.
6. **Feed back to upstream layers** — when it finds a pattern worth reusing, it feeds it back to slashCommands (new command flows) and methodPrompts (method refinements).

Obligations 3 and 5 are what make an agent's work reviewable by someone who was not present. Without them, a run produces tests that only the agent understands.

The Agent Workflow Checklist

The method prompt defines a four-phase Agent Workflow Checklist. This is the execution script every CodeAgent follows.

PHASE 1		PHASE 2		PHASE 3		PHASE 4
Understand	→	Design	→	Implement	→	Finalize
read-only		comments		RED → GREEN		clean up,
checkpoint 1		checkpoint 2		gate checkpoints		document,
						report

Phase 1: Understanding (Read-Only Analysis)

Before any design or implementation, the agent must:

1. **Read component interface files** — locate and read API headers; identify public functions, data structures, constants, signatures, and return types.
2. **Study existing related tests** — search for existing test files, review similar test patterns, identify reusable fixtures and helper functions.
3. **Identify dependencies and constraints** — check build files for dependencies, review design documentation, read source implementation when needed, note special build requirements.
4. **Clarify ambiguities with the developer** — if API behavior is unclear, ask specific questions. If requirements are ambiguous, propose alternatives. If context is insufficient, request specific files.

Checkpoint 1 output:

```
"I've analyzed [component]. It provides [key capabilities].

Files reviewed:
- Interface: Include/[HeaderFile.h]
- Implementation: Source/[SourceFile.c]
- Existing tests: Test/UT_[Related].cxx
- Documentation: README_[Topic].md

Key APIs: [list 3-5 main functions]
Dependencies: [list main dependencies]
Unclear aspects: [list questions if any]

Ready to proceed with test design?"
```

The agent stops here. It does not start designing until the developer confirms the understanding is right.

This stop is not politeness. Phase 1 is the cheapest place to discover that the agent read the wrong header — one question now versus a full design rewrite later.

Phase 2: Design (Comment Writing — No Code Yet)

The agent writes the test design as structured comments:

1. **Fill the OVERVIEW section** — WHAT, WHERE, WHY, SCOPE (in-scope vs out-of-scope), KEY CONCEPTS.
2. **Define Coverage Matrix dimensions** — pick 2–3 key dimensions for systematic coverage, build the table of combinations, and map combinations to candidate User Stories.
3. **Write User Stories (2–5)** — As a [role], I want [capability], so that [value]. Each story independently valuable, covering success and error scenarios.
4. **Write Acceptance Criteria (2–4 per US)** — GIVEN/WHEN/THEN format, independently testable, specific about expected behaviors and error codes.
5. **Detail Test Cases (1+ per AC)** — Name, Purpose, Brief, Steps, Expect, Notes. Classify with the priority framework, and mark every one ○ TODO.
6. **Populate the TODO tracking section** — every planned test case with its status, priority indicator, and dependency/blocker notes.

Checkpoint 2 output:

```
"Test design complete for [component]:
- Coverage: [X] User Stories, [Y] Acceptance Criteria, [Z] Test Cases
- Priority distribution: P0=[count], P1=[count], P2=[count]
- Key scenarios covered: [list 3–5 main scenarios]
- Estimated implementation effort: [rough estimate]

Shall I proceed with implementation?"
```

The agent stops here too. Design approval before implementation is mandatory.

Phase 3: Implementation (TDD Red→Green Cycle)

This is the longest phase, because it walks the full RED→GREEN cycle by priority level.

3A: Fast-Fail Six (Quick Validation)

The agent implements these six tests first, because they catch API contract violations early:

1. Null/Empty input handling
2. Zero/Negative timeout
3. Duplicate registration/subscription
4. Illegal call sequence (before init, after cleanup)
5. Buffer full/empty boundaries
6. Double-close/re-init idempotency

All six are marked ● RED, run, confirmed failing, and reported together. Six cheap tests, one report — that is the fastest way to learn whether the API contract is what everyone assumed.

3B: P0 Functional — ValidFunc (Typical + Edge)

The agent works through Typical, then Edge:

```

implement test (SETUP / BEHAVIOR / VERIFY / CLEANUP)
↓
mark ● RED → run → confirm it fails
↓
implement minimal production code
↓
run → confirm GREEN → mark ● GREEN → commit
↓
next test

```

3C: P0 Functional — InvalidFunc (Misuse + Fault)

The same cycle, for Misuse then Fault.

Gate P0 checkpoint — before proceeding to P1, the agent must verify:

```

✓ all P0 ValidFunc tests GREEN          (Typical + Edge)
✓ all P0 InvalidFunc tests GREEN        (Misuse + Fault)
✓ Fast-Fail Six all passing
✓ code coverage ≥80% for tested modules
✓ no memory leaks (run with sanitizers)
✓ no critical functional bugs

```

The agent reports gate status and asks: "P0 Complete. Proceed to P1?"

3D: P1 Design-Oriented Testing (If Applicable)

State → Capability → Interaction → Concurrency, following the same RED→GREEN cycle.

Gate P1 requirements: ThreadSanitizer/AddressSanitizer clean, no deadlocks, no race conditions, interaction/collaboration contracts verified against their design source, architecture validated against design requirements.

3E: P2 Quality-Oriented Testing (If Required)

Performance → Robust → Compatibility → Configuration → Diagnosis → Security.

Gate P2 requirements: performance SLOs met, stress/soak tests passing, diagnosis/observability evidence verified where required, security protection tests GREEN where a threat model or policy applies, production readiness criteria met.

Phase 4: Finalization and Documentation

The agent:

1. Refactors tests for clarity — extract common fixtures, remove duplicates, simplify
2. Updates documentation — comments match the real implementation, obsolete TODO items removed
3. Documents known limitations or issues
4. Marks all completed tests ● GREEN
5. Provides a completion report

Final checkpoint:

```

"Testing complete for [component]:
✓ Tests implemented: [count] (P0=[n], P1=[n], P2=[n])
✓ Test coverage: [percentage]%
✓ All tests passing: [Yes/No]
▲ Known issues: [list if any]
○ Blocked items: [list if any]

Next steps: [recommendations]"

```

Agent Troubleshooting: The Six Common Issues

The method prompt defines a troubleshooting guide for agents. Six issues cover most failure modes.

compile fails	→ check includes and real signatures
design looks wrong	→ re-check the TC → AC → US chain
behavior unclear	→ search, read docs, then ask with options
test fails oddly	→ check setup, add diagnostics, check isolation
blocked	→ mark ⊗ BLOCKED, propose options, keep working
passes when RED	→ investigate before celebrating

Issue 1: Test Compilation Fails

The agent resolves it by:

1. Checking `#include` statements against the project's include patterns
2. Verifying function signatures against header files — not against its own assumptions
3. Checking for missing test utility functions
4. Asking the developer with the specific error details and what was already checked

Issue 2: Test Design Seems Incomplete or Wrong

The agent resolves it by:

1. Verifying alignment of the TC → AC → US traceability chain
2. Checking coverage matrix completeness — expected scenarios versus actual ones
3. Validating test expectations against known behavior
4. Reviewing the Fast-Fail Six checklist for missing coverage

Issue 3: Production Code Behavior Unclear

The agent resolves it by:

1. Searching for similar patterns in the codebase — error codes, AC examples, naming patterns
2. Reading component documentation in order: spec → architecture → glossary → design docs → source notes
3. Examining existing tests for a behavior specification
4. Asking the developer with 2–3 concrete alternatives, not an open-ended "what should this do?"

Step 4 is the difference between a useful question and a stall:

weak:	"What should <code>IOC_post(NULL)</code> do?"
useful:	"Either (A) return <code>IOC_RESULT_INVALID_PARAM</code> and leave the queue untouched, or (B) treat <code>NULL</code> as a no-op and return <code>SUCCESS</code> . Existing tests in <code>IOC_Test.cxx</code> assume (A). Which is correct?"

Issue 4: Test Fails Unexpectedly

The agent resolves it by:

1. Verifying the test setup is correct — initialization, preconditions, resource creation
2. Adding diagnostic output: actual vs expected values, intermediate state
3. Checking test isolation — cleanup in previous tests, global state pollution

4. Reporting findings with specific expected/actual values and the diagnostic output

Issue 5: Unable to Proceed / Blocked

The agent resolves it by:

1. Clearly stating the blocker: what is missing, the impact, and the workarounds considered
2. Documenting it in the TODO section with a  BLOCKED marker, its dependency, and an effort estimate
3. Proposing concrete next steps with options — implement the missing API first, defer blocked tests, or use a hard-coded constant with a TODO
4. Continuing with unblocked work — never waiting idle

Issue 6: Test Passes When It Should Fail (RED Phase)

The agent resolves it by:

1. Verifying the test is actually executing — add a printf, add a temporary failing assertion
2. Checking whether the feature already exists — is there production code at the expected location?
3. Verifying the assertions are meaningful, not just "something happened"
4. Updating the test design if needed — mark GREEN if the feature is genuinely complete, strengthen the test if it is too weak

A test that passes before its feature exists is never neutral news. Either the test is weak, or the code was written before the test — and both need an answer.

The Agent's "Do and Don't" Contract

DO:

- Ask clarifying questions early, in Phase 1 — before design begins
- Wait for human approval at checkpoints — never auto-advance without confirmation
- Update the TODO section immediately after each test action — status must reflect reality
- Follow strict RED→GREEN discipline — never skip the RED phase
- Commit after each GREEN achievement — small, traceable commits
- Run tests frequently and report failures immediately — no surprises at the end

DON'T:

- Skip directly to implementation without design — design is the comment skeleton
- Implement production code before writing tests — that is a TDD violation
- Let tests stay RED without addressing them — RED means "action needed"
- Batch multiple features into one test — one TC, one behavior
- Guess requirements — ask instead
- Implement P1/P2 before completing P0 — priority discipline is not negotiable

DO → checkpoint, status, RED first, small commits
 DON'T → guess, batch, skip design, skip gates, reorder priority

Trace Collection and Reflection

A CaTDD code agent is not only an executor — it is a learner. After a session it collects two kinds of records.

Execution Traces

- What commands were invoked, and in what order
- What files were read and written
- What test cases were designed and implemented
- Which passed (GREEN), which failed (RED), which are blocked (🚫)
- What errors were encountered, and how they were resolved
- What questions were asked of the developer, and what answers came back

Reflection Notes

- Were there patterns that repeated across multiple test files?
- Were there categories that proved unnecessary for this component?
- Were there categories that, with hindsight, deserved an earlier priority?
- Did the developer's answers consistently suggest a gap in the method prompt?
- Should any troubleshooting resolution become standard agent behavior?

Feedback Loop

The agent feeds reflections back into the upstream layers:

```
Agent discovers pattern      → formalize as slash command → slashCommands/
Agent discovers method gap   → refine method prompt      → methodPrompts/
Agent discovers reusable skill → package as agent skill → agentSkills/
```

This is the bidirectional improvement loop in action: every agent execution improves the method for every future execution. A run that only produced tests spent half its value.

utCodeAgentCLI Architecture

At the current repository stage, `utCodeAgentCLI` is a documented architecture and design contract; the runnable CLI implementation is in progress. The architecture separates two systems:

1. **AgentSDK** — a generic LLM agent runtime library. It knows about goals, messages, tools, sessions, permissions, traces, hooks, adapters, and execution control. **It does not know CaTDD.**
2. **utCodeAgentCLI** — a CaTDD application built on top of AgentSDK. It parses CLI arguments, resolves CaTDD behaviors to portable slash commands, injects method prompt references, preserves US/AC/TC traceability, and records CaTDD execution traces. **It is an orchestrator, not a method owner.**

utCodeAgentCLI	CaTDD application layer behavior → slash command, traces
AgentSDK	generic agent runtime goals, tools, sessions, adapters knows nothing about CaTDD

This separation is the architecture's most important risk mitigation. A convenient CLI could easily drift into duplicating CaTDD semantics, inventing category meanings, or bypassing portable slash-command contracts. Keeping AgentSDK generic and utCodeAgentCLI thin is what prevents method drift.

The Test File State Model

utCodeAgentCLI tracks each test file through a formal state model:



File State	Description
EMPTY	No CaTDD skeleton TCs exist in the file
DESIGNED	All TCs are @[Status:PLANNED] — design complete, nothing implemented
PARTIAL	Mix of PLANNED, RED, and GREEN — some implemented, some not
FULLY_RED	All TCs are RED or GREEN, none PLANNED — all test code written
ALL_GREEN	All TCs are GREEN — tests passing, coverage achieved

The CLI owns the **PLANNED → RED** transition (writing test code). The **RED → GREEN** transition is **user-owned**: the CLI reads GREEN status but never writes it. That single boundary is what keeps the TDD contract intact — the human, or human-approved production code, turns RED into GREEN.

Behavior State Contract

Every `--behave` value has a precise contract with the file state:

Behavior	Requires	Produces (TC state)	Produces (file state)
<code>design*Skeleton</code>	Any	New TCs → PLANNED	DESIGNED or PARTIAL
<code>review*Skeleton</code>	DESIGNED, PARTIAL, FULLY_RED	No change	No change
<code>tellMeNextImplTest</code>	Has ≥1 PLANNED TC	No change	No change
<code>implTestCase</code>	Target TC is PLANNED	Target TC → RED	PARTIAL or FULLY_RED
<code>implTestFile</code>	Has ≥1 PLANNED TC	All PLANNED → RED	FULLY_RED
<code>designAndImplTest</code>	Any	All TCs → RED	FULLY_RED

State preservation guarantees:

1. Never downgrade status — RED → PLANNED never happens
2. Never overwrite an implemented TC without explicit intent
3. State-mismatched behaviors exit with clear errors
4. Every state transition is recorded in the execution trace

The contract is what makes the CLI safe to run repeatedly: a re-run is not a re-write.

Core Components

Component	Responsibility
Parser	Parse developer commands: <code>--goal</code> , <code>--target</code> , <code>--input</code> , <code>--behave</code> , <code>--model-tier</code>
Planner	Decompose goals into ordered sequences of slash command invocations
Executor	Invoke slash commands against a CodeAgent runtime (model provider + tool surface)
Adapter	Adapt portable command contracts to specific model runtimes (Copilot API, Cline protocol, direct LLM)
Trace	Collect structured execution logs: commands, TCs, status transitions, timestamps, errors
Diagnostics	Validate execution quality: RED→GREEN integrity, gate compliance, traceability preservation
State	Manage agent session state: active goal, current phase, pending checkpoints
Error	Handle agent errors: model failures, malformed outputs, missing artifacts, unmet preconditions

CLI Interface Design

```
utCodeAgentCLI \
  --goal "Design and implement P0 functional tests for EventQueue" \
  --target codeAgents/utCodeAgentCLI/SysTests/UT_EventQueue.ts \
  --input spec/EventQueue.h \
  --behave designFuncTestsSkeleton \
  --model-tier high-performance
```

`--target` defines the test-space scope: one TestCase in one TestFile, one TestFile, or several TestFiles. `--input` carries source and context: an interface, protocol, schema, draft, or production source. `--behave` names a compatible UT slash-command behavior or a stable CLI alias.

Runtime Decisions

- **V1 (PoC):** TypeScript/Node.js — rapid prototyping, easy Adapter SDK integration for Copilot and similar agents
- **V2 (production):** Go — compiled single binary, zero runtime dependencies, easy distribution
- **Python:** evaluated but not selected — performance concerns for concurrent agent sessions, weaker typing for protocol contracts

User Story Hierarchy

`utCodeAgentCLI`'s own requirements are organized by role, not by feature:

Role	Stories	Focus
USER	10 stories	Guided discovery (NEW-USER) and surgical control (EXPERIENCED-USER)
INVENTOR	3 stories	Method delegation verification, traceability integrity, diagnostic proof
DEVELOPER	5 stories	Error messages, logging, interactive mode, adapter interface, reliability policy execution

USER journeys: a NEW-USER follows guided discovery (validate → design all skeletons → review all tiers → pick next → design and implement). An EXPERIENCED-USER uses surgical control (validate → single-category design → tier review → implement one TC → review implementation). Both paths produce machine-readable execution traces.

INVENTOR requirements keep the CLI a faithful CaTDD delegate: it must reference `methodPrompts` for category semantics (never hardcode them), route through `slashCommands` for portable execution (never bypass them), and produce structured trace files proving both constraints hold.

DEVELOPER requirements cover runtime quality: deterministic error messages for every known failure mode, structured logging at configurable levels, an interactive confirmation mode for review gates with a CI-safe non-interactive fallback, a clean adapter boundary for different model runtimes, and executable reliability/safety contracts.

Non-Requirements (What utCodeAgentCLI Does NOT Own)

What the CLI refuses to own matters as much as what it does:

Concern	Owned By
Define CaTDD categories, discipline rules, or method meaning	<code>methodPrompts/</code>
Define portable slash-command execution logic	<code>slashCommands/</code>
Wrap CaTDD as a generic CodeAgent skill	<code>agentSkills/</code>
Compile, run, or verify test code	User's build system
Generate production/source code	CLI produces test code only
Manage git branches, commits, or version control	User's workflow
Transition TC from RED to GREEN	User's TDD workflow

Architecturally Significant Requirements (ASRs)

Six reliability and safety requirements sit at the architecture boundary:

ASR	Requirement	Signal
ASR-R1	Retry and correction loops shall be bounded and deterministic at budget exhaustion	Explicit retry-budget owner and deterministic exhaustion route
ASR-R2	Unknown or unsupported behavior routing shall be deterministic and diagnosable	No silent coercion; explicit diagnostics fallback
ASR-R3	Failure handling shall distinguish transient and permanent classes with explicit routing	Failure taxonomy and class-specific control flow
ASR-R4	Multi-step execution shall define snapshot/rollback or compensation boundary	Step-boundary consistency model and post-failure mutation control
ASR-R5	Escalation policy shall define threshold and non-interactive behavior	Explicit escalation trigger and CI-safe abort action
ASR-R6	Shell execution shall enforce safety policy and sensitive-path protection	Allowlist execution model, sensitive-path gating, redaction policy

These ASRs trace to executable US/AC contracts in the DEVELOPER requirements (US-DEV-05). Each maps to specific acceptance criteria — they are requirements with verification signals, not aspirations.

Architecture Decision Records (ADRs)

Key architecture decisions are formally recorded as ADRs. **ADR: Runtime Language** is the worked example:

- **V1 (PoC):** TypeScript on Node.js — the target adapter ecosystem (Copilot SDK, MCP, OpenCode) is Node/TypeScript-native, so TS/Node gives first-class adapters with no cross-language bridge
- **V2 (production):** Go — pre-selected for production distribution because of its static single-binary output; to be confirmed by a follow-up ADR when production scope opens
- **Python:** evaluated and not selected — scripting velocity does not outweigh adapter integration cost for a PoC

The ADR records the whole decision: issue, decision, status, an alternatives comparison matrix (5 criteria across 3 languages), argument, implications, and traceability to the source story and affected artifacts. That is DDD's knowledge crunching applied to architecture decisions.

Pipeline Integration

```
Developer goal → utCodeAgentCLI
├── Read methodPrompts/ (method constraints)
├── Read slashCommands/ (execution units)
├── Plan: decompose goal into command sequence
├── Execute: invoke commands via model runtime
├── Trace: collect structured execution logs
├── Reflect: analyze results, identify patterns
└── Output: verified tests + trace log + feedback to layers
```

specCodeAgentCLI Architecture

`specCodeAgentCLI` orchestrates the larger lifecycle. It follows Px-SpecFlow and reuses `utCodeAgentCLI` for the unit-testing phases.

Layer Contract

Concern	Owned By
SpecCoding lifecycle rules	Px-SpecFlow (in <code>slashCommands/flows</code>)
Lifecycle orchestration and sequencing	<code>specCodeAgentCLI</code>
Unit test design and implementation	<code>utCodeAgentCLI</code> (invoked as a sub-agent)
CaTDD method semantics	<code>methodPrompts</code>
Portable command execution units	<code>slashCommands</code>

Orchestration Flow

```

1. Receive: Import issue/feature request
2. Analyze: Convert to user story → todoUS/
3. Open: Move to doingUS/
4. Clear Intent: Align developer and agent intent
5. Plan: Classify work orientation, create task artifact
6. Route:
   └─ Requirement-oriented → updateUserStory → reviewUserStory → commit/close
   └─ Design-oriented → takeArchDesign → review → takeDetailDesign → review
   └─ Implementation-oriented → designUnitTests → implUnitTests → implProductCodes
                                   ↓
                                   reviewProductCodes
                                   ↓
                                   commitWorks → closeUserStory

```

At each step, `specCodeAgentCLI` invokes the corresponding SPEC command and passes control between three owners: the developer (product intent), the SPEC command (lifecycle rules), and `utCodeAgentCLI` (test execution).

The Sub-Agent Pattern

`specCodeAgentCLI` treats `utCodeAgentCLI` as a sub-agent:

```

specCodeAgentCLI: "Story US-5 is implementation-ready. Design and implement P0 tests."
    ↓
utCodeAgentCLI:  reads methodPrompts, reads test files, plans, executes, collects traces
    ↓
utCodeAgentCLI:  returns "3 Typical, 2 Edge, 1 Misuse tests designed.
                    3 implemented GREEN. 3 ○ TODO."
    ↓
specCodeAgentCLI: updates story state → routes to SPEC_reviewProductCodes

```

This separation keeps unit-testing logic out of the lifecycle orchestrator, and lifecycle logic out of the unit-testing agent. Each side can change without destabilizing the other.

Async Code Agent Workflow

Code agents run asynchronously: you define a goal, invoke the agent, and get results when it finishes. Three patterns follow from that.

```

FIRE-AND-FORGET  one goal, one agent, come back later
PARALLEL SESSIONS several agents, different scopes, review in a batch
PIPELINE CHAINING each finished step triggers the next one

```

Fire-and-Forget

```

Developer: utCodeAgentCLI --goal "Design all P0 skeletons for EventQueue"
Agent:     [works autonomously for several minutes]
Agent:     returns "P0 skeletons complete. 4 categories, 8 US, 14 AC, 22 TC.
                Ready for review."

```

You define the goal and go do other work. The agent comes back when it is done.

Parallel Agent Sessions

```
Session A: utCodeAgentCLI --goal "P0 tests for EventQueue"
Session B: utCodeAgentCLI --goal "P0 tests for CommandExecutor"
Session C: specCodeAgentCLI --goal "Analyze pending issues into stories"
```

Several agents run at once, each on a different scope. You review the results in a batch. The constraint is human, not technical: three sessions that all need the same architectural decision will produce three different answers.

Pipeline Chaining

```
specCodeAgentCLI (analyze issue → story in todoUS/)
  ↓ (triggered by: story moved to doingUS)
specCodeAgentCLI (take detail design → ACs designed)
  ↓ (triggered by: design review PASS)
utCodeAgentCLI (design P0 tests → implement → verify)
  ↓ (triggered by: all P0 tests GREEN)
specCodeAgentCLI (review codes → commit → close → doneUS)
```

Each agent invocation triggers the next pipeline step. Set the pipeline up once, then monitor the checkpoints.

The Agent's Relationship with Developer Checkpoints

Code agents are autonomous but not unsupervised. CaTDD embeds checkpoints where the developer must approve before the agent continues:

Checkpoint	Agent State	Developer Action
After Understanding (Phase 1)	"Ready to proceed with test design?"	Review the understanding summary. Confirm or correct.
After Design (Phase 2)	"Shall I proceed with implementation?"	Review the US/AC/TC design. Approve or request revisions.
After Gate P0	"P0 Complete. Proceed to P1?"	Review P0 results. Decide whether P1 is needed.
After Gate P1	"Architecture validated. Proceed to P2?"	Review architecture test results. Decide whether P2 is needed.
After Implementation	"Testing complete. Ready to commit?"	Review the final report. Commit or request changes.

```
agent runs → checkpoint → developer decides → agent continues
              ↑
              | never auto-advance past here |
              ↓
```

The agent honors the checkpoint contract: it stops at each gate and waits for developer input. It does not auto-advance. That is what preserves the developer's ownership of quality decisions — the agent moves fast, the human stays accountable for what "done" means.

Agent Skills vs Code Agents

Two related concepts, two different audiences:

Concept	What It Is	Where It Lives
agentSkills	Reusable capability packages for generic CodeAgents (Copilot, Cline)	<code>agentSkills/</code> — skill definitions + generated packages
codeAgents	CaTDD-native CLI agents with built-in methodology knowledge	<code>codeAgents/</code> — architecture + future implementation

```
agentSkills: "Here is a skill package. Any CodeAgent can load this and follow CaTDD."
codeAgents: "We are CaTDD-native. We know the method, we plan autonomously,
            and we feed learnings back."
```

- **agentSkills** is for developers using Copilot, Cline, or Continue who want their existing agent to follow CaTDD.
- **codeAgents** is for the CaTDD project itself — first-class automation that depends on no third-party agent.

A Copilot user loads the `comment-alive-test-driven-development` skill pack. A CaTDD-native user invokes `utCodeAgentCLI` directly. Both follow the same method and produce the same structured output. The difference is the execution engine.

The Future of Code Agents

The repository currently documents the intended layer contract for both agents; the runnable implementations are in design and development. The architecture includes:

- **TypeScript-based V1 (PoC)** for `utCodeAgentCLI` — rapid iteration on AgentSDK integration
- **Go-based V2 (production)** for `utCodeAgentCLI` — compiled binary, no runtime dependencies
- **SpecFlow-native execution** for `specCodeAgentCLI` — lifecycle orchestration with built-in traceability

The key insight: code agents are not an afterthought to the method. They are the **destination** of the CaTDD evolution path.

```
Manual execution    (methodPrompts)  - you read the method, you follow it
    ↓
Commandized execution (slashCommands) - you invoke one step at a time
    ↓
Agent-driven execution (codeAgents)   - the agent plans and runs the steps
    ↓
Packaged reusable capability (agentSkills) - other agents load and reuse it
```

Each layer automates more of the previous layer's manual work. The code agent is where manual becomes autonomous.

From Automation to Integration

Code agents automate the CaTDD workflow. But automation is built on enduring principles — the Knowledge Book of Software Engineering. The next chapter, **applyClassicSWE**, shows how TDD, BDD, and DDD find their full expression in the LLM era through CaTDD's synthesis of all three disciplines.

The agent generates the tests. The pipeline runs them. The team reviews them. CaTDD does not replace your engineering culture — it strengthens it with structured, traceable, LLM-readable verification design.

applyClassicSWE

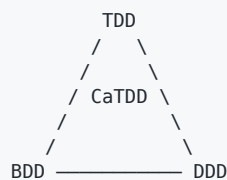
devBooks/Action/05_applyClassicSWE.md

The Knowledge Book of Software Engineering

Software engineering is not a pile of frameworks. It is a body of knowledge that survived decades because it keeps being true about the way humans build complex systems. Three disciplines in that body matter most here:

Discipline	Core Question	Signature Practice
TDD — Test-Driven Development	How do we prove that code works?	Write a failing test first, implement to pass, refactor
BDD — Behavior-Driven Development	How do we connect requirements to code?	GIVEN/WHEN/THEN specification, executable specifications
DDD — Domain-Driven Design	How do we model complex business domains?	Ubiquitous Language, Bounded Contexts, Aggregates, Entities

They form a triangle, and each side answers a different part of the lifecycle:



BDD says what to build. (specify behavior)
 DDD says how to model it. (model the domain)
 TDD says how to prove it. (verify behavior)

The LLM Era Changes Everything — and Nothing

Large Language Models changed how we write code. They did not change what makes software good:

- **Correctness** still matters — an LLM can generate code that looks right and is subtly wrong
- **Traceability** still matters — who asked for this feature, and how do we know it is done?
- **Maintainability** still matters — code that only the LLM understands is unmaintainable
- **Domain understanding** still matters — the LLM does not know your business rules unless you write them down

LLM × no structure = more chaos, produced faster
 LLM × structure = more verified work, produced faster

The classic disciplines supply that structure. CaTDD sits exactly at this intersection: **it translates the**

Knowledge Book into LLM-readable, comment-alive verification design. It does not replace TDD, BDD, or DDD. It synthesizes them for the LLM era.

TDD in the LLM Era

Classical TDD: The Foundation

Test-Driven Development, formalized by Kent Beck, has three laws:

1. Write a failing test before writing production code
2. Write only enough test to fail
3. Write only enough production code to pass

```
RED      write a test → it fails (the feature does not exist yet)
GREEN    write minimal production code → the test passes
REFACTOR improve code and test structure, tests stay green
```

TDD is not really about testing. It is about **design**. Writing the test first forces you to decide the interface, the behavior, and the contract before you write the implementation. The test is your API's first client.

TDD's Limitations Before LLMs

Classical TDD had two structural gaps:

1. **No design artifact** — the test is the design. But a test shows WHAT to verify, not WHY it matters. The business intent — who asked for this, what value it provides — stayed in the developer's head instead of in the file.
2. **A manual effort ceiling** — writing tests one at a time is disciplined but slow. Each test is conceived, written, debugged, and verified by hand. The number of useful scenarios always exceeds the hours available to write them.

CaTDD: TDD Enhanced for the LLM Era

CaTDD closes both gaps by embedding **design intent** and **LLM-readability** into the TDD cycle:

Classical TDD	CaTDD Enhancement
Test is the design	Comment skeleton IS the design — US/AC/TC structure makes business intent explicit
What to test is implicit	Priority framework (P0→P3) makes coverage strategy explicit
RED→GREEN→REFACTOR	○TODO→●RED→●GREEN, with quality gates between priority levels
One test at a time, manually	LLM reads the design skeleton, generates test code, implements minimal production code
Refactoring is ad hoc	TC-by-TC review, refactoring triggered by review gate failures
Tests are code artifacts	Tests are living design documents — comments evolve with the code

The LLM does not just write tests faster. It writes tests that are **traceable to design intent**, because the design intent lives in the same file the LLM reads.

The CaTDD TDD Cycle (with LLM)

1. DESIGN: write the comment skeleton (US/AC/TC) – this IS the design
2. TC SELECTION: LLM reads the skeleton, selects the next TC by priority
3. RED: LLM generates test code for the TC → ○→● → test fails
4. GREEN: LLM generates minimal production code → ●→● → test passes
5. REVIEW: LLM reviews the implementation against the design contract
6. REPEAT: advance to the next TC
7. GATE: stop at P0/P1/P2 gates, verify criteria, ask the developer

Steps 3–5 happen in a tight LLM loop. Steps 1, 2, 6, and 7 are where human judgment operates. The LLM handles the mechanical TDD work; the developer handles the design judgment.

Why TDD Matters More in the LLM Era

You might think: if an LLM can generate code, why bother with TDD at all? Three reasons:

- **LLMs generate plausible code faster than correct code.** Without a failing test, the LLM has no feedback. Without a passing test, you have no confidence.
- **LLMs lose context between conversations.** The test file is the context. The comment skeleton is the durable record of what the code should do, readable by the next session — and the next developer.
- **LLMs are optimistic generators.** They assume their code works. TDD forces verification: every GREEN test is a checkpoint the model cannot hallucinate past.

without TDD:	generate → looks right → ship → learn the truth in production
with TDD:	generate → RED → GREEN → review → ship

TDD is not less important in the LLM era. It is **more** important. It is the grounding wire that keeps generated code from floating into plausible-but-wrong territory.

BDD in the LLM Era

Classical BDD: From Requirements to Executable Specifications

Behavior-Driven Development, pioneered by Dan North, bridges business stakeholders and developers:

Business language:	"As a customer, I want to withdraw cash so I can buy things"
BDD specification:	Scenario: Withdraw from account
	Given the account has \$100
	When the customer withdraws \$20
	Then the balance is \$80
Automated test:	step definitions map Given/When/Then to test code

BDD's core insight: **specifications should be executable**. The GIVEN/WHEN/THEN scenario is both the business requirement and the automated test. When the test passes, the requirement holds. When it fails, the requirement is broken.

BDD's Limitations Before LLMs

Classical BDD needed three artifacts, maintained by humans:

1. **Feature files** (Gherkin `.feature`) — written by product owners or BAs
2. **Step definitions** (code mapping Gherkin steps to actions) — written by developers
3. **Production code** — written by developers

Three artifacts in two languages created maintenance friction. Feature files went stale, step definitions grew fragile, and the vision of "executable specifications" quietly became "specifications that once were executable."

CaTDD: BDD Embedded in Test Files

CaTDD puts BDD directly into the test file as structured comments, which removes the Gherkin-to-code gap:

Classical BDD:	CaTDD BDD:
Feature: CashWithdrawal.feature	UT_Account.cxx:
Scenario: Withdraw from account	// US-1: As a customer,
Given the account has \$100	// I want to withdraw cash
When the customer withdraws \$20	// So that I can buy things.
Then the balance is \$80	// AC-1: GIVEN account has \$100,
	// WHEN customer withdraws \$20,
Step definitions: CashWithdrawal.java	// THEN balance is \$80.
@Given("account has \$100")	// TC-1: verifyWithdraw_bySufficientBalance
public void accountHas100() { ... }	// _expectReducedBalance
Test runner: Cucumber/JBehave	TEST(Account, verifyWithdraw_
	bySufficientBalance_
	expectReducedBalance) { ... }

The US/AC/TC chain is the BDD specification, living in the test file instead of a separate `.feature` file. Step definitions and test implementations become one artifact: the TEST function. The gap between specification and verification disappears.

BDD as the CaTDD Acceptance Criteria Contract

Every CaTDD Acceptance Criterion follows the BDD format:

```
AC-n: GIVEN [preconditions and context],
      WHEN [specific action or event],
      THEN [expected observable outcome],
      AND [additional outcome if any].
```

This is not a coincidence. GIVEN/WHEN/THEN is the **canonical testable condition** in CaTDD. A living documentation reconciliation checklist keeps it honest.

BDD Reconciliation Checklist

Set up automated reconciliation tests that assert consistency between source code, BDD specifications, and external documentation:

- Every CaTDD AC maps to one or more TC implementations — no orphan acceptance criteria, no undocumented test cases
- AC descriptions stay distinct from step-specific implementation detail — a TC verifies one AC instead of duplicating another AC's steps
- Clean Markdown reports summarize which ACs are tested, which are planned, and which are blocked

Why BDD Matters More in the LLM Era

LLMs are pattern matchers, not business analysts. They produce convincing GIVEN/WHEN/THEN structures without being able to judge whether the THEN clause matches the real business rule. BDD inside CaTDD supplies the guardrail by splitting ownership:

developer writes the AC	the business rule, in GIVEN/WHEN/THEN
LLM generates the TC	the concrete test that verifies the AC
developer reviews the TC	does this test verify what the AC states?
LLM implements the code	the 4-phase test implementation
test passes	the AC is satisfied

The developer owns the business rule. The LLM owns the verification mechanism. That division is BDD optimized for the LLM era.

DDD in the LLM Era

Classical DDD: Modeling Complex Domains

Domain-Driven Design, introduced by Eric Evans, models complex business domains in software. Its core concepts:

Ubiquitous Language — one shared language between domain experts and developers, used consistently in code, conversation, and documentation. If the business says "policy," the code has a `Policy` class, not an `InsuranceMathHelper`.

Bounded Contexts — a boundary inside which a model applies. "Account" means different things in Billing (a payment account) and CRM (a customer account). Each context keeps its own model.

Strategic Design — the relationships between contexts: Core Domain (your competitive advantage), Supporting Subdomains (necessary, not differentiating), Generic Subdomains (buy, don't build).

Tactical Design — the building blocks inside a context: Entities (identity over attributes), Value Objects (attributes over identity), Aggregates (consistency boundaries), Domain Events (something happened), Repositories (retrieve aggregates), Services (stateless operations).

DDD's Limitations Before LLMs

DDD depended on deep domain knowledge that was hard to transfer to new team members:

- **Ubiquitous Language was fragile** — one developer wrote `Customer`, another `Client`, a third `Account`, all for the same concept. The model fragmented.
- **Bounded contexts were implicit** — they lived in architects' heads and on whiteboards, not in the code.
- **Model evolution was slow** — refactoring a domain model meant updating documentation, tests, production code, and everyone's understanding, by hand.

CaTDD: DDD as an LLM-Readable Domain Model

CaTDD brings DDD into the LLM era through four mechanisms.

1. The OVERVIEW Section as Bounded Context Declaration

Every CaTDD test file starts with an OVERVIEW that declares its bounded context:

```

/**
 * [WHAT] This file verifies event queue behavior
 * [WHERE] in the IOC Event Subsystem module
 * [WHY] to ensure reliable asynchronous event delivery
 *
 * SCOPE:
 * - In scope: Event posting, queue capacity, consumer callback
 * - Out of scope: Event persistence (see UT_EventPersistence.cxx)
 *
 * KEY CONCEPTS:
 * - Conet vs Conles: Connection-oriented vs connection-less event modes
 * - EvtDescQueue: Fixed-capacity event descriptor queue
 * - CbExecCmd_F: Callback function for immediate command processing
 */

```

That OVERVIEW is DDD's bounded context expressed as an LLM-readable comment. It tells the LLM: inside this file these are the concepts, and these other concepts belong to other files.

2. US/AC/TC as Executable Domain Specifications

Each User Story maps onto DDD's strategic design:

DDD Strategic Classification	CaTDD Priority Mapping
Core Domain (your competitive advantage) invest most effort here	→ P0 Functional Typical, Edge, Misuse, Fault business-critical workflows
Supporting Subdomain (necessary, not unique) invest enough to keep it running	→ P1 Design State, Capability, Interaction, Concurrency – architecture validation
Generic Subdomain (buy, don't build) minimal custom investment	→ P2 Quality Performance, Robust, Compatibility, Configuration, Diagnosis, Security
Documentation / Education (demos, examples, tutorials)	→ P3 Addons Demo/Example

The priority framework is not only a test order — it is a **domain classification**. P0 tests verify your core domain, P1 tests validate architecture, P2 tests check quality attributes. The framework tells the LLM where to focus first, instead of treating every test as equally important.

3. Design Skeletons as Ubiquitous Language

The skeleton establishes Ubiquitous Language inside the test file:

```

// @[Class]: P0 Functional / ValidFunc
// @[Category]: Typical
// @[Intent]: Prove the core event posting and delivery workflow
// @[UseWhen]: Producer and consumer are valid, queue has capacity
// @[AvoidWhen]: Queue is full, consumer is blocking, or producer is invalid
// @[US]: US-1 (event posting), US-2 (event delivery)
// @[AC]: AC-1 through AC-4
// @[TC]: TC-1 verifyEventPost_byAvailableCapacity_expectEventQueued
//         TC-2 verifyEventDelivery_byCallbackConsumer_expectEventProcessed

```

"Producer," "consumer," "queue," "capacity," "blocking" — these are not generic test words. They are the domain's Ubiquitous Language, and the LLM reads them before generating any code.

4. SpecFlow Artifacts as Domain Knowledge

Px-SpecFlow's lifecycle artifacts preserve domain knowledge in version-controlled files:

```
.catdd/spec/projectContext.md → domain vision, bounded contexts, conventions
README_ArchDesign.md         → strategic design, context mapping
README_DetailDesign.md       → tactical design, aggregate boundaries
README_VerifyDesign.md       → domain verification topology
```

These are DDD's knowledge-crunching output in Markdown form: readable by LLMs, reviewable by humans, and durable across code changes.

DDD Reconciliation Checklist

Use these checkpoints to verify DDD alignment:

- Scan test files for domain term consistency — is the same concept named the same way everywhere?
- Verify bounded context boundaries — does `UT_EventQueue.cxx` reference only EventQueue concepts, or does it leak `CommandExecutor` concepts?
- Check that the OVERVIEW declares its bounded context clearly enough for an LLM to understand
- For each aggregate root, verify a corresponding test file with US/AC/TC coverage exists

Why DDD Matters More in the LLM Era

LLMs are domain-agnostic. They know syntax, not semantics. They can generate code for any domain and understand none of them. DDD fills that gap:

- **Ubiquitous Language gives the LLM consistent terminology** — it reads "event producer," "event consumer," "EvtDescQueue" and learns the domain vocabulary from the comments
- **Bounded Contexts stop the LLM from mixing models** — a `Billing.Account` test file uses billing terms, a `CRM.Account` file uses CRM terms, and the OVERVIEW says which one you are in
- **Strategic Design tells the LLM what to prioritize** — P0 core domain, P1 supporting, P2 generic, which is exactly how it should spend a limited context window

Without DDD, the LLM treats every test file as a flat, interchangeable artifact. With DDD embedded in CaTDD comments, it understands the domain architecture.

CaTDD: The Synthesis of TDD + BDD + DDD

CaTDD is not a fourth methodology stacked on top of three others. It is the **synthesis** — the way all three work together in one LLM-readable artifact: the test file.

```

THE CaTDD TEST FILE

OVERVIEW: Domain Context Declaration (DDD)
WHAT, WHERE, WHY, SCOPE, KEY CONCEPTS
|
US: User Stories – Business Intent (BDD)
As a [role], I want [capability], So that [value]
| traces to
AC: Acceptance Criteria – Executable Specs (BDD)
GIVEN [context], WHEN [action], THEN [outcome]
| traces to
TC: Test Cases – Verification Design (TDD + BDD)
@[Name], @[Purpose], @[Brief], @[Steps], @[Expect]
| implements
TEST CODE: Implementation (TDD)
SETUP → BEHAVIOR → VERIFY → CLEANUP
Status: ○ TODO → ● RED → ● GREEN

```

One file, four kinds of knowledge:

- **DDD's bounded context** — the OVERVIEW section
- **BDD's executable specifications** — the US/AC chain with GIVEN/WHEN/THEN
- **TDD's verification cycle** — TC implementation with RED→GREEN markers
- **DDD's strategic prioritization** — the P0→P3 framework mapping to core/supporting/generic

The LLM as a Knowledge Book Reader

The Knowledge Book teaches a handful of principles. CaTDD turns each one into something an LLM can act on:

Principle	What It Means	How CaTDD Applies It
Design before code	Understand what to build before building it	Comment skeletons ARE the design
Verify before releasing	Prove correctness before shipping	RED→GREEN cycle + quality gates
Speak the domain language	Use business terms in code	OVERVIEW + US/AC use domain vocabulary
Bound the model	One model per context, don't mix them	One test file per aggregate/bounded context
Test from the outside in	Verify behavior, not implementation	GIVEN/WHEN/THEN focuses on observable outcomes
Small, reversible changes	Commit small, roll back easily	One TC at a time, RED→GREEN commits
Knowledge in the code	Don't let knowledge live only in heads	Comment-alive design preserves decisions

The LLM Reads the Knowledge Book Through CaTDD

When you share a CaTDD test file with an LLM, you are handing it the Knowledge Book encoded in comments:

LLM reads	OVERVIEW section	→ learns the bounded context and concepts
LLM reads	US section	→ learns business value and actor roles
LLM reads	AC section	→ learns the executable specifications (BDD)
LLM reads	TC section	→ learns what to verify and how
LLM reads	TODO tracking section	→ learns progress and next actions
LLM reads	TEST CODE section	→ learns the implementation patterns
LLM reads	@[Category] tags	→ learns priority and risk level

The LLM does not need to know the term "DDD" to benefit from DDD. It reads a file that declares its bounded context in plain English, uses consistent domain terms, and follows priority rules. The methodology is embedded in the artifact.

BDD → CaTDD: From Feature Files to Comment Chains

Classical BDD keeps Gherkin `.feature` files separate from test code:

```

Feature: Cash Withdrawal
  As a customer
  I want to withdraw cash from my account
  So that I can make purchases

  Scenario: Successful withdrawal
    Given my account has a balance of $100
    When I withdraw $20
    Then my balance should be $80
    And the ATM should dispense $20

  Scenario: Insufficient funds
    Given my account has a balance of $50
    When I withdraw $100
    Then the withdrawal should be rejected
    And my balance should remain $50

```

CaTDD converts it into a comment chain inside the test file:

```

// US-1: As a customer,
//       I want to withdraw cash
//       So that I can make purchases.
//
// AC-1: GIVEN account has balance $100,
//       WHEN customer withdraws $20,
//       THEN balance is $80,
//       AND ATM dispenses $20.
//
// AC-2: GIVEN account has balance $50,
//       WHEN customer withdraws $100,
//       THEN withdrawal is rejected,
//       AND balance remains $50.

// [@AC-1,US-1] P0 Functional / Typical
// TC-1:
//   @[Name]: verifyWithdraw_bySufficientBalance_expectReducedBalance
//   @[Purpose]: Ensure valid withdrawals are processed correctly
//   @[Brief]: Set up account with $100, withdraw $20, verify balance $80
//   @[Expect]: Balance is $80, dispense event is triggered

TEST(AccountWithdraw, verifyWithdraw_bySufficientBalance_expectReducedBalance) {
  //===SETUP===
  Account acc(100);

  //===BEHAVIOR===
  auto result = acc.withdraw(20);

  //===VERIFY===
  ASSERT_TRUE(result.success);
  ASSERT_EQ(80, acc.balance());
  ASSERT_TRUE(result.dispensed);
}

```

The structure of BDD survives; the separation between specification and test does not. GIVEN/WHEN/THEN lives in the AC comment, and the scenario implementation sits right below it. An LLM reading the file sees the whole chain — business requirement to executable verification — in one continuous context.

DDD → CaTDD: From Domain Models to Comment Skeletons

Classical DDD produces a domain model as class diagrams, context maps, and design documents:

```

DDD artifacts:
- Bounded Context: Account Management
- Aggregate Root: Account
- Entity: Transaction
- Value Object: Money
- Domain Service: TransferService
- Ubiquitous Language: balance, withdraw, deposit, transfer, overdraft

```

CaTDD embeds that model in the test file's OVERVIEW and design skeletons:

```
//=====
//====->BEGIN OF OVERVIEW=====
/**
 * [WHAT] This file verifies Account aggregate behavior
 * [WHERE] in the Account Management bounded context
 * [WHY] to ensure correct balance operations and transfer integrity
 *
 * SCOPE:
 * - In scope: Account withdrawal, deposit, transfer logic
 * - Out of scope: User authentication (see UT_Authentication.cxx)
 * - Out of scope: Notification delivery (see UT_Notification.cxx)
 *
 * KEY CONCEPTS:
 * - Account (Aggregate Root): Manages balance and transaction history
 * - Money (Value Object): Immutable amount + currency pair
 * - Transaction (Entity): Recorded entry in account history
 * - TransferService (Domain Service): Coordinates two-account transfer
 */
//====->END OF OVERVIEW=====
```

From this OVERVIEW the LLM learns: this file covers one bounded context; the aggregate root is Account, so tests target Account behavior; Money is a value object, so tests verify value equality rather than identity; Transaction is an entity, so tests verify it gets recorded; and authentication and notifications are explicitly out of scope.

The P0→P3 Framework as Strategic DDD

The priority framework maps directly onto DDD's strategic classification:

DDD Strategic Classification	CaTDD Priority Mapping
Core Domain (your competitive advantage) — what makes money — unique to your business — invest most effort here	P0 Functional Typical, Edge, Misuse, Fault — business-critical workflows — domain-specific behavior — complete before any other testing
Supporting Subdomain (necessary but not unique) — supports the core domain — could be outsourced — invest enough to keep running	P1 Design State, Capability, Interaction, Concurrency — architectural validation — lifecycle and FSM — complete after P0
Generic Subdomain (buy, don't build) — common to all businesses — use existing solutions — minimal custom investment	P2 Quality Performance, Robust, Compatibility, Configuration, Diagnosis, Security — non-functional quality — platform compatibility — complete after P1
Documentation / Education (demos, examples, tutorials)	P3 Addons Demo/Example

A CaTDD CodeAgent reading this mapping learns three things:

- "P0 tests are what make this domain valuable — invest heavily"
- "P1 tests support the core — test the architecture thoroughly"
- "P2 tests are infrastructure — verify quality, don't over-test"

That prevents the most common LLM anti-pattern in testing: treating every test as equally important and optimizing for a coverage number instead of for the domain.

Living Documentation in the LLM Era

The Knowledge Book says documentation must evolve with code. CaTDD makes that concrete through **comment-alive design**.

The Living Glossary

CaTDD test files establish a Ubiquitous Language through consistent domain terminology:

- Scan only the files inside the target bounded context
- Filter out test utilities, framework configuration, and boilerplate
- Map test file names to domain terms — `UT_Account.cxx` means the Account aggregate
- Let AC comments define term behavior — "GIVEN account has balance \$100" is where "balance" gets defined

The LLM builds its glossary from the OVERVIEW section and the AC comments. No separate glossary file required.

Living Diagrams

SpecFlow's architecture-oriented SPEC docs act as living diagrams:

<code>README_ArchDesign.md</code>	→	bounded context map
<code>README_DetailDesign.md</code>	→	aggregate boundaries and class relationships
<code>README_VerifyDesign.md</code>	→	test topology and verification coverage

These are text documents, which is why LLMs can parse them, but they can also be rendered as Mermaid or PlantUML diagrams when a human wants the picture. The text stays the source of truth; the diagram is a view.

BDD Reconciliation in CaTDD

In classical BDD, reconciliation means checking that feature files, step definitions, and production code all agree. In CaTDD the gap is gone, so reconciliation gets simpler:

feature file	=	AC comment in the test file
step definition	=	TC comment + TEST function in the same file
production code	=	written in the GREEN phase for that TC

The checks become:

- Every AC has at least one TC tracing to it
- Every TC's assertions actually verify the AC it claims to verify
- Every production code change was driven by a RED TC

All three can be automated by parsing the comment markers. No external BDD tool required.

Applying the Knowledge Book: A Complete Example

Here is one feature traced through all three disciplines and both roles.

1. BDD: Express the Requirement

```

US-1: As a customer,
      I want to transfer money between my accounts
      So that I can manage my funds across accounts.

AC-1: GIVEN checking account has $500 and savings account has $200,
      WHEN customer transfers $100 from checking to savings,
      THEN checking balance is $400,
      AND savings balance is $300.

AC-2: GIVEN checking account has $100,
      WHEN customer transfers $200 from checking to savings,
      THEN transfer is rejected,
      AND checking balance remains $100,
      AND savings balance is unchanged.

```

2. DDD: Model the Domain in OVERVIEW

```

/**
 * [WHAT] This file verifies cross-account money transfer
 * [WHERE] in the Account Management bounded context
 *
 * KEY CONCEPTS:
 * - Transfer (Domain Operation): Moves Money between two Accounts
 * - Money (Value Object): amount + currency, immutable
 * - Account (Aggregate Root): owns balance and transaction history
 * - TransferLimit (Policy): maximum per-transfer amount
 */

```

3. TDD: Design the Test Cases

```

// P0 Functional / Typical – Core Domain
// TC-1: verifyTransfer_byBasicTransfer_expectUpdatedBalances
//   @[Purpose]: Verify the fundamental transfer operation
//   @[Brief]: Set up two accounts, transfer $100, verify both balances

// P0 Functional / Edge – Domain boundaries
// TC-2: verifyTransfer_byExceedingBalance_expectRejected
//   @[Purpose]: Verify overdraw protection
//   @[Brief]: Attempt to transfer more than available balance

// P0 Functional / Edge – Domain precision
// TC-3: verifyTransfer_byZeroAmount_expectNoOperation
//   @[Purpose]: Verify zero-amount transfer is a no-op
//   @[Brief]: Transfer $0, verify no balance change

// P0 Functional / Misuse – Domain misuse
// TC-4: verifyTransfer_bySameAccount_expectRejected
//   @[Purpose]: Prevent transfer to the same account
//   @[Brief]: Set source and destination to same account

```

4. LLM: Implement the TDD Cycle

The LLM reads the US/AC/TC design — BDD specification plus DDD domain model — and implements each case:

```

TC-1: RED → implement TransferService.transfer($100) → GREEN
TC-2: RED → implement InsufficientFunds check → GREEN
TC-3: RED → implement zero-amount guard clause → GREEN
TC-4: RED → implement same-account check → GREEN

```

Each implementation is bounded by the domain: the LLM knows from the OVERVIEW that Transfer is a domain operation, Money is a value object, and Account is an aggregate root, so the generated code uses those names.

5. Review: Verify Against the Knowledge Book

- ✓ BDD: every AC has one or more TCs – executable specifications exist
- ✓ DDD: OVERVIEW declares the bounded context – the domain model is explicit
- ✓ TDD: all TCs followed RED→GREEN – verification is complete
- ✓ Synthesis: changes trace US → AC → TC → Code → Git commit

The Knowledge Book Applied

Knowledge Book Principle	CaTDD Practice
TDD: test first, code second	Comment skeleton first, LLM generates TC → RED → GREEN
BDD: specify behavior, not implementation	US/AC/TC chain is the executable specification
DDD: speak the domain language	OVERVIEW + US/AC use domain terms consistently
DDD: bound the model	One test file per bounded context / aggregate root
TDD: small, verified steps	One TC at a time, ○→●→● markers
BDD: keep specs and tests together	AC comments and TEST functions in the same file
DDD: strategic prioritization	P0 (core) → P1 (supporting) → P2 (generic) → P3 (docs)
SWE: knowledge in the code	Comment-alive design preserves decisions in the artifact

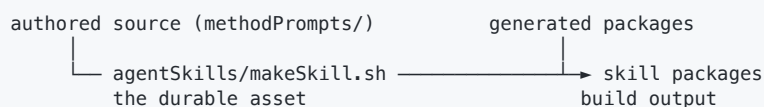
The LLM era does not make TDD, BDD, or DDD obsolete. It makes them **operational**. CaTDD is the operating system that runs all three on LLM hardware.

Packaging the Knowledge Book as Agent Skills

The Knowledge Book does not live only in books. CaTDD packages it into two reusable agent skills that any CodeAgent can load.

comment-alive-test-driven-development packages CaTDD as a reusable capability. When a Copilot user loads the skill and says "use CaTDD," the agent has the full Knowledge Book at its disposal: the complete `CaTDD_methodPrompt.md`, the `CaTDD_designAndImplTemplate.cxx` template, and the `slashCommands` flow materials. The skill tells the agent WHO uses it, WHAT to create (a test file as a living design document), WHEN to apply it, WHERE to place output, WHY it matters (structured comments give LLMs the context to generate correct code), and HOW to execute in four phases: Scope → Design → Implement → Validate.

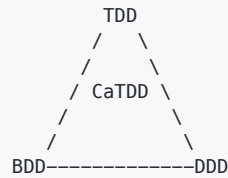
user-story-centered-spec-coding packages the SpecCoding lifecycle as a reusable workflow. It tells the agent how to move work through `pendingNews` → `todoUS` → `doingUS` → `doneUS`, with `abortUS` preserving unsafe active stories for later analysis, CaTDD as the default testing method, and typical TDD as an alternative when requested. The skill bundles the complete `Px-SpecFlow` lifecycle: the flow document, all 27 `SPEC_*` command files, project-root README SPEC templates, and CaTDD method references as the default testing engine.



Both skills are generated from authored source by `agentSkills/makeSkill.sh`. The authored source is

the durable asset; the generated packages are build output. That separation preserves the Knowledge Book's integrity: the method changes in one place, and the change propagates to every skill package through the build script.

Conclusion: The Triangle of Classic SWE



TDD proves behavior.
BDD specifies behavior.
DDD models behavior.
CaTDD synthesizes all three into
one LLM-readable test file.

The Knowledge Book of Software Engineering is not a museum of past practices. It is the operating manual for building correct, maintainable, domain-aligned software — with or without LLMs. CaTDD reads that manual and translates it into comment-alive verification design that LLMs, developers, and code agents can all understand, execute, and improve.

Comments is Verification Design. LLM Generates Code. Iterate Forward Together.

beyondXyzSpec

devBooks/Action/06_beyondXyzSpec.md

The Spec-Driven Development Landscape

The LLM era produced three major open-source approaches to specification-driven development. All three believe in **spec before code**. They differ in what they put at the center — and in where they stop.

	GitHub Spec Kit	OpenSpec	CaTDD Px-SpecFlow
Centers on	Documents (spec.md, plan.md)	Changes (openspec/changes/)	User Stories (US lifecycle)
Stars	111k	54k	—
CLI	Python (specify)	TypeScript/Node.js (openspec)	Markdown prompt commands (agent-agnostic)
Agent integrations	30+	25+	Copilot, Cline, Continue, utCodeAgentCLI
Testing methodology	Not embedded	Not embedded	Embedded — UT P0/P1/P2 with TDD

Spec Kit	centers on documents	spec.md → plan.md → tasks.md
OpenSpec	centers on changes	changes/<change-name>/
CaTDD	centers on the story	pendingNews → todoUS → doingUS → doneUS

This chapter examines all three, then explains why CaTDD's dual-flow architecture — SPEC lifecycle plus UT verification — goes beyond what standalone spec-driven tools can offer.

GitHub Spec Kit — Document-Centric Spec-Driven Development

GitHub Spec Kit (111k stars, [github/spec-kit](#)) is a Python toolkit that structures development around four core documents:

constitution.md → spec.md → plan.md → tasks.md → implement

Architecture

Artifact	Location	What It Holds
Constitution	<code>.specify/memory/constitution.md</code>	Project principles, quality standards, governance rules
Spec	<code>specs/001-feature/spec.md</code>	WHAT & WHY — user stories, functional requirements
Plan	<code>specs/001-feature/plan.md</code>	HOW — tech stack, architecture, implementation decisions
Tasks	<code>specs/001-feature/tasks.md</code>	Executable task list with dependencies and [P] parallel markers

<code>.specify/memory/constitution.md</code>	rules for every decision
<code>specs/001-feature/spec.md</code>	what and why
<code>specs/001-feature/plan.md</code>	how
<code>specs/001-feature/tasks.md</code>	in what order

Command Flow

<code>/speckit.constitution</code>	– Create project governing principles
<code>/speckit.specify</code>	– Define WHAT to build (requirements, user stories)
<code>/speckit.clarify</code>	– Clarify underspecified areas (required before plan)
<code>/speckit.plan</code>	– Define HOW to build (tech stack, architecture)
<code>/speckit.tasks</code>	– Break plan into actionable tasks with dependencies
<code>/speckit.analyze</code>	– Cross-artifact consistency & coverage analysis
<code>/speckit.checklist</code>	– Generate quality checklists ("unit tests for English")
<code>/speckit.implement</code>	– Execute all tasks to build the feature

Key Capabilities

- **Extensions & Presets** — customize or extend the workflow. Extensions add new commands (for example a Jira integration). Presets override templates (compliance formats, domain terminology). Project-local overrides in `.specify/templates/overrides/` take highest priority.
- **Installation** — `uv tool install specify-cli`, then `specify init <project> --integration copilot`
- **Parallel task execution** — tasks marked [P] run concurrently; tasks without [P] respect dependency ordering.

Where Spec Kit Stops

Spec Kit is a **spec-to-tasks pipeline**. It tells you what to build and in what order. It does not tell you how to verify correctness beyond generic checklists.

```

constitution → spec → plan → tasks → implement → code
                                     ↑
                                     └─ the spec's job ends here

```

`/speckit.checklist` generates quality validation items, but there is no embedded testing methodology, no US/AC/TC traceability chain, no TDD discipline, and no verification status tracking. Once `/speckit.implement` runs, the spec does not live on as a verifiable living artifact.

What CaTDD Adopted from Spec Kit

Px-SpecFlow explicitly documents six refinements adopted from Spec Kit, each adapted to the CaTDD method:

Spec Kit Concept	CaTDD Adoption
Constitution governs all decisions	<code>.catdd/spec/projectContext.md</code> as a shared guardrail — <code>SPEC_initProjectContext</code> records stable principles, constraints, and team conventions before story work begins
Prioritized, independently testable user stories	<code>SPEC_analyzeIssue</code> produces stories with actor, value, priority, independent-test intent, acceptance scenarios, edge cases, risks, and open questions — structured for testability from analysis
Clear intent before design	<code>SPEC_clearStoryIntent</code> records a Mutual Intent Contract after <code>SPEC_openUserStory</code> : developer intent, CodeAgent intent, scope, non-goals, success signal, assumptions, open questions
Separate WHAT from HOW	<code>SPEC_makePlan</code> creates a paired <code>*-TASKs.md</code> artifact that classifies work as intent-clearing, requirement-oriented, design-oriented, or implementation-oriented — technical choices land in project-root <code>README*</code> SPEC docs
Clarify/analyze before code	<code>SPEC_reviewArchDesign</code> and <code>SPEC_reviewDetailDesign</code> gate design quality; failed reviews route to <code>SPEC_update*Design</code> before downstream work
Explicit, parallel-aware execution slices	US/AC/TC slices with P0-first ordering, independent work marked for parallel execution

Spec Kit idea		CaTDD home
constitution	→	<code>.catdd/spec/projectContext.md</code>
spec.md (what/why)	→	User Story + Acceptance Criteria
plan.md (how)	→	<code>README*</code> design docs
tasks.md	→	<code>*-UserStory-Tasks.md</code>

CaTDD adopted Spec Kit's structural wisdom and embedded it in a user-story-centered lifecycle with an integrated verification engine.

OpenSpec — Change-Centric Spec-Driven Development

OpenSpec (54k stars, `Fission-AI/OpenSpec`) is a TypeScript tool built on a different philosophy: **fluid not rigid, iterative not waterfall, easy not complex, built for brownfield not just greenfield**.

Architecture

OpenSpec organizes work around **changes** — self-contained folders under `openspec/changes/`. Each change is a proposal for something to build or modify:

```

openspec/changes/add-dark-mode/
├─ proposal.md  - WHY we're doing this, WHAT is changing
├─ specs/       - Requirements and scenarios
├─ design.md    - Technical approach
└─ tasks.md     - Implementation checklist

```

Command Flow

Core workflow (3 commands):

```
/opsx:propose "add-dark-mode"  - Creates proposal + specs + design + tasks in one step
/opsx:apply                    - Implements all tasks
/opsx:archive                  - Moves to archive, updates specs
```

Expanded workflow (6 more commands):

```
/opsx:new          - Create a new change with guiding context
/opsx:continue     - Resume work on the active change
/opsx:ff           - Fast-forward through quick tasks
/opsx:verify       - Verify implementation against specs
/opsx:bulk-archive - Archive multiple completed changes at once
/opsx:onboard      - Walk through the system with guided exploration
```

Key Capabilities

- **Change folder isolation** — each change lives in its own folder, with no cross-contamination between features. Changes are self-contained and independently archivable.
- **Dashboard** — a visual overview of all changes, their status, and their relationships.
- **Brownfield-first** — designed for existing codebases, not just greenfield projects. You propose changes to what already exists.
- **Installation** — `npm install -g @fission-ai/openspec`, then `openspec init`
- **Self-comparison** — "vs Spec Kit — Thorough but heavyweight. Rigid phase gates. OpenSpec is lighter and lets you iterate freely."

Where OpenSpec Stops

OpenSpec is a **change management system** with lightweight spec scaffolding. Each change has a proposal, specs, design, and tasks. Like Spec Kit, it stops at implementation.

```
propose → apply → archive
           ↑
           └ "done" means archived, not verified
```

There is no embedded testing methodology, no US/AC/TC traceability, and no verification lifecycle. The spec describes what to change; it does not define how to prove the change is correct. A change is done when it is archived — there is no RED→GREEN test status driving closure.

CaTDD Px-SpecFlow — User-Story-Centered SpecCoding

Px-SpecFlow is neither document-centric nor change-centric. It is **user-story-centered**. The user story is the central lifecycle artifact, and every SPEC command serves its journey from incoming work to verified, reviewed, committed closure.

The Story Lifecycle

```
pendingNews/ → analyzedNews/ → todoUS/ → doingUS/ → doneUS/
 (raw input)  (input archive) (analyzed) (active)   (completed)
                                     ↳ suspendUS/ (paused, resumable)
                                     ↳ abortUS/  (preserved)
```


Seven lifecycle directories, all version-controlled under `.catdd/spec/`:

State	Directory	Meaning
pendingNews	<code>pendingNews/</code>	Raw issues, features, or imported user stories waiting for analysis
analyzedNews	<code>analyzedNews/</code>	Raw inputs preserved as source trace after analysis
todoUS	<code>todoUS/</code>	Analyzed user stories ready to be opened for work
doingUS	<code>doingUS/</code>	Active user stories under design, test, or implementation
suspendUS	<code>suspendUS/</code>	Suspended active stories preserved with a durable resume reference (branch/worktree)
abortUS	<code>abortUS/</code>	Aborted stories preserved for later re-analysis or improvement
doneUS	<code>doneUS/</code>	Completed stories after review, commit, and CI

The SPEC Command Family

Px-SpecFlow provides **34 SPEC commands** in three lifecycle phases.

PHASE A: pre-story	PHASE B: design & planning	PHASE C: implementation & closure
import, analyze, open →	intent, plan, design, review →	test, code, review, commit, close

Phase A — Pre-Story: Input & Analysis

<code>SPEC_initProjectContext</code>	→ <code>.catdd/spec/projectContext.md</code> (shared constitution)
<code>SPEC_updateProjectContext</code>	→ Update project context
<code>SPEC_importIssue</code>	→ Raw issue → <code>pendingNews/</code>
<code>SPEC_importFeature</code>	→ Raw feature → <code>pendingNews/</code>
<code>SPEC_importUserStory</code>	→ Structured US → <code>todoUS/</code> (skip analysis)
<code>SPEC_analyzeIssue</code>	→ pending input → user story in <code>todoUS/</code> + archive in <code>analyzedNews/</code>
<code>SPEC_analyzeFeature</code>	→ pending input → user story in <code>todoUS/</code> + archive in <code>analyzedNews/</code>
<code>SPEC_analyzeAbortedUserStory</code>	→ Selective re-analysis of an aborted story for a later round
<code>SPEC_openUserStory</code>	→ Move selected story from <code>todoUS/</code> to <code>doingUS/</code> (work begins)

Phase B — Design & Planning

<code>SPEC_clearStoryIntent</code>	→ Align developer + CodeAgent intent (Mutual Intent Contract)
<code>SPEC_makePlan</code>	→ Classify work orientation, create <code>*-UserStory-Tasks.md</code> , pick next command
<code>SPEC_updateUserStory</code>	→ Update module <code>README_UserStory.md</code> + <code>README_UserGuide.md</code>
<code>SPEC_reviewUserStory</code>	→ Gate requirement quality before downstream work
<code>SPEC_whatsNextTask</code>	→ Recommend the single next command from current state
<code>SPEC_takeArchDesign</code>	→ Initial architecture design (<code>README_ArchDesign.md</code> + 7 more)
<code>SPEC_reviewArchDesign</code>	→ Gate architecture quality
<code>SPEC_updateArchDesign</code>	→ Follow-up architecture revision
<code>SPEC_takeDetailDesign</code>	→ Initial detail design (<code>README_DetailDesign.md</code> + <code>README_StateDesign.md</code>)
<code>SPEC_reviewDetailDesign</code>	→ Gate detail design quality
<code>SPEC_updateDetailDesign</code>	→ Follow-up detail design revision

Phase C — Implementation & Closure

<code>SPEC_designUnitTests</code>	→ Enter CaTDD test design (routes to P0/P1/P2 UT flows)
<code>SPEC_implUnitTests</code>	→ Implement test cases (RED→GREEN via UT commands)
<code>SPEC_reviewImplUnitTests</code>	→ Gate unit-test implementation before product code
<code>SPEC_implProductCodes</code>	→ Implement production code to pass tests
<code>SPEC_reviewProductCodes</code>	→ Review implementation quality
<code>SPEC_refactUnitTests</code>	→ No-behavior-change cleanup of one GREEN unit test
<code>SPEC_suspendUserStory</code>	→ Pause active story → suspendUS/ with resume reference
<code>SPEC_resumeUserStory</code>	→ Move suspended story back to doingUS/ and continue
<code>SPEC_partialCloseUserStory</code>	→ Close accepted scope, move rejected scope to abortUS/
<code>SPEC_abortUserStory</code>	→ Abort active story → abortUS/ (preserved for re-analysis)
<code>SPEC_commitWorks</code>	→ Prepare and commit completed work
<code>SPEC_closeUserStory</code>	→ Move reviewed, committed story to doneUS/
<code>SPEC_mergeWorks</code>	→ Merge a closed story branch when integration is still required
<code>SPEC_patchOriginalCaTDD</code>	→ Patch installed-project CaTDD improvements back upstream

What Makes CaTDD Different

Six things in Px-SpecFlow go beyond what Spec Kit or OpenSpec offer.

1. User-story-centered, not document- or change-centered. Spec Kit centers on `spec.md` / `plan.md` / `tasks.md` — documents. OpenSpec centers on `changes/` — proposal folders. CaTDD centers on the **user story itself** as a living artifact moving through seven lifecycle directories. The story has explicit existence from raw import, through analyzed archive, analyzed todo, active doing, possible suspend or abort, to final closure.

2. Mutual Intent Contract. Neither other tool aligns human and LLM intent before design begins. `SPEC_clearStoryIntent` records what the developer thinks the story is about, what the CodeAgent infers, in-scope work, out-of-scope work, the success signal, assumptions, and open questions. Design does not proceed until intent is aligned.

3. `SPEC_abortUserStory` — explicit failure preservation. When an active story has a blocking scope problem, invalid assumptions, or a quality issue that should not be patched in place, CaTDD aborts the story into `abortUS/` as preserved history. That feeds back into `SPEC_analyzeAbortedUserStory` or `SPEC_importIssue` for deliberate re-analysis. Neither Spec Kit nor OpenSpec has an explicit abort-and-preserve path.

other tools:	bad story → delete the folder / close the issue
CaTDD:	bad story → abortUS/ with reasons → re-analysis later

4. `SPEC_makePlan` — work orientation classification. Before any downstream work, `SPEC_makePlan` classifies the story as intent-clearing, requirement-oriented, design-oriented, or implementation-oriented. It distinguishes initial design (`SPEC_take*Design`) from follow-up revision (`SPEC_update*Design`), and it creates a paired `*-UserStory-Tasks.md` artifact with Markdown checkbox tasks showing the next required steps.

5. Project-root README SPEC docs. CaTDD manages 13 project-root README SPEC document types, grouped into architecture-oriented, detail-oriented, and general/requirements docs, each with a purpose and an owner:

Architecture-Oriented	Detail-Oriented
<code>README_ArchDesign.md</code> — Module decomposition, dependencies, trade-offs	<code>README_DetailDesign.md</code> — Class design, API signatures
<code>README_UsageDesign.md</code> — Public boundaries, CLI/API contracts	<code>README_StateDesign.md</code> — State machines, lifecycle, concurrency
<code>README_ErrorDesign.md</code> — Fault-tolerance, fail-safe states	

Architecture-Oriented	Detail-Oriented
README_ResourceDesign.md — Resource allocations, memory/CPU budgets	
README_PerfDesign.md — Performance budgets, latency limits	
README_CompatDesign.md — Compatibility matrices, platform versions	
README_DiagnosisDesign.md — Observability, logging, telemetry	
README_VerifyDesign.md — Verification topology, test strategy	

Beyond those two groups, the general/requirements group holds README.md (project overview and master SPEC directories), README_UserStories.md (the mandatory project-level TODO/DONE story ledger with AC trace status), and README_UserGuide.md (user-facing usage guidance). The developer creates these first; SPEC_updateUserStory and SPEC_reviewUserStory update them later.

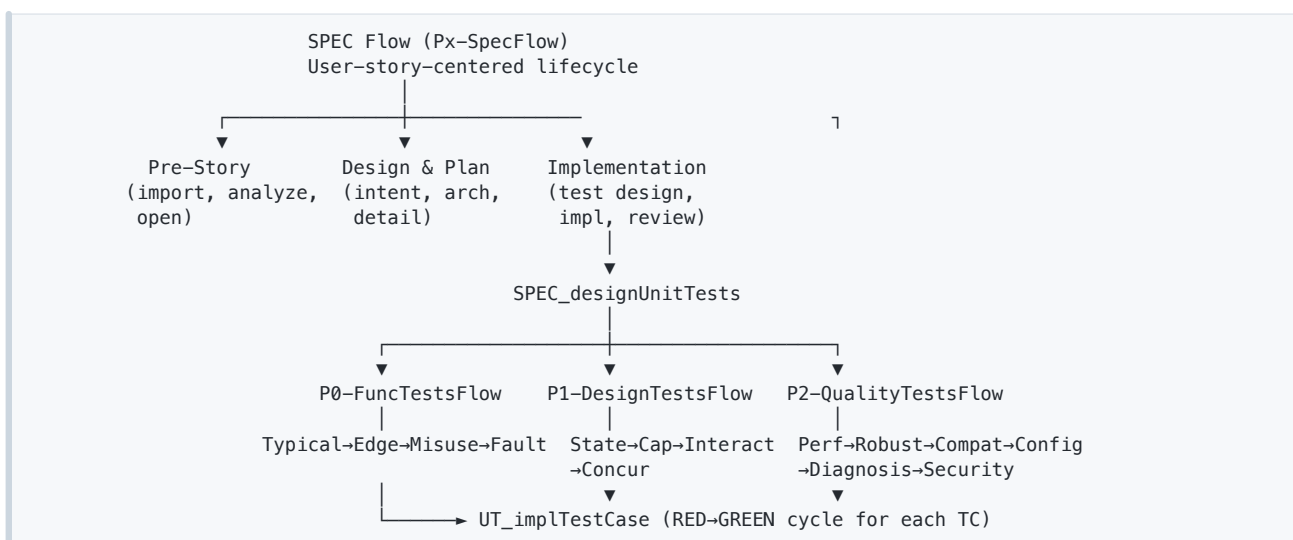
These are not templates dumped at initialization. SPEC commands create them on demand, only when the project needs that surface.

6. Model tier guidance per command. Every SPEC command carries a model tier — SOTA reasoning for architecture decisions, High Performance for multi-artifact reasoning and review, Flash Speed for deterministic lifecycle movements. That prevents overpaying for simple tasks and under-investing in architecture-significant ones.

The UT Flow — CaTDD's Embedded Verification Engine

This is the defining differentiator. **Neither Spec Kit nor OpenSpec has a testing methodology.** They describe what to build and in what order, and they stop at implementation. CaTDD embeds a complete category-driven TDD methodology inside the spec lifecycle.

The Dual-Flow Architecture



When SPEC_designUnitTests runs, it does not produce a generic "write tests" task. It routes into the UT

flows: P0 for functional verification, P1 for design-oriented tests, P2 for quality attributes. Each flow has its own command sequence, review gates, and Design Source Gate requirements.

Interaction (P1) and Diagnosis (P2) currently have no dedicated `UT_design*Skeleton` commands. Their flows route directly through the matching `CaTDD_methodPrompt4Cat-*.md` method prompts, drafting source-linked US/AC/TC into the canonical category files under the same Discovery Gate and review contract. Security (P2) does have one — `UT_designSecuritySkeleton`, gated on project-root `README_SecurityDesign.md` and constitutional invariants (\$K\$). Beside these lifecycle flows, `slashCommands` also carries an operational `HARNESS_*` family (`Px-HarnessKits`) for harness maintenance — and it never moves SpecFlow lifecycle state.

What the UT Flow Provides That No Other Spec Tool Does

1. Category-driven test design. 15 test categories, each with its own method prompt (`CaTDD_methodPrompt4Cat-*.md`) defining its position, use-when/avoid-when routing rules, test-point probes, a design skeleton contract, and a checklist; most prompts also carry naming examples and common-mistake guidance. A CodeAgent designing a Typical skeleton reads `CaTDD_methodPrompt4Cat-Typical.md` and knows exactly which patterns, vocabulary, and constraints apply.

2. US/AC/TC traceability embedded in the test file. The specification does not live in a separate document. User Story, Acceptance Criteria, and Test Case specifications live as structured comments in the same file as the test code. When the spec changes, the comment changes. When the comment changes, the LLM regenerates the test.

```
other tools: spec.md —(hope)—> code
CaTDD:      US/AC/TC comments + TEST code in one file, adjacent and in sync
```

3. Design Source Gate for P1 and P2. Before any P1 or P2 skeleton can be drafted, the required project-root design document must exist:

P1 Category	Required Source
State	<code>README_StateDesign.md</code> or State Design chapter in <code>README_ArchDesign.md</code>
Capability	<code>README_DetailDesign.md</code>
Interaction	Sequence, interaction, or collaboration sections in <code>README_ArchDesign.md</code>
Concurrency	<code>README_ResourceDesign.md</code>
P2 Category	Required Source
Performance	<code>README_PerfDesign.md</code>
Robust	<code>README_ErrorDesign.md</code>
Compatibility	<code>README_CompatDesign.md</code>
Configuration	<code>README_DetailDesign.md</code>
Diagnosis	<code>README_DiagnosisDesign.md</code> / <code>README_VerifyDesign.md</code> evidence requirements
Security	<code>README_SecurityDesign.md</code> (+ threat model, security policy, or constitutional invariants (\$K\$))

If the design source is missing, the UT command — or the direct method-prompt route — **stops and asks the developer**. No guessing, no inventing architectural decisions inside a test file.

4. TDD RED→GREEN discipline integrated into the lifecycle. Every test case has a status marker:

○ TODO/PLANNED → ● RED/FAILING → ● GREEN/PASSED

`SPEC_implUnitTests` invokes `UT_implTestCase`, which follows the strict cycle: write the test (RED), implement minimal production code (GREEN), review (TRACE). The story cannot close until all required TCs are GREEN.

5. Quality gates at every level. You do not advance from P0 to P1 until all P0 tests are GREEN. You do not advance from P1 to P2 until the architecture is validated: no deadlocks, no race conditions, ThreadSanitizer clean. Each gate is a checkpoint where the developer reviews and approves before the work proceeds.

Three-Way Comparison

Dimension	GitHub Spec Kit	OpenSpec	CaTDD Px-SpecFlow
Center of gravity	Documents (<code>spec.md</code> , <code>plan.md</code>)	Changes (<code>openspec/changes/</code>)	User Stories (lifecycle states)
Lifecycle states	3 implicit (<code>spec</code> → <code>plan</code> → <code>tasks</code>)	3 explicit (<code>propose</code> → <code>apply</code> → <code>archive</code>)	7 explicit (<code>pending</code> → <code>analyzed</code> → <code>todo</code> → <code>doing</code> → <code>suspend/abort</code> → <code>done</code>)
Abort mechanism	None (close issue/PR manually)	None (delete change folder manually)	SPEC_abortUserStory — preserved in <code>abortUS/</code> with a re-analysis path
Intent alignment	Free-form clarification chat	Free-form conversation	Mutual Intent Contract (<code>SPEC_clearStoryIntent</code>)
Testing methodology	None embedded (checklist only)	None embedded	Embedded CaTDD — P0/P1/P2 UT flows with 15 categories
Spec-code gap	<code>spec.md</code> → <code>plan.md</code> → <code>code</code> (3 documents)	<code>proposal.md</code> → <code>design.md</code> → <code>code</code> (3 documents)	No gap — US/AC/TC comments live in the same file as the test code
TDD integration	Not present	Not present	RED→GREEN cycle with ○→●→● status markers
Comment-alive design	No — docs separate from code	No — docs separate from code	Yes — the design skeleton lives in the test file
Priority framework	None	None	P0(core) → P1(design) → P2(quality) → P3(addons)
Design source gate	None	None	Yes — P1/P2 require confirmed <code>README_*</code> docs
Model tier guidance	Not present	Recommends high-reasoning models	Per-command tier mapping (SOTA / HighPerf / Flash)
Review gates	<code>/speckit.analyze</code> (cross-artifact)	<code>/opsx:verify</code> (post-implementation)	Arch → Detail → Story → Test → Code → Commit (6 review gates)
Artifact persistence	File system (specs directory)	File system (changes directory)	Team-shared <code>.catdd/spec/</code> with a commit policy per artifact
Project-root docs	Spec + plan, then manual	Proposal + design, then manual	13 README* SPEC doc types created on demand by SPEC commands

Dimension	GitHub Spec Kit	OpenSpec	CaTDD Px-SpecFlow
Brownfield	Primarily greenfield	Brownfield-first (designed for existing codebases)	Both — analyze existing code into US/AC/TC
Install method	<code>uv tool install specify-cli</code> (Python)	<code>npm install -g @fission-ai/openspec</code> (TypeScript)	Markdown prompt files (agent-agnostic)
Agent integrations	30+	25+	Copilot, Cline, Continue, utCodeAgentCLI
Customization	Extensions + presets + overrides	Community schemas	<code>methodPrompts/</code> as source of truth, <code>slashCommands/</code> as command layer, <code>agentSkills/</code> as packages

Why CaTDD Is BEYOND

Spec Kit and OpenSpec are valuable tools. They solve the spec-first problem — but only half of it.

Spec Kit gives you a structured pipeline: constitution → spec → plan → tasks → implement. It tells you what to build, in what order, with what technology. When implementation ends, you have code — without systematic proof that it is correct beyond a checklist.

OpenSpec gives you fluid change management: propose → apply → archive. It tells you what changed, what the design is, and what tasks to run. When the change is archived, you have completed work — without embedded verification that it satisfies its own acceptance criteria.

One bug, three tools

Here is the same defect seen through each tool. QA reports: "IOC_post returns success on a destroyed handle."

Spec Kit	→ tells you which document the fix belongs to, and which task to add to tasks.md
OpenSpec	→ tells you which change folder to create, and what the design says the behavior should be
CaTDD	→ tells you which story this is, which AC it violates, which TC proves the violation, and which gate the fix must pass before the story can close

The first two answers are about **planning the change**. The third is about **proving the change** — and the proof survives the conversation, because it lives in the test file as US/AC/TC comments with a RED→GREEN history.

What CaTDD adds

1. **It manages user stories through their full lifecycle**, from raw import to verified closure, with an explicit abort path for when scope or assumptions turn out wrong.
2. **It aligns human and LLM intent before design begins**, preventing the most expensive mistake in LLM-driven development: building the wrong thing perfectly.
3. **It embeds a complete testing methodology (UT P0/P1/P2)** in the spec lifecycle. Every story's

acceptance criteria are verified by category-driven test cases. Every test case drives a RED→GREEN cycle. Every GREEN test proves one piece of the story is correct.

4. **It eliminates the spec-code gap.** The specification lives as structured comments (`@[US]`, `@[AC]`, `@[TC]`) in the same file as the test code. When the code changes, the comment changes; when the comment changes, the test changes. Spec and verification are inseparable.
5. **It classifies verification by domain significance.** P0 tests your Core Domain — what makes the business unique. P1 tests your architecture. P2 tests your quality attributes. That is DDD's strategic design applied to verification: the LLM is told not only test this, but test this because it is core to the business.

The Spec-Driven Stack

Think of the three tools as layers of one stack:

CaTDD Px-SpecFlow + UT flows "Prove it's correct" user-story lifecycle + embedded TDD
Spec Kit / OpenSpec "Decide what to build and how" document-centric or change-centric
AI Coding Agents (Copilot, Cline, Claude, ...) "Generate the code"

Spec Kit and OpenSpec operate in the middle layer: they structure what to build. CaTDD operates at the top layer: it verifies that what was built is correct, and keeps the evidence in the artifact itself. They are not competitors — they are complementary.

But if you can only have one, and the choice is between knowing what to build and knowing that what you built is correct, CaTDD provides the verification guarantee that no spec-first tool alone can give you.

Comments is Verification Design. LLM Generates Code. Iterate Forward Together.